Python Programming Hands-On

Python Programming Hands-On

Learn Python from Basic to Advanced by Writing Real Programs, with Exercises, Solutions and Every Example Tested

Aadhavan Muthurengan

Python Programming Hands-On
Learn Python from Basic to Advanced by Writing Real Programs, with Exercises, Solutions and Every Example Tested

Copyright © 2026 Aadhavan Muthurengan. All rights reserved.

No part of this publication may be reproduced, distributed, or transmitted in any form or by any means without the prior written permission of the author, except for brief quotations in reviews and certain other noncommercial uses permitted by copyright law. Readers may freely use and adapt the example programs, exercises and project code in this book for their own work. All the code is available at github.com/aadhavanshines/python-programming-hands-on under the MIT License.

Disclaimer. This book is provided for educational and informational purposes only. AI products, features, and policies change frequently; verify current details with each provider. The author makes no guarantees about results and is not liable for any losses arising from the use of this information. Nothing in this book is legal, medical, financial, or other professional advice.

Trademarks. Python and the Python logos are trademarks or registered trademarks of the Python Software Foundation. Visual Studio Code and Windows are trademarks of Microsoft Corporation. Apple and macOS are trademarks of Apple Inc. Linux is a trademark of Linus Torvalds. GitHub is a trademark of GitHub, Inc. SQLite is in the public domain; pandas, matplotlib, pytest, mypy, FastAPI, httpx and uv are open-source projects and their names belong to their owners. All people, books, data and figures in the example programs are fictional. Product names are used solely for identification. This book is independent and is not affiliated with, sponsored by, or endorsed by the Python Software Foundation or any of these companies.

Examples. All people, books, sales figures and other data in the example programs are fictional, created for this book. Any resemblance to real persons or businesses is coincidental.

First edition, 2026.

To my father and my mother,
for your love, your sacrifices, and every lesson you taught me.

To my wife,
for your patience, your encouragement, and your belief in me.

And to our three children,
may you always build the things you dream of.

This book is for you.

Contents

Introduction 11

Part I: Getting Started 17

1. Welcome to Python 19
2. Variables, Numbers and Strings 25
3. Input, Output and Conversions 33
4. Making Decisions 39
5. Loops 45

Part II: Organizing Data and Code 51

6. Lists and Tuples 53
7. Dictionaries and Sets 61
8. Functions 69
9. Working with Text 79
10. Files and Exceptions 87
11. Modules and the Standard Library 97

Part III: Intermediate Python 105

12. Classes and Objects 107
13. Inheritance, Dataclasses and Special Methods 115
14. Comprehensions, Iterators and Generators 125
15. Testing with pytest 133

16. Debugging, Logging and Reading Tracebacks 141

17. Virtual Environments, Packages and Project Layout 149

Part IV: Advanced Python **159**

18. Decorators and Context Managers 161

19. Type Hints and Static Checking 169

20. Functional Tools and Pattern Matching 175

21. Concurrency: Threads, Processes and asyncio 183

22. Performance: Measuring, Profiling and Choosing Data Structures . . 193

Part V: Real-World Projects **201**

23. Project: Pocket Library with SQLite 203

24. Project: Data Analysis with CSV, pandas and Charts 215

25. Project: Web APIs: Calling Them and Building One 223

26. Project: Automating Everyday Tasks 235

27. Where to Go Next 247

Appendix A: Solutions to the Exercises **251**

Appendix B: Python Cheat Sheet **277**

Appendix C: Common Errors and How to Fix Them **283**

Appendix D: Glossary **287**

Appendix E: Resources **291**

About the Author **293**

Introduction

Python is one of the most widely used programming languages in the world. Scientists use it to analyze data, engineers to automate tests, web developers to build services, teachers to introduce programming, and many people simply to save themselves an afternoon of clicking. It reads almost like English, it runs on every common computer, and it is free.

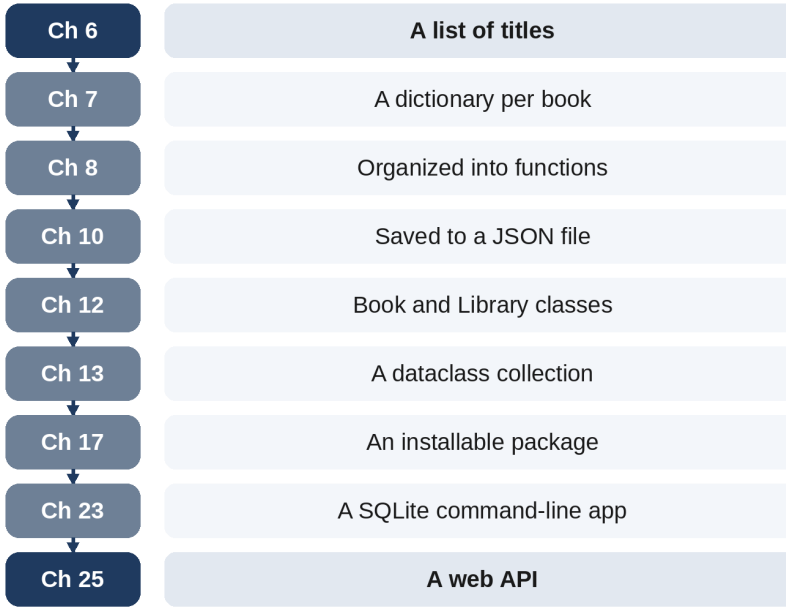
This book teaches you Python by having you write it. You'll start with one line that prints a greeting and finish with tested, packaged programs: a command-line app with a database, a data analysis with charts, a web API, and scripts that tidy your folders and back up your files. Along the way you'll learn the parts of the language that working programmers use every day, and how to read the error messages that every programmer sees every day.

What Makes This Book Different

Every example is tested. Every program in this book is a real file in the book's code repository on GitHub. Before publication, each one was run on Python 3.12, 3.13, 3.14 and 3.15, and its output was compared with the output printed in the book. If you type a program exactly as printed, you'll see exactly the output printed below it. Where output can't be identical (timings, for example), the book says so.

You learn by doing. Each chapter introduces a few ideas, shows them in short programs, and asks you to try them. Every chapter ends with exercises, and every exercise has a worked solution in Appendix A. The solutions are tested too.

One project grows with you. Pocket Library is a small program for keeping track of the books you own and read. It starts in Chapter 6 as a list in memory. It learns to save to a file in Chapter 10, becomes a set of classes in Chapter 12, a tested and installable package in Chapter 17, and a command-line app with a SQLite database in Chapter 23. In Chapter 25 it gets a web API. You'll see how a real program evolves, and why each new tool is worth learning.



Pocket Library grows through the book: a list of titles in Chapter 6, a dictionary per book in Chapter 7, functions in Chapter 8, a JSON file in Chapter 10, classes in Chapter 12, a dataclass collection in Chapter 13, an installable package in Chapter 17, a SQLite command-line app in Chapter 23 and a web API in Chapter 25.

Errors are part of the lesson. You'll make mistakes on purpose: a missing quote, a misspelled name, text added to a number. You'll see the exact message Python prints and learn to read it. Appendix C collects the most common errors and their fixes.

Who This Book Is For

- **Complete beginners.** You don't need any programming experience. You need a computer (Windows, macOS or Linux), the ability to install a program, and some patience.
- **People who have tried before.** If you've started a course or tutorial and stalled, the short steps and the tested examples here are designed to keep you moving.
- **Programmers who know another language.** Skim Part I, then use Parts II to IV to learn how things are done in Python.
- **Python users who want to go further.** Parts III to V cover classes, testing, packaging, type hints, decorators, concurrency, performance, databases, data analysis and web APIs.

How the Book Is Organized

Part	Chapters	What you'll learn
I. Getting Started	1 to 5	Running Python, values and variables, input and output, decisions and loops
II. Organizing Data and Code	6 to 11	Lists, dictionaries, functions, text, files, errors and modules
III. Intermediate Python	12 to 17	Classes, iteration, testing, debugging, packaging
IV. Advanced Python	18 to 22	Decorators, type hints, functional tools, concurrency, performance
V. Real-World Projects	23 to 27	A database app, data analysis, web APIs, automation, and what to learn next

Each chapter opening shows its level: **Beginner**, **Intermediate** or **Advanced**; the projects in Part V are marked **Project**. Read Parts I and II in order. After that, chapters mostly stand on their own, although the projects in Part V use ideas from Parts III and IV.

How to Read the Code

Programs look like this:

```
print("Hello, Python!")
```

Text the program prints, its **output**, follows it:

```
Hello, Python!
```

When a program asks you to type something, what you type appears in the output too, just as it does on your screen. Short sessions at Python's interactive prompt start with `>>>`. You type the text after the `>>>` and Python answers on the next line:

```
>>> 2 + 3
5
```

Commands you type in a terminal (not in Python) start with `$`, which you don't type:

```
$ python hello.py
```

Getting the Code

All the programs, data files, exercise solutions and tests are free to download from the book's code repository on GitHub:

github.com/aadhavanshines/python-programming-hands-on

On that page, click the green **Code** button and choose **Download ZIP**, or, if you use Git, clone the repository. The code is arranged by chapter: `code/ch01`, `code/ch02` and so on, each with a short README. A file named in the text, such as `variables.py`, is in the folder for that chapter, together with its expected output in a file such as `variables.out`. The repository also checks every program automatically on Python 3.12, 3.13, 3.14 and 3.15, and shows the results publicly. You'll learn more if you type the short programs yourself, at least in the early chapters. Typing mistakes are how you learn to read error messages.

> **Tip:** When an exercise feels hard, try it for fifteen minutes before you look at the solution. Then compare your answer with the book's. There is usually more than one correct answer.

Versions

The book uses Python 3.12 or newer and was tested on Python 3.12, 3.13, 3.14 and 3.15, the newest version at the time of writing. Everything in Parts I to IV uses only the standard library, the tools that come with Python. Parts III to V also use a few widely used packages: pytest, mypy, pandas, matplotlib, FastAPI, httpx and uvicorn. Chapter 17 shows how to install them, and the code repository's `requirements.txt` lists the exact versions the book was tested with.

Let's start.

PART I

Getting Started

CHAPTER 1 · BEGINNER

Welcome to Python

A program is a list of instructions for a computer, written in a language the computer can follow. Python is one such language. You write instructions in a text file, and a program called the **Python interpreter** reads the file and carries out the instructions, one after another, from top to bottom. In this chapter you'll install Python, talk to it directly, write your first program, and see your first error message.

Installing Python

You need Python 3.12 or newer. Many computers already have some version of Python, but it's often old or used by the system, so install your own from the official site, **python.org**.

Windows. Go to python.org, open Downloads, and install the **Python install manager**. It's also available in the Microsoft Store. The install manager installs the latest Python and keeps it up to date, and it gives you two commands: `python` runs Python, and `py` manages the versions you have installed. It needs Windows 10 or newer.

macOS. Go to python.org, open Downloads, and download the macOS installer for the latest Python 3 release. Run it and follow the steps. On a Mac, the command is `python3`, not `python`. Wherever this book says `python`, type `python3`.

Linux. Most distributions already include a recent Python 3, usually as `python3`. If yours is older than 3.12, install a newer one with your package manager, or with the `uv` tool described in Chapter 17.

To check that Python is installed, open a **terminal**: on Windows, open the Start menu and type *Terminal*; on macOS, open *Terminal* from Applications, Utilities; on Linux, it's usually called *Terminal* too. Then type:

```
$ python --version
```

Python answers with its version, such as `Python 3.15.0`. Any version from 3.12 onwards works for this book. If you see an error such as *command not found*, close the terminal, open a new one, and try again; on macOS, try `python3 --version`.

> **Note:** The terminal is a window where you type commands instead of clicking. You'll use only a handful of commands in this book: `python` to run a program, `cd` to move to a folder, and occasionally `pip` to install a package. Each is explained when you first need it.

Talking to Python

Type `python` in the terminal and press Enter. You'll see a few lines about the version and then a prompt: `>>>`. This is Python's **interactive shell**, sometimes called the REPL (read, evaluate, print, loop). Type something after the prompt, press Enter, and Python answers at once:

```
>>> 2 + 3
5
>>> 10 - 4 * 2
2
>>> 7 / 2
3.5
>>> print("Hello from the Python shell")
Hello from the Python shell
>>> "ha" * 3
'hahaha'
```

Python works as a calculator, following the usual rules: multiplication before subtraction, so `10 - 4 * 2` is 2. `print` displays text. The quotes mark where the text starts and ends; Python calls a piece of text a **string**. And `"ha" * 3`

repeats a string three times. You'll learn what each piece means in the next chapter. For now, notice that you typed something and Python did it.

To leave the shell, type `exit()` and press Enter.

The shell is perfect for trying out an idea in a few seconds. Everything you type there is lost when you close it, though, so real programs go in files.

Your First Program

A Python program is a plain text file whose name ends in `.py`. You can write one in any text editor, but an editor made for code helps: it colors the different parts of your program, indents lines for you, and runs the program with a key press. Two good choices:

- **IDLE** comes with Python. On Windows and macOS you'll find it with your other programs. Choose File, New File to write a program, and Run, Run Module (or press F5) to run it.
- **Visual Studio Code** is a free editor used by many professionals. Install it from code.visualstudio.com and add the Python extension when it suggests it.

Make a folder for your work, for example `python-practice` in your Documents folder. Create a file there called `hello.py` with one line:

```
print("Hello, Python!")
```

Save it, then run it. In IDLE, press F5. In a terminal, go to the folder with `cd` and run the file with `python`:

```
$ cd Documents/python-practice
$ python hello.py
```

Python prints:

```
Hello, Python!
```

Congratulations: you've written and run a program. `print` is a **function**, a named piece of code that does a job. You **call** it by writing its name followed

by parentheses, with what it needs, its **arguments**, inside them. Here the argument is the string `"Hello, Python!"`.

A Program with More Lines

Python runs the lines of a file in order, from top to bottom. Save this as `first_program.py`:

```
# My first real program: a few lines that Python runs from top to bottom.
print("Welcome to Python!")
print("2 + 3 =", 2 + 3)
print("Python" * 3)
print()  # an empty line
print("Done.")
```

Output:

```
Welcome to Python!
2 + 3 = 5
PythonPythonPython

Done.
```

A few things to notice:

- `print` can take several arguments separated by commas. It prints them with a space between them, so `print("2 + 3 =", 2 + 3)` prints the text, a space, and the result of the sum.
- Python works out `2 + 3` before printing it. The text `"2 + 3 ="` is printed exactly as written, because it's inside quotes.
- `print()` with no arguments prints an empty line.
- Everything after a `#` is a **comment**. Python ignores it. Comments are notes for the people who read your code, including you in six months.

Your First Error

Everyone who writes programs makes mistakes, and Python tells you about them in **error messages**. Learning to read them calmly is one of the most valuable skills in this book. Here is `hello.py` with the closing quote missing:

```
print("Hello, Python!)
```

Run it, and Python refuses:

```
  File "typo.py", line 1
    print("Hello, Python!)
          ^
SyntaxError: unterminated string literal (detected at line 1)
```

Read error messages from the bottom up. The last line says what kind of error it is, a `SyntaxError`, which means Python couldn't understand the code, and why: the string was never closed. The lines above say where: the file, the line number, the line itself, and a caret (^) pointing at the place where the problem starts. Add the missing quote, save, and run it again.

> **Tip:** When you get an error, check the line it points to, and the line before it. A missing bracket or quote on one line often shows up as an error on the next.

The exact wording and the markers under the line vary a little between Python versions. Newer versions give more helpful messages, often with a suggestion such as *Did you mean ...?*. That's one reason to use a recent Python.

How Python Runs Your Program

When you type `python hello.py`, the interpreter reads the whole file, checks that it's valid Python, and then runs it one line at a time. That's why a syntax error stops the program before anything is printed, even if the error is on the last line. Other errors, which you'll meet in the next chapter, happen only when Python reaches the line that causes them. Lines before the error run normally.

Try It

1. Change `hello.py` so that it greets you by name. Then add a second `print` line that says what you'd like to build with Python.
2. In the shell, work out how many minutes there are in a week, and how many hours in a year (use 365 days).
3. Make your own syntax errors: remove a parenthesis from `first_program.py`, then misspell `print` as `pritn`. Read each message from the bottom up. What is the error type in each case?

Exercises

1. **About me.** Write a program `about_me.py` that prints your name, a sentence about why you're learning Python, and the number of hours in a year, calculated by Python, not by you.
2. **Calculator.** Use the shell to calculate the number of minutes in a week and the number of hours in a year.
3. **Cheering.** In the shell, use `print` and the `*` operator on strings to print `Hip, hip, Hip, hip, hooray!` with one line of code.

Key Takeaways

- Install Python 3.12 or newer from python.org. Check it with `python --version` (`python3` on a Mac).
- The interactive shell (`>>>`) is for trying things quickly; programs go in `.py` files, which you run with `python name.py`.
- Python runs a program from top to bottom. `print` displays values; `#` starts a comment.
- Read error messages from the bottom up: the type of error, the reason, then the line and position.

CHAPTER 2 · BEGINNER

Variables, Numbers and Strings

Programs work with values: a price, a name, a count, a temperature. In this chapter you'll learn to give values names, so you can use them again and change them, and you'll meet the three kinds of value you'll use most: whole numbers, decimal numbers and text. You'll also meet the two errors beginners see most often, and learn what they mean.

Variables

A **variable** is a name that refers to a value. You create one with `=`, the **assignment** operator: the name on the left, the value on the right.

```
name = "Meera"
age = 29
height_m = 1.64

print(name)
print(age)
print(height_m)

age = age + 1  # a variable can change: Meera has a birthday
print(name, "is now", age)
```

Output:

```
Meera
29
1.64
Meera is now 30
```

Read `=` as "now refers to", not "equals". The line `age = age + 1` would make no sense in mathematics, but in Python it means: take the current value of `age`, add 1, and make `age` refer to the result. Python always works out the right-hand side first.

Variable names can contain letters, digits and underscores, but can't start with a digit. They are case-sensitive: `age`, `Age` and `AGE` are three different names. By convention, Python programmers write variable names in lower case with underscores between words, like `height_m` or `total_price`. Choose names that say what the value means. `height_m` tells the reader that the height is in meters; `h` tells them nothing.

A few words are reserved by Python and can't be used as names, such as `if`, `for`, `class` and `True`. You'll meet most of them in the next chapters.

Numbers

Python has two main kinds of number. An **integer** (`int`) is a whole number: `42`, `-7`, `0`. A **floating-point number** (`float`) has a decimal point: `3.14`, `1.64`, `-0.5`. Every value has a **type**, and the `type` function tells you what it is:

```
>>> type(42)
<class 'int'>
>>> type(3.14)
<class 'float'>
>>> type("hello")
<class 'str'>
>>> 0.1 + 0.2
0.30000000000000004
>>> round(0.1 + 0.2, 2)
0.3
>>> 2 ** 100
1267650600228229401496703205376
>>> 1_000_000 * 3
3000000
>>> divmod(17, 5)
(3, 2)
```

Three things in this session surprise many beginners:

- `0.1 + 0.2` is not exactly `0.3`. Computers store floats in binary, and most decimal fractions can't be stored exactly, just as 1/3 can't be written exactly in decimal. The error is tiny, and `round` hides it when you display a result. For money, where every cent matters, Python has a `decimal` module, which you'll meet in Chapter 11.
- Integers can be as large as you like: `2 ** 100` is calculated exactly.
- You can write underscores in long numbers to make them readable: `1_000_000` is one million.

Here are the arithmetic operators:

```
a = 17
b = 5

print("a + b  =", a + b)
print("a - b  =", a - b)
print("a * b  =", a * b)
print("a / b  =", a / b)  # true division: always a float
print("a // b =", a // b)  # floor division: whole part only
print("a % b  =", a % b)  # remainder
print("a ** 2 =", a**2)  # power
print("round(a / b) =", round(a / b))
```

Output:

```
a + b  = 22
a - b  = 12
a * b  = 85
a / b  = 3.4
a // b = 3
a % b  = 2
a ** 2 = 289
round(a / b) = 3
```

`/` always gives a float, even when the result is whole (`10 / 2` is `5.0`). `//` divides and keeps only the whole part, and `%`, the **remainder** or **modulo** operator, gives what's left over: 17 divided by 5 is 3 with 2 left over. These two are more useful than they look. `n % 2` is 0 for even numbers and 1 for odd ones, and `//` and `%` together split seconds into minutes and seconds, as you'll do in the exercises. The `divmod` function in the session gives both at once.

Python follows the usual order of operations: `**` first, then `*`, `/`, `//` and `%`, then `+` and `-`. Use parentheses when you want a different order, or simply to make your intention clear: `(price + tax) * quantity`.

Strings

A **string** (`str`) is a piece of text. You can write it in single or double quotes, `'hello'` or `"hello"`; they mean the same. Use the other kind when the text contains a quote: `"It's raining"`.

```
first = "Ada"
last = "Lovelace"
born = 1815

full = first + " " + last
print(full)
print(len(full), "characters")
print(full.upper())
print(full.lower())
print("First letter:", full[0])
print("Last letter:", full[-1])
print(f"{first} was born in {born}.")  # an f-string: values go in {}
```

Output:

```
Ada Lovelace
12 characters
ADA LOVELACE
ada lovelace
First letter: A
Last letter: e
Ada was born in 1815.
```

- `+` joins strings together, which is called **concatenation**. Notice the `" "` in the middle; without it you'd get `AdaLovelace`.
- `len` gives the length of a string: the number of characters, including spaces.
- `upper` and `lower` are **methods**: functions that belong to a value. You call a method with a dot after the value: `full.upper()`. Methods never change the original string; they give you a new one. Chapter 9 covers many more.
- Square brackets pick out one character. Positions, called **indexes**, start at 0, so `full[0]` is the first character. Negative indexes count from the end: `full[-1]` is the last.

The last line is an **f-string**, a string with an `f` before the opening quote. Inside it, anything in curly braces is replaced by its value. f-strings are the easiest way to build text from values, and you'll use them constantly. Any expression can go inside the braces, such as `{age + 1}` or `{name.upper()}`.

> **Tip:** The shell is the best place to experiment with strings. Try `"python"[1:4]`, `"=" * 20` and `len("")` and see what you get. Chapter 9 explains slices like `[1:4]`.

Two Errors You'll See Often

If you use a variable that doesn't exist, Python raises a `NameError`. Usually it's a typo:

```
price = 120
print("Total:", prices)
```

```
Traceback (most recent call last):
  File "name_error.py", line 2, in <module>
    print("Total:", prices)
                    ^^^^^^
NameError: name 'prices' is not defined. Did you mean: 'price'?
```

The message tells you the line, points at the name, and even suggests the name you probably meant. Unlike a syntax error, this error happens only when Python reaches the line: if line 1 printed something, you'd see that output first.

The other common error comes from mixing types:

```
age = 30
print("Age: " + age)
```

```
Traceback (most recent call last):
  File "type_error.py", line 2, in <module>
    print("Age: " + age)
          ~~~~~~~~^~~~~
TypeError: can only concatenate str (not "int") to str
```

Python won't guess whether you meant to add numbers or join text, so it refuses to add a string and an integer. Fix it by converting the number to a

string with `str(age)`, by passing two arguments to `print` (`print("Age:", age)`), or, best of all, with an f-string: `print(f"Age: {age}")`.

> **Note:** A `TypeError` means you used a value of the wrong type for an operation. A `NameError` means Python doesn't know a name. You'll meet a third common error, `ValueError`, in the next chapter.

Try It

1. In the shell, assign your age to a variable and print how many days old you are, roughly (ignore leap years).
2. What does `7 // 2` give? And `-7 // 2`? Look up why in the shell by trying `-7 / 2` first.
3. Make a string variable `word` and print it backwards with `word[::-1]`. (You'll learn how this works in Chapter 9.)

Exercises

1. **Rectangle.** Store the width (7.5) and height (4) of a rectangle in variables and print its area and perimeter.
2. **Seconds.** Convert 10,000 seconds into hours, minutes and seconds using `//` and `%`, and print `10000 seconds is 2 h 46 min 40 s`.
3. **Initials.** Given `first = "grace"` and `last = "hopper"`, print the initials in capitals with dots: `G.H.`.

Key Takeaways

- A variable is a name for a value, created and changed with `=`. Python works out the right-hand side first.
- `int` is for whole numbers and `float` for decimals; `/` always gives a float, `//` and `%` give the whole part and the remainder.
- Strings are text in quotes. Join them with `+`, measure them with `len`, index them from 0, and build them with f-strings.

- A `NameError` means an unknown name (usually a typo); a `TypeError` means a value of the wrong type for the operation.

CHAPTER 3 · BEGINNER

Input, Output and Conversions

So far your programs have done the same thing every time. Real programs respond to the people who use them. In this chapter you'll ask the user for input, convert what they type into numbers, and format the results neatly. At the end you'll build a tip calculator that splits a restaurant bill.

Asking for Input

The `input` function shows a message, called a **prompt**, waits for the user to type something and press Enter, and gives you what they typed as a string:

```
name = input("What is your name? ")
city = input("Where do you live? ")
print(f"Hello, {name}! I hear {city} is lovely.")
```

When you run it, the program waits after each question. Here's a run where the user typed `Ravi` and `Pune`:

```
What is your name? Ravi
Where do you live? Pune
Hello, Ravi! I hear Pune is lovely.
```

Notice the space at the end of each prompt, inside the quotes. Without it, the user's answer would be squashed against the question mark.

Input Is Always Text

Here's a program that tries to add two numbers:

```
a = input("First number: ")
b = input("Second number: ")
print("Sum:", a + b)  # input() always returns text!
```

```
First number: 12
Second number: 30
Sum: 1230
```

12 plus 30 is not 1230. The problem is that `input` always returns a string, even when the user types digits. `a` is the string `"12"` and `b` is `"30"`, and `+` on two strings joins them. To do arithmetic, convert the strings to numbers first:

```
a = int(input("First number: "))
b = int(input("Second number: "))
print("Sum:", a + b)
```

```
First number: 12
Second number: 30
Sum: 42
```

`int(...)` converts a string to an integer. The line `a = int(input("First number: "))` works from the inside out: `input` asks the question and returns the text, `int` converts the text, and `=` stores the number in `a`.

There are conversion functions for each type:

Function	Converts to	Example	Result
`int`	integer	`int("42")`	`42`
`float`	decimal number	`float("3.5")`	`3.5`
`str`	string	`str(42)`	`"42"`
`int`	integer, dropping the decimals	`int(3.9)`	`3`
`round`	nearest integer	`round(3.9)`	`4`

When Conversion Fails

What happens if the user types something that isn't a number?

```
age = int(input("Your age: "))
print("Next year you will be", age + 1)
```

```
Your age: twelve
Traceback (most recent call last):
  File "bad_number.py", line 1, in <module>
    age = int(input("Your age: "))
ValueError: invalid literal for int() with base 10: 'twelve'
```

A `ValueError` means the type was right (`int` accepts strings) but the value wasn't: `'twelve'` is not a number in digits. For now, your programs will trust the user to type numbers. In Chapter 10 you'll learn to catch this error and ask again.

Formatting Output

You met f-strings in the last chapter. After the value in the braces you can add a colon and a **format specification**, which controls how the value looks:

```
price = 1234.5
share = 0.256
item = "Notebook"

print(f"{price:.2f}")  # two decimal places
print(f"{price:,.2f}")  # with a thousands separator
print(f"{share:.1%}")  # as a percentage
print(f"[{item:<12}]")  # left-aligned in 12 characters
print(f"[{item:>12}]")  # right-aligned
print(f"[{item:^12}]")  # centered
print(f"{42:05d}")  # padded with zeros
print(f"{price=}")  # name and value, handy for debugging
```

Output:

```
1234.50
1,234.50
25.6%
[Notebook    ]
[    Notebook]
[  Notebook  ]
00042
price=1234.5
```

The most useful specifications:

- `.2f` shows a float with exactly two decimal places, rounding as needed. Use it for money.

- `,` adds thousands separators.
- `%` multiplies by 100 and adds a percent sign: `0.256` becomes `25.6%`.
- A number on its own is a width: the value is padded with spaces to fill it. `<`, `>` and `^` align the value left, right or center within that width. Text aligns left by default and numbers align right, which is usually what you want for tables.
- `05d` pads an integer with zeros to five digits.
- `{price=}` prints the expression, an equals sign and the value. It's a quick way to check a value while you're developing.

> **Tip:** You can combine these. `{total:>10,.2f}` right-aligns a number in 10 characters, with thousands separators and two decimal places.

Project: Tip Calculator

Now put it together. The tip calculator asks for the bill, the tip percentage and the number of people, and prints what each person pays:

```
# Tip calculator: split a restaurant bill, including the tip.
bill = float(input("Bill amount: "))
tip_percent = float(input("Tip (%): "))
people = int(input("Number of people: "))

tip = bill * tip_percent / 100
total = bill + tip
each = total / people

print()
print(f"Tip:        {tip:8.2f}")
print(f"Total:      {total:8.2f}")
print(f"Each pays:  {each:8.2f}")
```

A run, with the user's answers:

```
Bill amount: 86.40
Tip (%): 15
Number of people: 3

Tip:           12.96
Total:         99.36
Each pays:     33.12
```

The program has the shape of most simple programs: **input** (three questions), **processing** (three calculations) and **output** (three formatted lines). The width of 8 in `{tip:8.2f}` lines the numbers up on the decimal point. Notice that `bill` and `tip_percent` are floats, because people type amounts like `86.40` and tips like `12.5`, but `people` is an integer, because you can't split a bill among two and a half people.

Try It

1. Run the tip calculator with a bill of `100`, a tip of `20` and `4` people. Check the answer in your head.
2. Change the program so that it also prints the tip per person.
3. What happens if you enter `0` people? Read the error message. You'll learn how to prevent it in the next chapter.

Exercises

1. **Temperature.** Ask for a temperature in degrees Celsius and print it in Fahrenheit (multiply by 9, divide by 5, add 32), both with one decimal place.
2. **Days alive.** Ask for the user's age in years and print roughly how many days old they are, with a thousands separator.
3. **Receipt line.** Ask for an item name, a quantity and a unit price, and print a two-line receipt with aligned columns: `Item`, `Qty`, `Price` and `Total`.

Key Takeaways

- `input` shows a prompt and returns what the user typed, always as a string.
- Convert text to numbers with `int` or `float` before doing arithmetic. A `ValueError` means the text wasn't a valid number.
- Format specifications in f-strings control decimals (`.2f`), separators (`,`), percentages (`%`), widths and alignment (`<`, `>`, `^`).
- Most simple programs follow the pattern input, processing, output.

CHAPTER 4 · BEGINNER

Making Decisions

A program that always does the same thing is a calculator. Programs become useful when they decide: charge less for children, warn when a value is too high, accept a password only if it's long enough. In this chapter you'll learn how Python compares values, how it chooses what to do with `if`, `elif` and `else`, and why indentation matters so much in Python. At the end you'll write the pricing rules for a cinema.

True or False

A comparison gives one of two values: `True` or `False`. These are **Booleans**, named after the mathematician George Boole, and their type is `bool`:

```
>>> 5 > 3
True
>>> 5 == 5.0
True
>>> "apple" < "banana"
True
>>> 3 != 3
False
>>> age = 15
>>> age >= 13 and age <= 19
True
>>> 13 <= age <= 19
True
>>> not (age > 18)
True
>>> bool(0), bool(42), bool(""), bool("hi"), bool([])
(False, True, False, True, False)
```

The comparison operators are:

Operator	Meaning
`==`	equal to
`!=`	not equal to
`<` and `>`	less than, greater than
`<=` and `>=`	less than or equal to, greater than or equal to

Don't confuse `==` with `=`. A single `=` assigns a value to a name; `==` asks whether two values are equal. Strings compare alphabetically, which is why `"apple" < "banana"` is `True`; capital letters come before lower-case ones, so `"Zebra" < "apple"` is also `True`.

You can combine conditions with `and` (both must be true), `or` (at least one must be true) and `not` (the opposite). Python also lets you chain comparisons the way mathematicians write them: `13 <= age <= 19` means "age is between 13 and 19, inclusive".

The last line of the session shows that every value can be treated as true or false. Zero, the empty string `""` and empty collections like `[]` count as false; almost everything else counts as true. You'll see this used in conditions such as `if name:`, which means "if the name isn't empty".

if and else

The `if` statement runs a block of code only when a condition is true:

```
number = int(input("Enter a whole number: "))
if number % 2 == 0:
    print(number, "is even.")
else:
    print(number, "is odd.")
```

```
Enter a whole number: 7
7 is odd.
```

The structure is always the same: the keyword `if`, a condition, a colon, and then the lines to run, **indented** by four spaces. The `else:` part is optional; its block runs when the condition is false. Exactly one of the two blocks runs.

Indentation Is Part of the Language

Many languages use brackets to group lines. Python uses indentation: the lines indented under the `if` belong to it, and the first line that's indented less ends the block. This forces Python code to look like what it does, but it means indentation is not optional:

```
age = 20
if age >= 18:
print("You can vote.")
```

```
  File "indent_error.py", line 3
    print("You can vote.")
    ^^^^^
IndentationError: expected an indented block after 'if' statement on line 2
```

Use four spaces for each level. Code editors insert them for you when you press Tab after a colon. Never mix tabs and spaces in one file.

More Than Two Choices: elif

To choose among several options, add `elif` ("else if") branches:

```
score = int(input("Score (0-100): "))

if score >= 90:
    grade = "A"
elif score >= 80:
    grade = "B"
elif score >= 70:
    grade = "C"
elif score >= 60:
    grade = "D"
else:
    grade = "F"

print(f"A score of {score} is a {grade}.")
```

```
Score (0-100): 82
A score of 82 is a B.
```

Python tests the conditions in order and runs the block of the **first** one that's true, then skips the rest. A score of 82 fails `score >= 90`, passes `score >= 80`, and gets a B; the later tests are never checked. That's why the conditions don't need upper limits like `score >= 80 and score < 90`: the order takes

care of it. If you put `score >= 60` first, everyone with 60 or more would get a D.

Project: Cinema Ticket Prices

A cinema charges 11.00 for adults, 6.00 for children under 12 and 7.50 for people aged 65 and over, and everything is half price on Tuesdays:

```
# Cinema tickets: children and seniors pay less, and Tuesday is half price.
age = int(input("Age: "))
day = input("Day of the week: ").strip().lower()

if age < 0 or age > 120:
    print("That age doesn't look right.")
else:
    if age < 12:
        price = 6.00
    elif age >= 65:
        price = 7.50
    else:
        price = 11.00

    if day == "tuesday":
        price = price / 2

    print(f"Your ticket costs {price:.2f}.")
```

A run for a 70-year-old on a Tuesday:

```
Age: 70
Day of the week: Tuesday
Your ticket costs 3.75.
```

This program shows three useful ideas:

- **Check the input first.** An age of -5 or 300 is a typing mistake, so the program says so instead of quoting a price.
- **Nested decisions.** The `if` and `elif` that set the price are inside the `else` block, indented one level further. The Tuesday check comes after the age rules, so it halves whatever price they chose.
- **Tidy the input.** `.strip()` removes spaces the user might have typed before or after the day, and `.lower()` converts it to lower case, so `Tuesday`, `TUESDAY` and `tuesday` all match. Chapter 9 covers these string methods.

Conditional Expressions

When you only need to choose between two values, a **conditional expression** fits on one line:

```
temperature = 31
advice = "Take water" if temperature > 30 else "Enjoy your walk"
print(advice)
```

```
Take water
```

Read it as English: "take water if the temperature is above 30, else enjoy your walk". Use it for simple choices. When either side needs more than a value, use a full `if` statement; it's easier to read.

Try It

1. Run `ticket_price.py` for a child on a Saturday and for an adult on a Tuesday. Then enter `-1` as the age.
2. In `grade.py`, move the `score >= 60` test to the top and run it with a score of 95. Explain the result.
3. In the shell, check whether `"Zebra" < "apple"` and `10 == 10.0` are `True`.

Exercises

1. **Leap year.** A year is a leap year if it's divisible by 4, except that years divisible by 100 aren't, unless they're also divisible by 400. Ask for a year and say whether it's a leap year. Test 2024, 1900 and 2000.
2. **Largest of three.** Ask for three numbers and print the largest, using `if` statements (not Python's `max` function).
3. **BMI.** Ask for weight in kilograms and height in meters, calculate the body mass index (weight divided by height squared), and print it with one decimal place and its category: under 18.5 is underweight, under 25 healthy, under 30 overweight, otherwise obese.

Key Takeaways

- Comparisons give `True` or `False`. Combine them with `and`, `or` and `not`. `==` compares; `=` assigns.
- `if`, `elif` and `else` choose which block runs. Python runs the first block whose condition is true.
- Indentation (four spaces) defines blocks in Python. A missing indent is an `IndentationError`.
- Order your `elif` tests carefully, check input before using it, and use a conditional expression for simple two-way choices.

CHAPTER 5 · BEGINNER

Loops

Computers are good at doing the same thing many times without getting bored or making mistakes. A **loop** repeats a block of code: until a condition changes, or once for each item in a sequence. In this chapter you'll learn Python's two loops, `while` and `for`, and the statements that control them. At the end you'll write a number-guessing game.

while Loops

A `while` loop repeats its block as long as its condition is true:

```
count = 5
while count > 0:
    print(count)
    count = count - 1
print("Lift-off!")
```

```
5
4
3
2
1
Lift-off!
```

Each time round, Python checks the condition `count > 0`. If it's true, the block runs: it prints the count and makes it one smaller. When `count` reaches 0, the condition is false, the loop ends, and Python carries on with the first line after the loop.

> **Warning:** If the condition never becomes false, the loop runs forever. Delete the line `count = count - 1` and the program prints 5 until you stop it. To stop a running program, press **Ctrl+C** in the terminal (in IDLE, choose Shell, Interrupt Execution). Every programmer writes an endless loop now and then.

Shortcuts for Updating Variables

Adding to a variable is so common that Python has a shortcut: `count += 1` means `count = count + 1`. The same works for the other operators: `-=`, `*=`, `/=` and `//=`. You'll see these in most loops from now on.

Loops That Wait for the User

Often you don't know in advance how many times to repeat. This program adds numbers until the user types `done`:

```
# Add up numbers until the user types "done".
total = 0
count = 0
while True:
    entry = input("Number (or 'done'): ")
    if entry == "done":
        break
    total += float(entry)
    count += 1

if count > 0:
    print(f"{count} numbers, total {total}, average {total / count:.2f}")
else:
    print("No numbers entered.")
```

```
Number (or 'done'): 12
Number (or 'done'): 7.5
Number (or 'done'): 30
Number (or 'done'): done
3 numbers, total 49.5, average 16.50
```

`while True` would loop forever, so something inside must end it. That's `break`: it leaves the loop at once and continues after it. This pattern, an endless loop with a `break` when a condition is met, is the usual way to write "keep asking until...".

The program also counts the numbers, so it can calculate the average, and checks that there was at least one before dividing. Without that check, typing `done` straight away would divide by zero.

for Loops and range

A `for` loop runs its block once for each item in a sequence. The simplest sequence is `range`, which produces numbers:

```
for n in range(5):
    print("n is", n)

print("Odd numbers below 10:")
for n in range(1, 10, 2):
    print(n, end=" ")
print()

for letter in "Python":
    print(letter.upper(), end="-")
print()
```

```
n is 0
n is 1
n is 2
n is 3
n is 4
Odd numbers below 10:
1 3 5 7 9
P-Y-T-H-O-N-
```

- `range(5)` gives 0, 1, 2, 3 and 4: five numbers, starting at 0 and stopping **before** 5.
- `range(1, 10, 2)` starts at 1, stops before 10, and steps by 2.
- A string is a sequence of characters, so `for letter in "Python"` visits each letter.

Normally `print` ends with a new line. The `end` argument replaces it: `end=" "` prints a space instead, so the numbers appear on one line. The `print()` after the loop finishes that line.

Use `for` when you're going through something item by item or counting a known number of times; use `while` when you're repeating until something happens.

Skipping with continue

`continue` skips the rest of the block and moves on to the next item:

```
readings = [12, -3, 8, -1, 15]
for value in readings:
    if value < 0:
        continue  # skip this one and go to the next
    print("Reading:", value)
```

```
Reading: 12
Reading: 8
Reading: 15
```

`readings` is a **list**, a collection of values in square brackets. You'll learn all about lists in the next chapter; for now, all you need to know is that a `for` loop visits each value in turn. The two negative readings are skipped.

Loops Inside Loops

A loop's block can contain another loop. The inner loop runs completely for each pass of the outer one:

```
size = 5
for row in range(1, size + 1):
    for col in range(1, size + 1):
        print(f"{row * col:4}", end="")
    print()
```

```
   1   2   3   4   5
   2   4   6   8  10
   3   6   9  12  15
   4   8  12  16  20
   5  10  15  20  25
```

For each row, the inner loop prints five products on one line, each in a width of 4, and `print()` then ends the line. Five rows of five give the 25 products.

Project: Number-Guessing Game

The computer picks a number from 1 to 100 and the player guesses until they find it, with a hint after each guess:

```
import random

random.seed(7)  # remove this line to get a different number each game
secret = random.randint(1, 100)
tries = 0

print("I'm thinking of a number from 1 to 100.")
while True:
    guess = int(input("Your guess: "))
    tries += 1
    if guess < secret:
        print("Higher!")
    elif guess > secret:
        print("Lower!")
    else:
        print(f"Correct! You found it in {tries} tries.")
        break
```

A game:

```
I'm thinking of a number from 1 to 100.
Your guess: 50
Lower!
Your guess: 25
Higher!
Your guess: 37
Higher!
Your guess: 43
Lower!
Your guess: 40
Higher!
Your guess: 42
Correct! You found it in 6 tries.
```

`import random` gives the program access to Python's **random** module, and `random.randint(1, 100)` picks a whole number from 1 to 100, including both ends. You'll learn about modules in Chapter 11.

The line `random.seed(7)` makes the "random" choice the same every time the program runs. That's how the book can print a real game with a known answer, and it's useful when you're testing a program. Delete the line to get a different number each game.

The player above used the best strategy: always guess the middle of the range that's left. Each guess halves the range, so no game takes more than seven

guesses. This idea, **binary search**, is one of the most important in computing, and you'll meet it again in Chapter 22.

Try It

1. Change `countdown.py` to count down from 10 in steps of 2.
2. Use `range` to print the numbers from 10 down to 1 with a `for` loop. (Hint: the step can be negative.)
3. Play the guessing game without the `random.seed(7)` line. Can you always win in seven guesses?

Exercises

1. **FizzBuzz.** Print the numbers from 1 to 20, but print `Fizz` instead of numbers divisible by 3, `Buzz` instead of numbers divisible by 5, and `FizzBuzz` for numbers divisible by both. This classic exercise is still used in job interviews.
2. **Digit sum.** Ask for a whole number and add up its digits, using a `while` loop, `% 10` to get the last digit and `// 10` to remove it.
3. **Three tries.** Ask for a password up to three times. Print `Welcome in!` and stop when it's right. After three wrong attempts, print `Too many attempts. Locked out.` (Hint: a `for` loop can have an `else` block, which runs only if the loop finished without a `break`.)

Key Takeaways

- `while` repeats as long as a condition is true; make sure something changes so that the loop ends. Ctrl+C stops a runaway program.
- `for` goes through a sequence item by item. `range(start, stop, step)` produces numbers up to, but not including, `stop`.
- `break` leaves a loop; `continue` skips to the next pass. `while True` with a `break` is the usual pattern for "repeat until".
- `+=` and its relatives update a variable in place. `print(..., end="")` keeps output on one line.

PART II

Organizing Data and Code

CHAPTER 6 · BEGINNER

Lists and Tuples

Most programs work with collections of values: the songs in a playlist, the readings from a sensor, the books on your shelf. A **list** keeps any number of values in order, and lets you add, remove, sort and search them. A **tuple** is its fixed-size, unchangeable cousin. In this chapter you'll learn both, and you'll start the project that runs through the rest of the book: Pocket Library.

Making and Using Lists

A list is written as values separated by commas, inside square brackets. Lists can grow and shrink, and their contents can change:

```
fruits = ["mango", "banana", "apple"]
print(fruits)
print("First:", fruits[0])
print("Last:", fruits[-1])
print("How many:", len(fruits))

fruits.append("cherry")  # add to the end
fruits.insert(1, "kiwi")  # add at position 1
print(fruits)

fruits.remove("banana")  # remove by value
last = fruits.pop()  # remove and return the last item
print("Popped:", last)
print(fruits)

print("Is apple there?", "apple" in fruits)
fruits.sort()
print("Sorted:", fruits)
```

```
['mango', 'banana', 'apple']
First: mango
Last: apple
How many: 3
['mango', 'kiwi', 'banana', 'apple', 'cherry']
Popped: cherry
['mango', 'kiwi', 'apple']
Is apple there? True
Sorted: ['apple', 'kiwi', 'mango']
```

Indexes work as they do for strings: `fruits[0]` is the first item and `fruits[-1]` the last. The rest are **methods**, which change the list itself:

Method	What it does
`append(x)`	Adds `x` to the end
`insert(i, x)`	Adds `x` at position `i`, moving the rest along
`remove(x)`	Removes the first item equal to `x` (a `ValueError` if there's none)
`pop()`	Removes the last item and gives it back; `pop(i)` removes item `i`
`sort()`	Sorts the list in place
`index(x)`	Gives the position of the first item equal to `x`
`count(x)`	Counts the items equal to `x`

The `in` operator checks whether a value is in a list, and `len` counts the items. A list can hold any type of value, including a mixture, but most lists hold values of one kind.

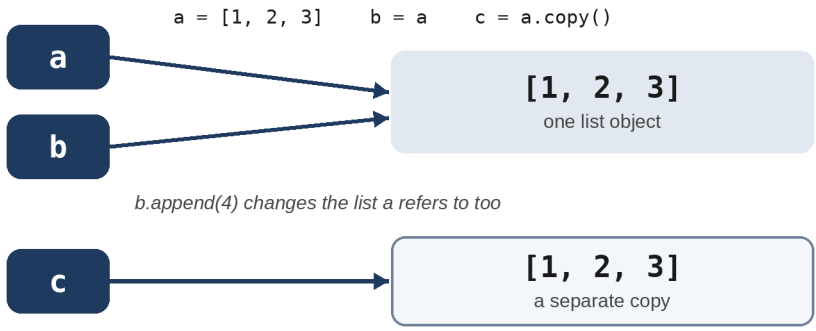
Slices, Copies and Useful Functions

```
>>> numbers = [10, 20, 30, 40, 50]
>>> numbers[1:3]
[20, 30]
>>> numbers[:2]
[10, 20]
>>> numbers[-2:]
[40, 50]
>>> numbers[::-1]
[50, 40, 30, 20, 10]
>>> a = [1, 2, 3]
>>> b = a
>>> b.append(4)
>>> a
[1, 2, 3, 4]
>>> c = a.copy()
```

```
>>> c.append(5)
>>> a
[1, 2, 3, 4]
>>> sum([3, 1, 4]), min([3, 1, 4]), max([3, 1, 4])
(8, 1, 4)
>>> sorted(["pear", "Fig", "apple"])
['Fig', 'apple', 'pear']
>>> sorted(["pear", "Fig", "apple"], key=str.lower)
['apple', 'Fig', 'pear']
```

A **slice** takes part of a list: `numbers[1:3]` takes the items from index 1 up to, but not including, index 3. Leave out the start to begin at the beginning, or the end to go to the end; a third number is the step, and a step of -1 gives you the list reversed. Slices work on strings too.

The middle of the session shows something that surprises nearly everyone. `b = a` doesn't copy the list: it makes `b` a second name for the **same** list, so appending through `b` changes what `a` shows. When you want an independent copy, use `a.copy()` (or `a[:]`).



The names a and b both point to one list object, so a change made through b shows through a. The name c points to a separate copy.

`sum`, `min` and `max` work on any list of numbers. `sorted` gives you a new sorted list and leaves the original alone, unlike the `sort` method. Capital letters sort before lower-case ones, so `"Fig"` comes first; `key=str.lower` sorts as if everything were in lower case. You'll learn more about `key` in Chapter 7.

Looping Over Lists

A `for` loop visits each item in turn. Two built-in functions make loops more useful:

```
tasks = ["Buy milk", "Call Priya", "Fix bike"]

for task in tasks:
    print("-", task)

for number, task in enumerate(tasks, start=1):
    print(f"{number}. {task}")

days = ["Mon", "Tue", "Wed"]
hours = [7.5, 8, 6]
for day, h in zip(days, hours):
    print(f"{day}: {h} h")
```

```
- Buy milk
- Call Priya
- Fix bike
1. Buy milk
2. Call Priya
3. Fix bike
Mon: 7.5 h
Tue: 8 h
Wed: 6 h
```

`enumerate` gives you a number with each item; `start=1` counts from 1, as people do. `zip` walks through two lists side by side, pairing their items. In both loops there are two names after `for`, because each step gives two values.

> **Tip:** Don't add or remove items in a list while you loop over it; the loop can skip items. Loop over a copy (`for task in tasks.copy():`), or build a new list instead.

Tuples

A **tuple** is like a list that can't be changed. Tuples use parentheses, or no brackets at all:

```
point = (3, 4)
x, y = point  # unpacking
print("x =", x, "and y =", y)

red = (255, 0, 0)
print("Red has", len(red), "values")
```

```
a, b = 1, 2
a, b = b, a  # swap two values in one line
print(a, b)

point[0] = 10  # tuples can't be changed
```

```
x = 3 and y = 4
Red has 3 values
2 1
Traceback (most recent call last):
  File "tuples.py", line 12, in <module>
    point[0] = 10  # tuples can't be changed
    ~~~~~^^^
TypeError: 'tuple' object does not support item assignment
```

Use a tuple for a small, fixed group of values that belong together: a point's coordinates, a color's red, green and blue values, a date. **Unpacking** assigns the parts of a tuple to several names at once: `x, y = point`. `a, b = b, a` uses the same trick to swap two values without a temporary variable.

The last line tries to change a tuple and fails. That's the point: when you see a tuple, you know its contents won't change behind your back.

Project: Pocket Library, Version 1

Pocket Library is a program for keeping track of your books. Over the coming chapters it will learn to save to a file, use classes, run tests, and store its books in a database. Version 1 is a list of titles that you can add to:

```
# Pocket Library, version 1: a list of book titles.
books = ["The Hobbit", "Dune", "Educated"]

print("Add books to your library. Press Enter on an empty line to stop.")
while True:
    title = input("Title: ").strip()
    if title == "":
        break
    if title in books:
        print("  You already have that one.")
    else:
        books.append(title)

print()
print(f"Your library has {len(books)} books:")
for number, title in enumerate(sorted(books), start=1):
    print(f"{number:2}. {title}")
```

A session, adding three books (one of them twice) and then pressing Enter on an empty line:

```
Add books to your library. Press Enter on an empty line to stop.
Title: Sapiens
Title: Dune
  You already have that one.
Title: The Midnight Library
Title:

Your library has 5 books:
 1. Dune
 2. Educated
 3. Sapiens
 4. The Hobbit
 5. The Midnight Library
```

Everything in this program is something you've already learned: a `while True` loop with a `break`, a check with `in` so that a title isn't added twice, `append`, and `enumerate` to number the list. `sorted(books)` prints the titles alphabetically without changing the order in which they were added.

Version 1 has two problems. When the program ends, the books are gone. And a title isn't enough: you'll want the author, the year and whether you've read it. Chapter 7 fixes the second problem and Chapter 10 the first.

Try It

1. In the shell, make a list of five numbers and print the first three with a slice, and the list reversed.
2. Add `print(fruits.index("kiwi"))` to `lists_basics.py`. What happens if you ask for the index of a fruit that isn't there?
3. Change `library_v1.py` so that the duplicate check ignores case: `dune` should count as a duplicate of `Dune`. (Hint: compare lower-case versions.)

Exercises

1. **Scores.** Given the scores `[72, 88, 95, 61, 79, 88]`, print the highest, the lowest, the average with one decimal place, and the scores sorted from highest to lowest.

2. **Unique names.** Given a list of names with repeats, build a new list containing each name once, in the order they first appear.
3. **Rotate.** Rotate a list two places to the right, so that `[1, 2, 3, 4, 5]` becomes `[4, 5, 1, 2, 3]`, using slices.

Key Takeaways

- A list holds values in order and can change: `append`, `insert`, `remove`, `pop` and `sort` change it in place.
- Indexes start at 0; slices like `[1:3]` take part of a list; `in` checks membership.
- `b = a` gives the same list a second name. Use `copy()` for an independent copy.
- `enumerate` numbers items in a loop and `zip` pairs two lists. Tuples are fixed groups of values; unpacking splits them into names.

CHAPTER 7 · BEGINNER

Dictionaries and Sets

A list finds things by position: the third item, the last item. Very often you want to find things by name instead: the phone number for Ben, the price of apples, the number of times a word appears. That's what a **dictionary** does. In this chapter you'll learn dictionaries and their simpler relative, **sets**, write a word counter, and give each book in Pocket Library an author, a year and a "read" mark.

Dictionaries

A dictionary stores **key-value pairs**. You write it in curly braces, with a colon between each key and its value. You look a value up by its key, in square brackets:

```
phone_book = {"Asha": "555-0101", "Ben": "555-0102"}

print(phone_book["Asha"])
phone_book["Chen"] = "555-0103"  # add a new entry
phone_book["Ben"] = "555-0199"  # change an existing one
del phone_book["Asha"]  # delete one

print(phone_book)
print("Dev's number:", phone_book.get("Dev", "unknown"))

for name, number in phone_book.items():
    print(f"{name:6} {number}")
print("Names:", list(phone_book.keys()))
```

```
555-0101
{'Ben': '555-0199', 'Chen': '555-0103'}
Dev's number: unknown
Ben    555-0199
Chen   555-0103
Names: ['Ben', 'Chen']
```

Assigning to a key adds the pair if the key is new, or replaces the value if it already exists; `del` removes a pair. Keys are unique: a dictionary can't hold two entries for `"Ben"`. Keys are usually strings or numbers; values can be anything, including lists and other dictionaries.

`items()` gives you the key-value pairs, which is the most common way to loop over a dictionary. `keys()` and `values()` give you just the keys or just the values. Dictionaries remember the order in which keys were added, and loops follow that order.

Missing Keys

Looking up a key that isn't there is an error:

```
>>> stock = {"apples": 12, "pears": 0}
>>> stock["apples"]
12
>>> stock["plums"]
Traceback (most recent call last):
  ...
KeyError: 'plums'
>>> stock.get("plums", 0)
0
>>> "pears" in stock
True
>>> len(stock)
2
>>> stock.update({"plums": 7, "pears": 3})
>>> stock
{'apples': 12, 'pears': 3, 'plums': 7}
>>> sorted(stock, key=stock.get, reverse=True)
['apples', 'plums', 'pears']
```

A `KeyError` names the missing key. There are two ways to avoid it: check first with `in`, or use `get`, which returns a default value (or `None`, Python's "nothing" value) instead of failing. `update` adds or replaces several pairs at once.

The last line sorts the keys by their values. `sorted(stock)` would sort the keys alphabetically; `key=stock.get` tells `sorted` to call `stock.get` on each

key and sort by the result, which is the quantity. `reverse=True` puts the largest first. You'll use `key=` often; Chapter 20 shows all it can do.

Project: Word Counter

Counting things is one of the most common jobs for a dictionary. This program counts the words in a text and shows the five most common:

```
# Count the words in a text, then show the five most common.
text = """The cat sat on the mat. The dog sat on the log.
The cat and the dog are friends, and the mat is the cat's."""

counts = {}
for word in text.lower().split():
    word = word.strip(".,!?;:\"'")
    if word.endswith("'s"):
        word = word[:-2]
    counts[word] = counts.get(word, 0) + 1

print(len(counts), "different words")
most_common = sorted(counts, key=counts.get, reverse=True)
for word in most_common[:5]:
    print(f"{word:8} {counts[word]}")
```

```
11 different words
the      8
cat      3
sat      2
on       2
mat      2
```

The text is a **triple-quoted string**, which can run over several lines. The program converts it to lower case, so that "The" and "the" count as one word, and `split()` breaks it into words at spaces and line breaks. `strip` removes punctuation from both ends of each word, and the program removes a trailing `'s` so that "cat's" counts as "cat".

The heart of the program is one line:

```
counts[word] = counts.get(word, 0) + 1
```

If the word is new, `get` returns 0, so it's stored with a count of 1. Otherwise, its count goes up by one. Remember this line; you'll write it many times.

When two words have the same count, such as "sat", "on" and "mat", `sorted` keeps them in the order they were first added. That's called a **stable** sort.

Sets

A **set** is an unordered collection with no duplicates. You write it in curly braces without colons. Sets are fast at checking membership, and they support the operations you may remember from school mathematics:

```
alice = {"python", "chess", "hiking", "jazz"}
bob = {"jazz", "football", "python"}

print("Both like:", sorted(alice & bob))
print("Either likes:", sorted(alice | bob))
print("Only Alice:", sorted(alice - bob))

tags = ["red", "blue", "red", "green", "blue"]
print("Unique tags:", sorted(set(tags)))
print("Is 'chess' in alice?", "chess" in alice)
```

```
Both like: ['jazz', 'python']
Either likes: ['chess', 'football', 'hiking', 'jazz', 'python']
Only Alice: ['chess', 'hiking']
Unique tags: ['blue', 'green', 'red']
Is 'chess' in alice? True
```

- `a & b`, the **intersection**, holds the items in both sets.
- `a | b`, the **union**, holds the items in either set.
- `a - b`, the **difference**, holds the items in `a` but not in `b`.

`set(tags)` is the quickest way to remove duplicates from a list, although it loses the order. Sets have no order, so the program sorts them before printing to make the output predictable. An empty set is written `set()`, because `{}` is an empty dictionary.

Choosing a Collection

You need	Use
Items in order, which can change	a list
A small, fixed group of values	a tuple
Values looked up by a key	a dictionary
Unique items, or fast "is it there?" checks	a set

Project: Pocket Library, Version 2

A book is more than a title. Version 2 stores each book as a dictionary, and the library as a list of dictionaries, a structure you'll meet in nearly every real program:

```
# Pocket Library, version 2: each book is a dictionary.
books = [
    {
        "title": "The Hobbit",
        "author": "J. R. R. Tolkien",
        "year": 1937,
        "read": True,
    },
    {
        "title": "Dune",
        "author": "Frank Herbert",
        "year": 1965,
        "read": False,
    },
    {
        "title": "The Silmarillion",
        "author": "J. R. R. Tolkien",
        "year": 1977,
        "read": False,
    },
    {
        "title": "Educated",
        "author": "Tara Westover",
        "year": 2018,
        "read": True,
    },
]

print(f"{'Title':<18}{'Author':<18}{'Year':>5}  Read")
for book in books:
    mark = "yes" if book["read"] else ""
    print(
        f"{book['title']:<18}{book['author']:<18}{book['year']:>5}  {mark}"
    )

print()
to_read = []
for book in books:
    if not book["read"]:
        to_read.append(book["title"])
print("Still to read:", ", ".join(to_read))

per_author = {}
for book in books:
    per_author[book["author"]] = per_author.get(book["author"], 0) + 1
print("Books per author:")
for author, count in per_author.items():
    print(f"  {author}: {count}")
```

```
Title             Author             Year  Read
The Hobbit        J. R. R. Tolkien   1937  yes
Dune              Frank Herbert      1965
The Silmarillion  J. R. R. Tolkien   1977
Educated          Tara Westover      2018  yes

Still to read: Dune, The Silmarillion
Books per author:
  J. R. R. Tolkien: 2
  Frank Herbert: 1
  Tara Westover: 1
```

To reach a value, you index twice: `books[0]` is the first book's dictionary, and `books[0]["title"]` is its title. In the loops, `book["title"]` does the same for each book in turn. Inside an f-string that's already in double quotes, the keys use single quotes: `{book['title']}`.

The program builds two summaries. The "still to read" list uses a loop and `append`, and `", ".join(to_read)` joins the titles into one string with a comma and a space between them. The count per author uses the same `get` pattern as the word counter.

Try It

1. In `dict_basics.py`, try to print `phone_book["Asha"]` after it's been deleted. What error do you get?
2. Change `word_counter.py` to show the ten most common words, using your own paragraph of text.
3. Add a book of your own to `library_v2.py` and run it. Do the columns still line up? What happens if the title is longer than 18 characters?

Exercises

1. **Invert.** Given a dictionary of countries and their capitals, build a new dictionary that maps each capital to its country, and use it to look up the country whose capital is Tokyo.
2. **Letter count.** Count how often each letter appears in "Mississippi", ignoring case, and print the letters in alphabetical order with their counts.

3. **Common friends.** Priya's friends are Asha, Ben, Chen and Dev; Omar's are Chen, Dev and Ella. Using sets, print the friends they have in common, and Priya's friends whom Omar doesn't know yet.

Key Takeaways

- A dictionary maps unique keys to values: `d[key]` reads, `d[key] = value` adds or changes, `del d[key]` removes.
- A missing key raises `KeyError`; use `in` or `get(key, default)` to avoid it. `counts[k] = counts.get(k, 0) + 1` counts things.
- Loop over `items()` to get keys and values together. `sorted(d, key=d.get)` sorts keys by their values.
- Sets hold unique items and support `&`, `|` and `-`. A list of dictionaries is the natural way to store records.

CHAPTER 8 · BEGINNER

Functions

You've been calling functions since Chapter 1: `print`, `input`, `len`, `sorted`. In this chapter you'll write your own. A function gives a name to a piece of code, so you can use it as often as you like without repeating it. Functions are how programmers break a large problem into small ones that are easy to understand, test and reuse. You'll build a unit converter and reorganize Pocket Library into functions.

Defining a Function

You define a function with `def`, a name, parentheses with its **parameters**, and a colon. The indented block below is the function's **body**:

```
def greet(name):
    """Print a friendly greeting."""
    print(f"Hello, {name}!")


def area(width, height):
    """Return the area of a rectangle."""
    return width * height


def price_with_tax(price, rate=0.18):
    """Return the price including tax: 18% unless you say otherwise."""
    return round(price * (1 + rate), 2)


greet("Lena")
greet("Tom")
print("Area:", area(3, 4.5))
print("With tax:", price_with_tax(100))
print("With 5% tax:", price_with_tax(100, 0.05))
```

```
print("Keyword arguments:", price_with_tax(rate=0.12, price=250))
```

```
Hello, Lena!
Hello, Tom!
Area: 13.5
With tax: 118.0
With 5% tax: 105.0
Keyword arguments: 280.0
```

- `def` only defines the function; nothing happens until you **call** it, by writing its name with arguments in parentheses. `greet("Lena")` runs the body with `name` set to `"Lena"`.
- `return` sends a value back to the caller and ends the function. `area(3, 4.5)` becomes `13.5` wherever it's used. A function without `return` gives back `None`.
- A parameter can have a **default value**: `rate=0.18`. Callers can leave it out and get the default, or pass their own.
- **Keyword arguments** name the parameter they're for, like `rate=0.12`, so their order doesn't matter. They also make a call easier to read.
- The string on the first line of the body is a **docstring**. It says what the function does. Editors show it when you use the function, and `help(area)` in the shell prints it.

> **Tip:** Make each function do one job, and name it after that job with a verb: `greet`, `calculate_total`, `add_book`. If you can't describe a function in one sentence, it probably does too much.

Printing versus Returning

Beginners often confuse `print` and `return`. `greet` prints, which is useful only for showing something to a person. `area` returns, so the caller decides what to do with the result: print it, store it, or use it in another calculation, such as `area(3, 4) * 2`. Prefer functions that return values; they're far easier to reuse and to test.

Local and Global Variables

Variables created inside a function are **local**: they exist only while the function runs.

```
counter = 0  # a global variable


def add_one(value):
    result = value + 1  # a local variable: it exists only in here
    return result


print(add_one(41))
print(counter)
print(result)  # result is not defined out here
```

```
42
0
Traceback (most recent call last):
  File "scope.py", line 11, in <module>
    print(result)  # result is not defined out here
          ^^^^^^
NameError: name 'result' is not defined
```

`result` belongs to `add_one`, and it's gone when the function returns, so the last line raises a `NameError`. Variables created outside any function, like `counter`, are **global**: functions can read them, but assigning to one inside a function creates a new local variable with the same name instead. The clean way to get information into a function is through its parameters, and out of it through `return`.

Functions as Building Blocks: A Unit Converter

The converter has a small function for each conversion, a dictionary that links menu choices to those functions, and a `main` function that runs the menu:

```
def km_to_miles(km):
    return km * 0.621371


def celsius_to_fahrenheit(c):
    return c * 9 / 5 + 32


def kg_to_pounds(kg):
    return kg * 2.20462
```

```
CONVERSIONS = {
    "1": ("km to miles", km_to_miles),
    "2": ("°C to °F", celsius_to_fahrenheit),
    "3": ("kg to pounds", kg_to_pounds),
}


def show_menu():
    for key, (label, _) in CONVERSIONS.items():
        print(f"  {key}. {label}")
    print("  q. quit")


def main():
    while True:
        show_menu()
        choice = input("Choose: ").strip()
        if choice == "q":
            print("Goodbye!")
            break
        if choice not in CONVERSIONS:
            print("Please choose 1, 2, 3 or q.")
            continue
        label, convert = CONVERSIONS[choice]
        value = float(input("Value: "))
        print(f"  {label}: {value} -> {convert(value):.2f}")


main()
```

A session, converting a marathon distance and a temperature, then making a mistake and quitting:

```
  1. km to miles
  2. °C to °F
  3. kg to pounds
  q. quit
Choose: 1
Value: 42.195
  km to miles: 42.195 -> 26.22
  1. km to miles
  2. °C to °F
  3. kg to pounds
  q. quit
Choose: 2
Value: -40
  °C to °F: -40.0 -> -40.00
  1. km to miles
  2. °C to °F
  3. kg to pounds
  q. quit
Choose: 5
Please choose 1, 2, 3 or q.
  1. km to miles
  2. °C to °F
  3. kg to pounds
  q. quit
```

```
Choose: q
Goodbye!
```

Look at `CONVERSIONS`. Its values are tuples that hold a label and a function: the name `km_to_miles` without parentheses refers to the function itself, without calling it. Later, `label, convert = CONVERSIONS[choice]` unpacks the tuple, and `convert(value)` calls whichever function was chosen. Functions are values in Python, just like numbers and strings. That idea will take you a long way, and Chapter 18 builds on it.

The name `CONVERSIONS` is in capitals because it's a **constant**: a value that's set once and never changed. Python doesn't enforce this; it's a convention that tells readers not to change it. In `show_menu`, the underscore in `for key, (label, _)` is a name for a value the loop doesn't need.

Adding a fourth conversion means writing one small function and one line in the dictionary. Nothing else changes. That's the payoff of organizing code into functions.

Flexible Arguments

Sometimes a function should accept any number of arguments, as `print` does:

```
def total(*numbers):
    """Add any number of numbers."""
    result = 0
    for n in numbers:
        result += n
    return result


def describe(name, **details):
    """Print a name followed by any details given as keyword arguments."""
    print(name)
    for key, value in details.items():
        print(f"  {key}: {value}")


print(total(1, 2, 3))
print(total(10, 20, 30, 40, 50))
describe("Pocket Library", version=3, books=4, language="Python")
```

```
6
150
Pocket Library
  version: 3
  books: 4
  language: Python
```

A parameter with one star, `*numbers`, collects all the positional arguments into a tuple. A parameter with two stars, `**details`, collects keyword arguments into a dictionary. By convention they're often called `*args` and `**kwargs`, and you'll see them in many libraries.

Functions That Call Themselves

A function can call itself. This is called **recursion**:

```
def factorial(n):
    """Return n! = n * (n - 1) * ... * 1."""
    if n <= 1:
        return 1
    return n * factorial(n - 1)


def count_down(n):
    if n == 0:
        print("Go!")
        return
    print(n)
    count_down(n - 1)


print(factorial(5))
print(factorial(20))
count_down(3)
```

```
120
2432902008176640000
3
2
1
Go!
```

The factorial of 5 is 5 × 4 × 3 × 2 × 1. `factorial(5)` works it out as 5 times `factorial(4)`, which is 4 times `factorial(3)`, and so on, until `factorial(1)` returns 1 without calling itself. That stopping case, the **base case**, is essential: without it, the function would call itself until Python gave up with a `RecursionError`. Recursion is natural for problems that contain

smaller copies of themselves, like folders inside folders, but a loop is usually simpler for counting.

Project: Pocket Library, Version 3

Version 3 keeps the dictionaries from Chapter 7 and organizes the code into functions:

```
# Pocket Library, version 3: the same data, organized into functions.


def make_book(title, author, year, read=False):
    return {"title": title, "author": author, "year": year, "read": read}


def add_book(books, book):
    for existing in books:
        if existing["title"].lower() == book["title"].lower():
            return False
    books.append(book)
    return True


def books_by(books, author):
    found = []
    for book in books:
        if author.lower() in book["author"].lower():
            found.append(book)
    return found


def show(books):
    for book in books:
        mark = "x" if book["read"] else " "
        print(
            f"[{mark}] {book['title']} ({book['author']}, {book['year']})"
        )


library = []
add_book(
    library, make_book("The Hobbit", "J. R. R. Tolkien", 1937, read=True)
)
add_book(library, make_book("Dune", "Frank Herbert", 1965))
add_book(library, make_book("The Silmarillion", "J. R. R. Tolkien", 1977))
print(
    "Added again?",
    add_book(library, make_book("dune", "Frank Herbert", 1965)),
)

show(library)
print()
print("By Tolkien:")
show(books_by(library, "tolkien"))
```

```
Added again? False
[x] The Hobbit (J. R. R. Tolkien, 1937)
[ ] Dune (Frank Herbert, 1965)
[ ] The Silmarillion (J. R. R. Tolkien, 1977)

By Tolkien:
[x] The Hobbit (J. R. R. Tolkien, 1937)
[ ] The Silmarillion (J. R. R. Tolkien, 1977)
```

Each function does one job. `make_book` builds a book dictionary, with `read` defaulting to `False`. `add_book` adds a book only if no book with the same title, ignoring case, is already there, and returns `True` or `False` to say whether it did. `books_by` finds books whose author contains some text, and `show` prints a list of books. Because `books_by` returns a list rather than printing, its result can go straight into `show`, or be counted, or sorted.

Notice that `add_book` changes the list it's given. Lists are passed to functions as themselves, not as copies, so `library` outside the function sees the new book.

Try It

1. Call `help(price_with_tax)` in the shell after pasting the function in. What does it show?
2. Add a "miles to km" conversion to the unit converter. How many lines did you have to change?
3. Remove the `if n <= 1` test from `factorial` and call `factorial(5)`. Read the last line of the error.

Exercises

Write these three functions in one file, with docstrings, and call each one to test it:

1. **Palindrome.** `is_palindrome(text)` returns `True` if the text reads the same backwards, ignoring case, spaces and punctuation. "Never odd or even" is a palindrome. (Hint: a string's `isalnum()` method is true for letters and digits.)
2. **Vowels.** `count_vowels(text)` returns the number of vowels in the text.

3. **Largest.** `largest(numbers)` returns the largest number in a non-empty list, without using `max`.

Key Takeaways

- `def` defines a function; calling it runs the body with the parameters set to the arguments. `return` sends a value back.
- Parameters can have defaults, and arguments can be passed by keyword. A docstring says what a function does.
- Variables inside a function are local. Pass information in through parameters and out through `return`.
- Functions are values: you can store them in dictionaries and call them later. `*args` and `**kwargs` accept any number of arguments.

CHAPTER 9 · INTERMEDIATE

Working with Text

A great deal of programming is working with text: cleaning up what users type, reading names and addresses, splitting lines into fields, checking passwords, finding email addresses in a document. Python's strings come with dozens of methods for this, and the `re` module adds **regular expressions** for finding patterns. In this chapter you'll learn the methods you'll use most, build a password checker, and take your first steps with regular expressions.

Slicing Strings

Strings are sequences of characters, so the slices you used on lists in Chapter 6 work on strings too:

```
>>> word = "Programming"
>>> word[0:3]
'Pro'
>>> word[3:]
'gramming'
>>> word[::2]
'Pormig'
>>> word.upper().center(17, "*")
'***PROGRAMMING***'
>>> "a-b-c".split("-", maxsplit=1)
['a', 'b-c']
>>> "Hello".ljust(8, ".") + "World"
'Hello...World'
>>> word[0] = "p"
Traceback (most recent call last):
  ...
TypeError: 'str' object does not support item assignment
```

`word[0:3]` takes characters 0, 1 and 2; `word[3:]` takes everything from index 3; `word[::2]` takes every second character; and `word[::-1]`, as you saw in Chapter 2, reverses a string.

The last line shows that strings are **immutable**: you can't change a character in place. Every string method returns a new string and leaves the original as it was. To "change" a string, you build a new one and assign it to the name: `word = "p" + word[1:]`.

String Methods

These are the methods you'll reach for most often:

```
raw = "   python is FUN   "
print(repr(raw.strip()))
print(raw.strip().capitalize())
print(raw.strip().title())

sentence = "red,green,,blue"
parts = sentence.split(",")
print(parts)
print(" | ".join(parts))

email = "meera@example.com"
print(email.startswith("meera"), email.endswith(".org"))
print(email.find("@"), email.count("e"))
print(email.replace("example.com", "example.org"))
print("2026".isdigit(), "20x6".isdigit())
```

```
'python is FUN'
Python is fun
Python Is Fun
['red', 'green', '', 'blue']
red | green |  | blue
True False
5 4
meera@example.org
True False
```

Method	What it does
`strip()`	Removes spaces (and line breaks) from both ends; `lstrip` and `rstrip` do one end
`lower()`, `upper()`, `capitalize()`, `title()`	Change case
`split(sep)`	Splits into a list at each `sep`; with no argument, at any run of spaces

Method	What it does
`sep.join(list)`	Joins a list of strings with `sep` between them
`startswith(s)`, `endswith(s)`	Check the beginning or end
`find(s)`	Position of `s`, or -1 if it isn't there
`count(s)`	How many times `s` appears
`replace(old, new)`	Replaces every `old` with `new`
`isdigit()`, `isalpha()`, `isupper()`, `islower()`	Check what kind of characters a string contains

`repr` shows a value the way you'd type it in code, with quotes. It's useful for seeing spaces at the ends of a string, which `print` hides. Notice that `split(",")` keeps the empty string between the two commas: splitting on a specific separator never drops empty fields, which matters when you read data files.

> **Tip:** `split` and `join` are opposites. `"-".join("2026-10-15".split("-"))` gives back the original string. Together they're the quickest way to rearrange the parts of a string.

Analyzing Text

This program reports on the first lines of a well-known poem by William Wordsworth:

```
poem = """I wandered lonely as a cloud
That floats on high o'er vales and hills,
When all at once I saw a crowd,
A host, of golden daffodils;"""

lines = poem.splitlines()
words = poem.replace(",", " ").replace(";", " ").split()

print("Lines:", len(lines))
print("Words:", len(words))
print("Characters:", len(poem))
longest = ""
for word in words:
    if len(word) > len(longest):
        longest = word
print("Longest word:", longest)
for number, line in enumerate(lines, start=1):
```

```
    print(f"{number}: {line[:20]:20} ... ({len(line)} chars)")
```

```
Lines: 4
Words: 27
Characters: 131
Longest word: daffodils
1: I wandered lonely as ... (28 chars)
2: That floats on high  ... (41 chars)
3: When all at once I s ... (31 chars)
4: A host, of golden da ... (28 chars)
```

`splitlines` splits a text into lines. To count words, the program first replaces commas and semicolons with spaces, so that `crowd,` counts as `crowd`, then splits on spaces. The loop to find the longest word keeps the first word of the greatest length, so "daffodils" wins over "wandered", which comes first but is one letter shorter. The last loop uses a slice and a width, `{line[:20]:20}`, to show the first 20 characters of each line in a column of exactly 20.

Project: Password Checker

A good password is long and mixes kinds of characters. This checker lists everything that's wrong with a password, not just the first problem, so the user can fix it in one go:

```
def password_problems(password):
    """Return the ways a password is too weak (an empty list if none)."""
    problems = []
    if len(password) < 10:
        problems.append("use at least 10 characters")
    has_upper = has_lower = has_digit = has_symbol = False
    for ch in password:
        if ch.isupper():
            has_upper = True
        elif ch.islower():
            has_lower = True
        elif ch.isdigit():
            has_digit = True
        else:
            has_symbol = True
    if not has_upper:
        problems.append("add an upper-case letter")
    if not has_lower:
        problems.append("add a lower-case letter")
    if not has_digit:
        problems.append("add a digit")
    if not has_symbol:
        problems.append("add a symbol such as ! or #")
    if password.lower() in ("password123", "qwertyuiop", "letmein123"):
        problems.append("don't use a well-known password")
    return problems
```

```
while True:
    password = input("Try a password (or press Enter to stop): ")
    if not password:
        break
    problems = password_problems(password)
    if problems:
        print("Weak:")
        for problem in problems:
            print("  -", problem)
    else:
        print("Strong password!")
```

```
Try a password (or press Enter to stop): sunshine
Weak:
  - use at least 10 characters
  - add an upper-case letter
  - add a digit
  - add a symbol such as ! or #
Try a password (or press Enter to stop): Sunshine2026
Weak:
  - add a symbol such as ! or #
Try a password (or press Enter to stop): Sun$hine-2026
Strong password!
Try a password (or press Enter to stop):
```

The function goes through the password one character at a time and records which kinds it has seen, using `isupper`, `islower` and `isdigit`; anything else counts as a symbol. The line `has_upper = has_lower = has_digit = has_symbol = False` sets four variables to the same value at once. The function returns a list of problems, and an empty list means the password is strong. This design keeps the checking separate from the asking and printing, so the function could be reused in a web form or tested on its own, as you'll do in Chapter 15.

> **Note:** Real password rules are more subtle. Current security advice favors long passphrases and checking against lists of leaked passwords over rules about symbols. Treat this program as practice with strings, not as a security tool.

Regular Expressions

Some text problems are about patterns: "something that looks like an email address" or "three digits, a space, four digits". **Regular expressions** describe such patterns in a compact language of their own. Python's `re` module uses them:

```
import re

text = """Contact Meera at meera.k@example.com or 020 7946 0018.
Orders: orders@example.org, phone 020 7946 0991. Office hours 9-17."""

emails = re.findall(r"[\w.]+@[\w.]+\.\w+", text)
print("Emails:", emails)

phones = re.findall(r"\d{3} \d{4} \d{4}", text)
print("Phones:", phones)

masked = re.sub(r"(\d{3} \d{4}) \d{4}", r"\1 ****", text)
print(masked)

match = re.search(r"hours (\d+)-(\d+)", text)
if match:
    print("Opens at", match.group(1), "and closes at", match.group(2))
```

```
Emails: ['meera.k@example.com', 'orders@example.org']
Phones: ['020 7946 0018', '020 7946 0991']
Contact Meera at meera.k@example.com or 020 7946 ****.
Orders: orders@example.org, phone 020 7946 ****. Office hours 9-17.
Opens at 9 and closes at 17
```

Write patterns as **raw strings**, with an `r` before the quote, so that backslashes reach the `re` module unchanged. The pieces used here:

Pattern	Matches
`\d`	any digit
`\w`	a letter, digit or underscore
`[\w.]`	any one of the characters in the brackets
`+`	one or more of the thing before it
`{4}`	exactly four of the thing before it
`( )`	a group: a part you want to use separately

`re.findall` returns every match in a list. `re.sub` replaces matches; `\1` in the replacement means "whatever group 1 matched", which is how the program

keeps the first part of each phone number and masks the rest. `re.search` finds the first match and returns a **match object**, or `None` if there isn't one, so always check it before using it. `match.group(1)` and `match.group(2)` give the two groups.

Regular expressions are powerful, but they quickly become hard to read. Use string methods when they're enough, and keep patterns short and commented when they're not. The email pattern here is deliberately simple: a pattern that accepts every valid email address and nothing else is very long, and the only real test of an email address is to send it a message.

Try It

1. In the shell, split `"2026-10-15"` into year, month and day, and join them back as `15/10/2026`.
2. Try `" lots of spaces ".split()` and `.split(" ")`. Why are the results different?
3. Change the phone pattern in `regex_intro.py` so that it also finds numbers written with hyphens, like `020-7946-0018`. (Hint: `[ -]` matches a space or a hyphen.)

Exercises

1. **Caesar cipher.** Write `caesar(text, shift)`, which shifts each letter by `shift` places in the alphabet, wrapping from Z back to A, keeping upper and lower case and leaving other characters alone. Shifting by -3 should undo a shift of 3. (Hint: `ord("A")` is the number for a character and `chr(65)` the character for a number.)
2. **Title case.** Write `my_title(text)`, which capitalizes the first letter of each word and makes the rest lower case, without using `title()`.
3. **Hashtags.** Write `hashtags(text)`, which returns the hashtags in a text, without the `#`, using a regular expression.

Key Takeaways

- Strings can be sliced like lists but can't be changed in place; every method returns a new string.
- `strip`, `lower`, `split`, `join`, `replace`, `startswith` and `find` handle most everyday text work.
- Keep checking logic in functions that return results, separate from input and output.
- Regular expressions find patterns: `re.findall`, `re.search` and `re.sub`, with raw-string patterns like `r"\d{4}"`. Use them when string methods aren't enough.

CHAPTER 10 · INTERMEDIATE

Files and Exceptions

Until now, everything your programs knew was lost when they ended. Files fix that: a program can save its data and read it back next time. Files also bring new ways for things to go wrong, such as a missing file, damaged data, or a disk that's full, so this chapter also covers **exceptions**: how to catch errors, recover from them and raise your own. At the end, Pocket Library will remember your books between runs.

Reading and Writing Text Files

The easiest way to work with files is the `Path` class from the `pathlib` module. A `Path` represents a file or folder name and has methods to read, write and inspect it:

```
from pathlib import Path

notes = Path("notes.txt")

notes.write_text("Buy bread\nCall the plumber\n", encoding="utf-8")
with notes.open("a", encoding="utf-8") as f:  # "a" adds to the end
    f.write("Return library books\n")

print(notes.read_text(encoding="utf-8"))

with notes.open(encoding="utf-8") as f:
    for number, line in enumerate(f, start=1):
        print(number, line.rstrip())

print("Exists?", notes.exists(), "| Size:", notes.stat().st_size, "bytes")
```

```
Buy bread
Call the plumber
Return library books

1 Buy bread
2 Call the plumber
3 Return library books
Exists? True | Size: 48 bytes
```

- `write_text` creates the file, or replaces it completely if it exists, and writes a string to it. `\n` is the **newline** character, which ends a line.
- `open("a")` opens the file for **appending**: writing at the end, keeping what's there. `"w"` would empty it first and `"r"`, the default, opens it for reading.
- `read_text` reads the whole file into one string. For large files, loop over the open file instead, as the second block does: each pass gives one line, including its newline, which `rstrip` removes.
- `exists` checks whether the file is there; `stat().st_size` gives its size in bytes.

The `with` statement opens the file and guarantees that it's closed again when the block ends, even if an error happens inside it. Always use `with` when you open a file. Chapter 18 explains how it works.

> **Tip:** Always pass `encoding="utf-8"`. Without it, Python uses your computer's default encoding, which differs between Windows and other systems, and a file with accented letters or symbols written on one computer may not read correctly on another.

Relative file names like `notes.txt` refer to the **current working directory**: the folder you were in when you ran the program. If a program can't find a file that's clearly there, check which folder you ran it from.

CSV Files

A **CSV** (comma-separated values) file is a simple spreadsheet: one row per line, with the fields separated by commas. Spreadsheet programs can open and save CSV files, which makes them the most common way to exchange tabular data. Here's `books.csv`:

```
title,author,year,pages,rating
The Hobbit,J. R. R. Tolkien,1937,310,5
Dune,Frank Herbert,1965,412,4
Educated,Tara Westover,2018,334,5
The Midnight Library,Matt Haig,2020,288,4
Sapiens,Yuval Noah Harari,2011,443,3
```

The `csv` module reads and writes them correctly, including awkward cases like a comma inside a field:

```
import csv
from pathlib import Path

with open("books.csv", newline="", encoding="utf-8") as f:
    books = list(csv.DictReader(f))

print(f"{len(books)} books")
total_pages = 0
for book in books:
    total_pages += int(book["pages"])
    stars = "*" * int(book["rating"])
    print(f"{book['title']:<22}{book['pages']:>5} pages  {stars}")
print(f"Total: {total_pages} pages, average {total_pages / len(books):.0f}")

with open("favorites.csv", "w", newline="", encoding="utf-8") as f:
    writer = csv.writer(f)
    writer.writerow(["title", "rating"])
    for book in books:
        if book["rating"] == "5":
            writer.writerow([book["title"], book["rating"]])

print(Path("favorites.csv").read_text(encoding="utf-8"))
```

```
5 books
The Hobbit              310 pages  *****
Dune                    412 pages  ****
Educated                334 pages  *****
The Midnight Library    288 pages  ****
Sapiens                 443 pages  ***
Total: 1787 pages, average 357
title,rating
The Hobbit,5
Educated,5
```

`csv.DictReader` reads each row as a dictionary, using the first line for the keys. Everything it reads is a string, so convert numbers with `int` or `float` before calculating, and compare with strings, as in `book["rating"] == "5"`. `csv.writer` writes rows given as lists. Open CSV files with `newline=""`, as here; the `csv` module handles line endings itself.

JSON Files

JSON (JavaScript Object Notation) stores nested data, like dictionaries containing lists, as text. It looks almost exactly like Python, and it's the standard format for settings files and for data sent over the web:

```
import json
from pathlib import Path

settings = {
    "theme": "dark",
    "font_size": 14,
    "recent": ["notes.txt", "todo.txt"],
}

Path("settings.json").write_text(
    json.dumps(settings, indent=2), encoding="utf-8"
)
print(Path("settings.json").read_text(encoding="utf-8"))

loaded = json.loads(Path("settings.json").read_text(encoding="utf-8"))
print(loaded["recent"][0], loaded["font_size"] + 2)
```

```
{
  "theme": "dark",
  "font_size": 14,
  "recent": [
    "notes.txt",
    "todo.txt"
  ]
}
notes.txt 16
```

`json.dumps` turns Python values into JSON text, and `indent=2` makes it readable; `json.loads` turns JSON text back into Python values. JSON writes `True`, `False` and `None` as `true`, `false` and `null`, and it has no tuples or sets: tuples come back as lists, and sets can't be saved directly.

Exceptions

When something goes wrong while a program runs, Python **raises an exception**. If nothing handles it, the program stops and prints a traceback. You've seen many: `NameError`, `TypeError`, `ValueError`. A `try` statement lets you handle them:

```
def safe_divide(a, b):
    try:
        result = a / b
    except ZeroDivisionError:
        print(f"  Can't divide {a} by zero.")
        return None
    else:
        print(f"  {a} / {b} = {result}")
        return result
    finally:
        print("  (finished trying)")


safe_divide(10, 4)
safe_divide(10, 0)

for text in ["42", "4.2", "forty-two"]:
    try:
        number = int(text)
        print(f"{text!r} -> {number}")
    except ValueError as error:
        print(f"{text!r} failed: {error}")
```

```
  10 / 4 = 2.5
  (finished trying)
  Can't divide 10 by zero.
  (finished trying)
'42' -> 42
'4.2' failed: invalid literal for int() with base 10: '4.2'
'forty-two' failed: invalid literal for int() with base 10: 'forty-two'
```

Python runs the `try` block. If an exception of the type named in `except` happens, Python jumps to the `except` block instead of stopping. The optional `else` block runs only if there was no exception, and `finally` runs in every case, which makes it the place for clean-up. `except ValueError as error` gives the exception a name, so you can print its message.

Catch only the exceptions you expect and know how to handle, and keep the `try` block short. A bare `except:` that catches everything also hides typos and real bugs.

Asking Until the Answer Is Valid

In Chapter 3, typing `twelve` instead of a number crashed the program. Now you can fix that:

```
def ask_int(prompt, low, high):
    """Keep asking until the user types a whole number from low to high."""
    while True:
        text = input(prompt)
        try:
            value = int(text)
        except ValueError:
            print("  Please type a whole number.")
            continue
        if low <= value <= high:
            return value
        print(f"  Please choose from {low} to {high}.")


rating = ask_int("Rate the book (1-5): ", 1, 5)
print("Thanks! You gave it", "*" * rating)
```

```
Rate the book (1-5): great
  Please type a whole number.
Rate the book (1-5): 9
  Please choose from 1 to 5.
Rate the book (1-5): 4
Thanks! You gave it ****
```

This function handles both kinds of bad input: text that isn't a number (`int` raises `ValueError`, and the `except` block asks again) and a number outside the range (checked with an `if`). It returns only when it has a valid answer. You'll find a function like this useful in many programs.

When a File Is Missing

Files are a common source of exceptions: the file isn't there, it's in a different folder, or you don't have permission to read it.

```
from pathlib import Path

path = Path("does_not_exist.txt")
try:
    text = path.read_text(encoding="utf-8")
except FileNotFoundError:
    print(f"No file called {path.name}; starting with an empty list.")
    text = ""
print("Lines:", len(text.splitlines()))

# The same again, without try: the program stops with a traceback
```

```
path.read_text(encoding="utf-8")
```

```
No file called does_not_exist.txt; starting with an empty list.
Lines: 0
Traceback (most recent call last):
  File "missing_file.py", line 12, in <module>
    path.read_text(encoding="utf-8")
    ~~~~~~~~~~~~~~~^^^^^^^^^^^^^^^^^^
  File ".../lib/python3.15/pathlib/__init__.py", line 986, in read_text
    with self.open(mode='r', encoding=encoding, errors=errors,
    ↪ newline=newline) as f:
         ~~~~~~~~~^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^
         ↪ ^^^^^
  File ".../lib/python3.15/pathlib/__init__.py", line 970, in open
    return io.open(self, mode, buffering, encoding, errors, newline)
           ~~~~~~~^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^
FileNotFoundError: [Errno 2] No such file or directory: 'does_not_exist.txt'
```

The first attempt catches `FileNotFoundError` and carries on with an empty text. The second has no `try`, and the program stops. Notice that the traceback is longer this time: the error happened inside Python's own `pathlib` code, which your program called. Read it from the bottom: the last line says what went wrong, and the **first** frame that names your file says which of your lines caused it. The frames in between are Python's own code; their details differ between Python versions.

Raising Your Own Exceptions

Your functions can raise exceptions too, when they're asked to do something that makes no sense:

```
def set_rating(book, rating):
    if not 1 <= rating <= 5:
        raise ValueError(f"rating must be 1 to 5, not {rating}")
    book["rating"] = rating


book = {"title": "Dune"}
set_rating(book, 4)
print(book)
try:
    set_rating(book, 9)
except ValueError as error:
    print("Error:", error)
```

```
{'title': 'Dune', 'rating': 4}
Error: rating must be 1 to 5, not 9
```

`raise ValueError(...)` stops the function and hands the problem to the caller, with a message explaining it. That's better than quietly storing a rating of 9, which would cause a confusing problem much later. The caller decides what to do: here, it catches the error and prints the message.

Project: Pocket Library, Version 4

Version 4 loads the library from `library.json` when it starts and saves it when it ends:

```
# Pocket Library, version 4: books are saved in library.json between runs.
import json
from pathlib import Path

DATA = Path("library.json")


def load_books():
    if not DATA.exists():
        return []
    try:
        return json.loads(DATA.read_text(encoding="utf-8"))
    except json.JSONDecodeError:
        print("library.json is damaged; starting with an empty library.")
        return []


def save_books(books):
    DATA.write_text(json.dumps(books, indent=2), encoding="utf-8")


def ask_year(prompt):
    while True:
        text = input(prompt)
        try:
            return int(text)
        except ValueError:
            print("  The year must be a number, such as 1997.")


books = load_books()
print(f"Loaded {len(books)} books.")
while True:
    title = input("New title (Enter to finish): ").strip()
    if not title:
        break
    author = input("Author: ").strip()
    year = ask_year("Year: ")
    books.append(
        {"title": title, "author": author, "year": year, "read": False}
    )

save_books(books)
print(f"Saved {len(books)} books to {DATA}.")
```

```
for book in books:
    print(f"- {book['title']} ({book['year']})")
```

Starting with the three books in `library.json` and adding one more:

```
Loaded 3 books.
New title (Enter to finish): Sapiens
Author: Yuval Noah Harari
Year: twenty eleven
  The year must be a number, such as 1997.
Year: 2011
New title (Enter to finish):
Saved 4 books to library.json.
- The Hobbit (1937)
- Dune (1965)
- Educated (2018)
- Sapiens (2011)
```

Run it again and the new book is still there. `load_books` handles a missing file (the first run, before anything has been saved) by returning an empty list, and a damaged file by saying so and starting again. `ask_year` uses the pattern from `ask_int`. The rest of the program doesn't need to know about JSON at all: it calls `load_books` and `save_books`. If you later change how books are stored, as Chapter 23 does with a database, only those two functions change.

Try It

1. Run `library_v4.py` twice, adding a book each time. Open `library.json` in a text editor and look at it.
2. Damage `library.json` by deleting a bracket, and run the program again. Then put the bracket back.
3. In `exceptions.py`, call `safe_divide("10", 2)`. Which exception is it, and why isn't it caught?

Exercises

1. **File statistics.** Write `file_stats(path)`, which returns a tuple of the number of lines, words and characters in a text file.
2. **Average rating.** Write `average_rating(csv_path)`, which returns the average of the `rating` column in a CSV file like `books.csv`.

3. **Safe division.** Write `divide(a_text, b_text)`, which divides two numbers given as strings and returns the result as a string, or a friendly message for text that isn't a number and for division by zero.

Key Takeaways

- `Path.read_text` and `write_text` read and write whole files; `with open(...)` handles files line by line and closes them safely. Always give `encoding="utf-8"`.
- Use the `csv` module for spreadsheets and the `json` module for nested data and settings.
- `try` and `except` handle exceptions you expect; `else` runs when there was none and `finally` always runs. Catch specific exceptions only.
- `raise` reports a problem to the caller. Keep file handling in a few functions so that the rest of the program doesn't depend on the format.

CHAPTER 11 · INTERMEDIATE

Modules and the Standard Library

As programs grow, one file isn't enough. A **module** is simply a Python file whose functions and variables other files can use. Python comes with hundreds of modules, called the **standard library**, which handle dates, random numbers, statistics, files and folders, command-line arguments and much more. People often say Python comes with "batteries included". In this chapter you'll write your own module, take a tour of the most useful standard modules, and build two small tools: a folder tidier and a command-line program.

Importing a Module

Any `.py` file is a module. Here's a small one, `shapes.py`:

```
"""A small module of geometry helpers."""

import math

UNIT = "cm"


def circle_area(radius):
    return math.pi * radius**2


def rectangle_area(width, height):
    return width * height


if __name__ == "__main__":
```

```
    # This runs only when you run shapes.py itself, not on import.
    print("Testing shapes.py")
    print(f"{circle_area(1):.4f}")
```

A second file in the same folder can use it:

```
import shapes
from shapes import rectangle_area

print(f"Circle: {shapes.circle_area(2):.2f} {shapes.UNIT}²")
print(f"Rectangle: {rectangle_area(3, 4)} {shapes.UNIT}²")
```

```
Circle: 12.57 cm²
Rectangle: 12 cm²
```

There are two forms of import:

- `import shapes` runs `shapes.py` once and makes its contents available with a prefix: `shapes.circle_area`, `shapes.UNIT`. The prefix makes clear where each name comes from.
- `from shapes import rectangle_area` makes one name available directly, without a prefix. It's shorter, but readers have to look at the imports to see where `rectangle_area` comes from.

You can also rename an import: `import statistics as stats`. Avoid `from shapes import *`, which imports everything; it makes code hard to follow and can silently replace names you already have. Put imports at the top of the file.

Scripts and Modules: `__name__`

When `use_shapes.py` imports `shapes`, nothing is printed, even though `shapes.py` ends with two `print` calls. That's because of the last block in `shapes.py`. Python sets a special variable, `__name__`, to `"__main__"` in the file you run, and to the module's name in a file that's imported. So the code under `if __name__ == "__main__":` runs only when you run `shapes.py` directly:

```
Testing shapes.py
3.1416
```

This lets a file be both a useful module and a program you can run to try it out. You'll see this pattern in most Python files, including the exercise solutions in this book.

A Tour of the Standard Library

These modules come with every Python installation:

```
import math
import random
import statistics
from collections import Counter, defaultdict
from decimal import Decimal

print(math.sqrt(2), math.floor(7.8), math.ceil(7.2), math.pi)

random.seed(1)  # same "random" results every run
print(random.randint(1, 6), random.choice(["red", "green", "blue"]))
deck = list(range(1, 11))
random.shuffle(deck)
print(deck)

marks = [72, 88, 95, 61, 79, 88]
print(
    "mean",
    statistics.mean(marks),
    "median",
    statistics.median(marks),
    "mode",
    statistics.mode(marks),
)

words = "the cat and the hat and the bat".split()
print(Counter(words).most_common(2))

by_letter = defaultdict(list)
for word in ["apple", "avocado", "banana", "blueberry", "cherry"]:
    by_letter[word[0]].append(word)
print(dict(by_letter))

# Decimal: exact decimal arithmetic, for money
print(0.1 + 0.2, Decimal("0.10") + Decimal("0.20"))
price = Decimal("19.99")
print(price * 3, (price * Decimal("1.08")).quantize(Decimal("0.01")))
```

```
1.4142135623730951 7 8 3.141592653589793
2 blue
[8, 3, 1, 9, 6, 7, 4, 10, 5, 2]
mean 80.5 median 83.5 mode 88
[('the', 3), ('and', 2)]
{'a': ['apple', 'avocado'], 'b': ['banana', 'blueberry'], 'c': ['cherry']}
0.30000000000000004 0.30
59.97 21.59
```

- **math** has mathematical functions and constants: square roots, rounding down (`floor`) and up (`ceil`), trigonometry and `pi`.
- **random** makes random choices. `random.seed(1)` makes the sequence repeatable, as in Chapter 5. `choice` picks one item from a list and `shuffle` mixes a list in place.
- **statistics** calculates the mean (average), median (middle value) and mode (most common value), and more.
- **collections** has specialized containers. A `Counter` counts things for you, replacing the `get` pattern from Chapter 7, and `most_common(2)` gives the top two. A `defaultdict(list)` creates an empty list the first time you use a new key, so you can append straight away.
- **decimal** does exact decimal arithmetic, which is what you want for money. Create decimals from strings, like `Decimal("19.99")`, and `quantize` rounds to a given number of places.

Dates and Times

The `datetime` module handles dates, times and the differences between them:

```
from datetime import date, datetime, timedelta

today = date(2026, 10, 15)  # a fixed date, so the output matches the book
deadline = date(2026, 12, 24)
print("Days to go:", (deadline - today).days)
print("In two weeks:", today + timedelta(weeks=2))
print(today.strftime("%A %d %B %Y"))

meeting = datetime(2026, 10, 15, 14, 30)
print(meeting.isoformat(), "->", meeting.strftime("%H:%M on %d/%m"))
parsed = datetime.strptime("2026-11-02 09:15", "%Y-%m-%d %H:%M")
print("Parsed:", parsed, "| weekday:", parsed.strftime("%a"))

# For the real date and time, use date.today() and datetime.now().
```

```
Days to go: 70
In two weeks: 2026-10-29
Thursday 15 October 2026
2026-10-15T14:30:00 -> 14:30 on 15/10
Parsed: 2026-11-02 09:15:00 | weekday: Mon
```

Subtracting two dates gives a `timedelta`, whose `days` is the number of days between them. Adding a `timedelta` moves a date forward. `strftime` ("string

format time") formats a date with codes such as `%A` for the weekday name, `%d` for the day, `%B` for the month name and `%Y` for the four-digit year; `strptime` ("string parse time") reads a date from text in a given format. `isoformat` gives the international standard format, `2026-10-15T14:30:00`, which sorts correctly and is the best choice for files.

The program uses fixed dates so that its output always matches the book. In your own programs, `date.today()` and `datetime.now()` give the current date and time.

Project: Tidy a Folder

Everybody has a downloads folder full of mixed files. This program sorts a folder's files into subfolders by type:

```
# Sort the files in a folder into sub-folders by type.
from pathlib import Path

FOLDERS = {
    ".jpg": "images",
    ".png": "images",
    ".pdf": "documents",
    ".txt": "documents",
    ".xlsx": "documents",
    ".mp3": "music",
}

folder = Path("files_to_tidy")
for path in sorted(folder.iterdir()):
    if path.is_file():
        target = folder / FOLDERS.get(path.suffix.lower(), "other")
        target.mkdir(exist_ok=True)
        path.rename(target / path.name)
        print(f"{path.name:12} -> {target.name}/")

print()
for path in sorted(folder.rglob("*")):
    print(path.relative_to(folder))
```

```
budget.xlsx  -> documents/
notes.txt    -> documents/
photo1.jpg   -> images/
photo2.jpg   -> images/
report.pdf   -> documents/
song.mp3     -> music/

documents
documents/budget.xlsx
documents/notes.txt
documents/report.pdf
```

```
images
images/photo1.jpg
images/photo2.jpg
music
music/song.mp3
```

The `/` operator joins paths: `folder / "images"` is the `images` folder inside `files_to_tidy`, written correctly for your operating system. `iterdir` lists what's in a folder, `suffix` gives a file's extension, `mkdir(exist_ok=True)` creates a folder unless it's already there, and `rename` moves a file. Finally, `rglob("*")` lists everything in the folder and all its subfolders. On Windows, the paths print with backslashes instead of forward slashes.

> **Warning:** Programs that move, rename or delete files can do a lot of damage very quickly. Try them on a copy of a folder first, or make them print what they would do before they do it. Chapter 26 shows how to add a "dry run" option.

Project: A Command-Line Tool

Many useful programs take their settings from the command line instead of asking questions. The `argparse` module reads command-line arguments, checks them, and writes the help text for you:

```
import argparse

parser = argparse.ArgumentParser(
    description="Greet someone, politely or loudly."
)
parser.add_argument("name", help="who to greet")
parser.add_argument(
    "--times", type=int, default=1, help="how many greetings"
)
parser.add_argument(
    "--shout", action="store_true", help="greet in capitals"
)
args = parser.parse_args()

for _ in range(args.times):
    message = f"Hello, {args.name}!"
    print(message.upper() if args.shout else message)
```

Run with `python greet_cli.py Zara --times 2 --shout`, it prints:

```
HELLO, ZARA!
HELLO, ZARA!
```

`name` is a **positional argument**: required, and identified by its position. `--times` and `--shout` are **options**: they're optional and identified by name. `type=int` converts the text to a number (and gives a clear error if it isn't one), and `action="store_true"` makes `--shout` a switch that's `True` when present. Run `python greet_cli.py --help` and `argparse` prints a usage summary built from the descriptions; run it without a name and it explains what's missing.

Finding More

The standard library is documented at docs.python.org, under "Library Reference". When you need to do something common, such as compressing files, sending email, reading web pages or working with times in other time zones, check there first: it's often already done. Beyond the standard library, the **Python Package Index** (PyPI) has hundreds of thousands of packages written by others. Chapter 17 shows how to install them safely.

Try It

1. Run `python shapes.py`, then `python use_shapes.py`. Explain why "Testing shapes.py" appears only once.
2. Use `datetime` to calculate how many days old you are, using your real birthday and `date.today()`.
3. Run `greet_cli.py` with `--help`, then with `--times two`. Read the messages.

Exercises

1. **Dice.** Write `roll_dice(rolls, seed=None)`, which rolls two dice `rolls` times and returns a `Counter` of the totals. Print a bar of `#` signs for each total from 2 to 12. Which total is most common? (Hint:

`random.Random(seed)` makes a separate random number generator, so a seed doesn't affect the rest of the program.)

2. **Days between.** Write `days_between(a, b)`, which takes two dates as `"YYYY-MM-DD"` strings and returns the number of days between them, whichever comes first. (Hint: `date.fromisoformat`.)

Key Takeaways

- Any `.py` file is a module. `import module` gives access with a prefix; `from module import name` imports one name.
- Code under `if __name__ == "__main__":` runs only when the file is run directly, not when it's imported.
- The standard library covers math, randomness, statistics, counting, exact decimals, dates, files and folders, and command-line arguments, among much else.
- `pathlib` paths join with `/` and can be listed, created and moved. `argparse` turns command-line arguments into a tidy object, with help text for free.

PART III

Intermediate Python

CHAPTER 12 · INTERMEDIATE

Classes and Objects

In Pocket Library, a book is a dictionary, and the functions that work with books are separate from it. That works, but nothing stops a typo like `book["auther"]`, and nothing tells you which functions belong with which data. **Classes** bring data and the functions that work on it together. A class describes a kind of thing; an **object** is one particular thing of that kind. In this chapter you'll write classes, learn the vocabulary that comes with them, and rebuild Pocket Library with a `Book` class and a `Library` class.

Your First Class

Every value in Python is already an object: `"hello"` is an object of class `str`, and `[1, 2]` is an object of class `list`. That's why they have methods, like `"hello".upper()`. Writing a class lets you create your own kinds of object:

```
class BankAccount:
    """A simple bank account that won't let the balance go below zero."""

    def __init__(self, owner, balance=0):
        self.owner = owner
        self.balance = balance

    def deposit(self, amount):
        if amount <= 0:
            raise ValueError("deposit must be positive")
        self.balance += amount

    def withdraw(self, amount):
        if amount > self.balance:
            raise ValueError(f"only {self.balance} available")
        self.balance -= amount
```

```
    def describe(self):
        return f"{self.owner}: balance {self.balance}"


meera = BankAccount("Meera", 100)
tom = BankAccount("Tom")

meera.deposit(50)
tom.deposit(20)
print(meera.describe())
print(tom.describe())

try:
    tom.withdraw(500)
except ValueError as error:
    print("Withdrawal refused:", error)
print(tom.describe())
```

```
Meera: balance 150
Tom: balance 20
Withdrawal refused: only 20 available
Tom: balance 20
```

- `class BankAccount:` starts the definition. Class names use **CamelCase**: capital letters at the start of each word, no underscores.
- Functions defined inside a class are its **methods**.
- `__init__` is a special method that Python calls when you create an object, to set it up. Its name has two underscores on each side; Python programmers call such names **dunder** (double underscore) names.
- Calling the class, as in `BankAccount("Meera", 100)`, creates a new object, an **instance** of the class, and runs `__init__` with the arguments.
- `self` is the object the method is working on. Python passes it automatically: `meera.deposit(50)` calls `deposit` with `self` set to `meera` and `amount` set to 50.
- `self.owner = owner` creates an **attribute**: a variable that belongs to the object. Each account has its own `owner` and `balance`.

The class enforces its own rules. `withdraw` refuses to take more than the balance, by raising an exception, and `deposit` refuses amounts that aren't positive. Code that uses the class doesn't have to remember those rules, because the class can't be put into an invalid state through its methods.

Objects in the Shell

```
>>> class Point:
...     def __init__(self, x, y):
...         self.x = x
...         self.y = y
...
>>> p = Point(2, 5)
>>> p.x, p.y
(2, 5)
>>> p.x = 10
>>> p.x
10
>>> type(p).__name__
'Point'
>>> isinstance(p, Point)
True
>>> q = Point(2, 5)
>>> p == q
False
>>> isinstance("text", str), isinstance(3, (int, float))
(True, True)
```

A class can be defined in the shell, too: the `...` prompt means Python is waiting for the rest of the block, and an empty line ends it. You read and change attributes with a dot. `isinstance` checks whether a value is an instance of a class, or of any class in a tuple. And two separate objects with the same attributes are not equal: == on objects of your own classes compares whether they're the same object, unless you tell Python otherwise. Chapter 13 shows how.

Class Attributes

Attributes created on `self` belong to one object. Attributes defined directly in the class body belong to the class, and are shared by all its instances:

```
class Ticket:
    price = 12.0  # a class attribute, shared by every ticket
    sold = 0

    def __init__(self, seat):
        self.seat = seat  # an instance attribute, one per ticket
        Ticket.sold += 1


a = Ticket("A1")
b = Ticket("A2")
print(a.seat, b.seat, "| price:", a.price, "| sold:", Ticket.sold)

Ticket.price = 14.0  # changes the price for every ticket
```

```
print(a.price, b.price)
```

```
A1 A2 | price: 12.0 | sold: 2
14.0 14.0
```

Each ticket has its own `seat`, but they share `price` and `sold`. Changing `Ticket.price` changes it for every ticket. Use class attributes for constants and shared settings; for anything that differs between objects, use instance attributes set in `__init__`.

> **Warning:** Never use a list or dictionary as a class attribute when each object needs its own, as in `books = []` in the class body. All instances would share one list. Create it in `__init__` instead: `self.books = []`.

Why Classes?

A class bundles three things:

- **State**: the data each object holds, in its attributes.
- **Behavior**: what the object can do, in its methods.
- **Rules**: checks in the methods that keep the state valid.

This makes programs easier to understand: `account.withdraw(20)` says what it does, and all the code about accounts is in one place. It also makes them easier to change, because the rest of the program uses the methods without needing to know how the data is stored inside.

Not everything needs a class, though. A function is the right tool for a calculation, and a dictionary is fine for simple data that's passed around. Reach for a class when you have data and operations that belong together, especially when the data has rules.

Project: Pocket Library, Version 5

Version 5 has a `Book` class for one book and a `Library` class for the collection, including saving and loading:

```
# Pocket Library, version 5: a Book class and a Library class.
import json
from pathlib import Path


class Book:
    def __init__(self, title, author, year, read=False):
        self.title = title
        self.author = author
        self.year = year
        self.read = read

    def mark_read(self):
        self.read = True

    def describe(self):
        mark = "x" if self.read else " "
        return f"[{mark}] {self.title} by {self.author} ({self.year})"

    def to_dict(self):
        return {
            "title": self.title,
            "author": self.author,
            "year": self.year,
            "read": self.read,
        }


class Library:
    def __init__(self, path):
        self.path = Path(path)
        self.books = []

    def add(self, book):
        self.books.append(book)

    def find(self, word):
        found = []
        for book in self.books:
            if word.lower() in book.title.lower():
                found.append(book)
        return found

    def unread(self):
        found = []
        for book in self.books:
            if not book.read:
                found.append(book)
        return found

    def save(self):
        data = []
        for book in self.books:
```

```
            data.append(book.to_dict())
        self.path.write_text(json.dumps(data, indent=2), encoding="utf-8")

    def load(self):
        self.books = []
        for item in json.loads(self.path.read_text(encoding="utf-8")):
            self.books.append(Book(**item))


library = Library("my_books.json")
library.add(Book("The Hobbit", "J. R. R. Tolkien", 1937, read=True))
library.add(Book("Dune", "Frank Herbert", 1965))
library.add(Book("Dune Messiah", "Frank Herbert", 1969))
library.save()

again = Library("my_books.json")
again.load()
for book in again.find("dune"):
    print(book.describe())
again.find("dune")[0].mark_read()
print("Unread:", len(again.unread()))
for book in again.books:
    print(book.describe())
```

```
[ ] Dune by Frank Herbert (1965)
[ ] Dune Messiah by Frank Herbert (1969)
Unread: 1
[x] The Hobbit by J. R. R. Tolkien (1937)
[x] Dune by Frank Herbert (1965)
[ ] Dune Messiah by Frank Herbert (1969)
```

The program saves three books to `my_books.json`, loads them into a new `Library` object, finds the two books with "dune" in the title, marks the first one as read, and lists everything.

Look at how the code reads now. `again.find("dune")[0].mark_read()` says exactly what it does. Each class looks after its own details: `Book` knows how to describe itself and how to turn itself into a dictionary for saving, and `Library` knows how to find, save and load books. `Book.to_dict` and `Library.load` are opposites: one turns a book into plain data for JSON, the other turns plain data back into books.

`Book(**item)` is a shortcut worth knowing. Two stars before a dictionary in a call pass its key-value pairs as keyword arguments, so `Book(**{"title": "Dune", "author": "Frank Herbert", "year": 1965, "read": False})` is the same as `Book(title="Dune", author="Frank Herbert", year=1965, read=False)`. It works because the dictionary keys match the parameter names of `__init__`.

> **Note:** Version 5 still has room for improvement. `Book` needs a lot of repetitive code just to store four values, and printing a `Book` object directly shows something unhelpful like `<__main__.Book object at 0x...>`. The next chapter solves both problems.

Try It

1. Add a `transfer(self, other, amount)` method to `BankAccount` that withdraws from one account and deposits into another. What should happen if the withdrawal fails?
2. Print a `Book` object directly with `print(book)`. What do you see?
3. Add a `by_author(self, name)` method to `Library` and use it.

Exercises

1. **Rectangle.** Write a `Rectangle` class with `width` and `height`, and methods `area`, `perimeter` and `is_square`.
2. **Tally counter.** Write a `Tally` class with a `count` that starts at 0, a `click(times=1)` method that adds to it and returns the new count, and a `reset` method.
3. **Thermostat.** Write a `Thermostat` class with a target temperature and allowed limits (5 to 30 degrees by default). Its `set_target(value)` method should store the value but keep it within the limits, so `set_target(42)` sets 30. (Hint: `min(max(value, low), high)`.)

Key Takeaways

- A class describes a kind of object; calling it creates an instance and runs `__init__`.
- Methods are functions inside a class. Their first parameter, `self`, is the object they work on; attributes set on `self` belong to that object.
- Class attributes are shared by all instances. Create mutable attributes like lists in `__init__`.

- Classes keep data, behavior and rules together, which makes code easier to read and change.

CHAPTER 13 · INTERMEDIATE

Inheritance, Dataclasses and Special Methods

Chapter 12 ended with two problems: the `Book` class needed a lot of repetitive code just to hold four values, and its objects printed as something like `<__main__.Book object at 0x...>`. This chapter solves both, and goes further. You'll learn **inheritance**, which lets one class build on another; **special methods**, which let your objects work with `+`, `==`, `len`, `for` and `print`; **dataclasses**, which write the routine code for you; and **properties**, which check values as they're set. Pocket Library becomes a collection that works like Python's own.

Inheritance

Different kinds of shape have a lot in common: a name, an area, a description. Inheritance lets you write the common parts once, in a **base class**, and the differences in **subclasses**:

```
import math


class Shape:
    def __init__(self, name):
        self.name = name

    def area(self):
        raise NotImplementedError("each shape must define area()")

    def describe(self):
        return f"{self.name} with area {self.area():.2f}"
```

```
class Circle(Shape):
    def __init__(self, radius):
        super().__init__("circle")
        self.radius = radius

    def area(self):
        return math.pi * self.radius**2


class Rectangle(Shape):
    def __init__(self, width, height):
        super().__init__("rectangle")
        self.width = width
        self.height = height

    def area(self):
        return self.width * self.height


class Square(Rectangle):
    def __init__(self, side):
        super().__init__(side, side)
        self.name = "square"


shapes = [Circle(1), Rectangle(2, 3.5), Square(4)]
for shape in shapes:
    print(shape.describe())

print(isinstance(Square(1), Rectangle), issubclass(Circle, Shape))
```

```
circle with area 3.14
rectangle with area 7.00
square with area 16.00
True True
```

`class Circle(Shape)` makes `Circle` a subclass of `Shape`: it **inherits** all of `Shape`'s methods, and can add new ones or **override** existing ones with its own versions. `describe` is written once in `Shape`, and calls `self.area()`, which runs the version for whatever kind of shape `self` actually is. That's why one loop can describe three different shapes. This idea, one method call that does the right thing for each type, is called **polymorphism**.

`super().__init__(...)` calls the base class's `__init__`, so that the shared set-up (here, storing the name) is done in one place. `Square` inherits from `Rectangle` and simply passes the same value twice. `Shape.area` raises `NotImplementedError` as a reminder that every subclass must provide its own.

A subclass is a special kind of its base class, so `isinstance(Square(1), Rectangle)` is `True`. A good test for inheritance is whether you can say "a square **is a** rectangle". If you'd instead say "a library **has** books", use an attribute, not inheritance.

> **Tip:** Keep inheritance shallow. One or two levels are easy to follow; five levels, where a method might be defined in any of them, are not. Many experienced programmers prefer small classes that hold other objects as attributes.

Special Methods

Python's operators and built-in functions call special methods, with dunder names, on your objects. `a + b` calls `a.__add__(b)`, `a == b` calls `a.__eq__(b)`, `len(x)` calls `x.__len__()`, and so on. Define them, and your objects work with Python's syntax:

```
class Money:
    """An amount of money in one currency, stored in cents."""

    def __init__(self, amount, currency="EUR"):
        self.cents = round(amount * 100)
        self.currency = currency

    def __repr__(self):
        return f"Money({self.cents / 100!r}, {self.currency!r})"

    def __str__(self):
        return f"{self.cents / 100:,.2f} {self.currency}"

    def __add__(self, other):
        if self.currency != other.currency:
            raise ValueError("can't add different currencies")
        return Money((self.cents + other.cents) / 100, self.currency)

    def __eq__(self, other):
        return (self.cents, self.currency) == (other.cents, other.currency)

    def __lt__(self, other):
        return self.cents < other.cents

    def __bool__(self):
        return self.cents != 0


coffee = Money(3.20)
cake = Money(4.75)
print(coffee + cake)
print(repr(coffee + cake))
```

```
print(Money(0.1) + Money(0.2) == Money(0.3))
print(sorted([cake, coffee]))
print(bool(Money(0)), bool(coffee))
```

```
7.95 EUR
Money(7.95, 'EUR')
True
[Money(3.2, 'EUR'), Money(4.75, 'EUR')]
False True
```

Method	Called by
`__str__`	`print(x)` and `str(x)`: a readable version for users
`__repr__`	`repr(x)`, the shell, and inside lists: an unambiguous version for programmers, ideally looking like the code that creates the object
`__add__`	`x + y`
`__eq__`	`x == y`
`__lt__`	`x < y`, and therefore `sorted`
`__bool__`	`if x:` and `bool(x)`
`__len__`	`len(x)`
`__contains__`	`y in x`
`__iter__`	`for item in x`

`Money` stores amounts as whole cents, so `Money(0.1) + Money(0.2) == Money(0.3)` is `True`, unlike the floats in Chapter 2. Notice that `print(coffee + cake)` uses `__str__`, while the sorted list uses `__repr__` for each item. If you define only one, define `__repr__`; Python uses it for `str` too when there's no `__str__`.

Dataclasses

Many classes exist mainly to hold data. For those, the `dataclasses` module writes `__init__`, `__repr__` and `__eq__` for you:

```
from dataclasses import dataclass, field


@dataclass
class Book:
    title: str
    author: str
```

```
    year: int
    tags: list[str] = field(default_factory=list)


@dataclass(frozen=True, order=True)
class Version:
    major: int
    minor: int


dune = Book("Dune", "Frank Herbert", 1965)
dune.tags.append("novel")
print(dune)
print(dune == Book("Dune", "Frank Herbert", 1965, ["novel"]))

versions = sorted([Version(3, 15), Version(3, 9), Version(3, 12)])
print("Oldest:", versions[0])
print("Newest:", versions[-1])
v = Version(3, 15)
v.minor = 16
```

```
Book(title='Dune', author='Frank Herbert', year=1965, tags=['novel'])
True
Oldest: Version(major=3, minor=9)
Newest: Version(major=3, minor=15)
Traceback (most recent call last):
  File "dataclasses_demo.py", line 27, in <module>
    v.minor = 16
    ^^^^^^^
  File "<string>", line 32, in __setattr__
dataclasses.FrozenInstanceError: cannot assign to field 'minor'
```

The `@dataclass` line above the class is a **decorator**: it takes the class and adds methods to it. (Chapter 18 shows how decorators work.) Each line in the class body names a **field** and its type, such as `title: str`. These **type hints** document what each field should hold; Chapter 19 covers them in detail. Python itself doesn't check them.

The dataclass gives you, for free, an `__init__` that takes the fields in order, a `__repr__` that shows them, and an `__eq__` that compares them, so two books with the same fields are equal. Fields with defaults must come after fields without. For a list default, use `field(default_factory=list)`, which creates a new list for each object, for the reason given in the warning in Chapter 12.

`frozen=True` makes objects that can't be changed after they're created, like tuples, and `order=True` adds `<` and the other comparisons, comparing fields in order. That's why the versions sort correctly as numbers: 3.9 comes before 3.12, which wouldn't happen if they were compared as text. The last line tries to change a frozen object and fails.

Properties

Sometimes an attribute needs checking whenever it's set, or should be calculated from other attributes. A **property** looks like an attribute from the outside but runs a method:

```
class Temperature:
    def __init__(self, celsius):
        self.celsius = celsius  # goes through the setter below

    @property
    def celsius(self):
        return self._celsius

    @celsius.setter
    def celsius(self, value):
        if value < -273.15:
            raise ValueError("below absolute zero")
        self._celsius = value

    @property
    def fahrenheit(self):
        return self._celsius * 9 / 5 + 32


t = Temperature(25)
print(t.celsius, t.fahrenheit)
t.celsius = 100
print(t.fahrenheit)
try:
    t.celsius = -300
except ValueError as error:
    print("Refused:", error)
```

```
25 77.0
212.0
Refused: below absolute zero
```

`@property` turns the `celsius` method into something you read without parentheses: `t.celsius`. `@celsius.setter` defines what happens when you assign to it: here, a check, then storing the value in `_celsius`. `fahrenheit` has no setter, so it's read-only and always up to date. The underscore at the start of `_celsius` is a convention meaning "internal: use the property instead".

Start with plain attributes. If you later need a check or a calculation, you can turn an attribute into a property without changing any code that uses it.

Project: Pocket Library, Version 6

Version 6 makes `Book` a dataclass and gives `Library` special methods, so that it works like a built-in collection:

```
# Pocket Library, version 6: a dataclass, and a Library that works
# like a collection.
from dataclasses import dataclass


@dataclass
class Book:
    title: str
    author: str
    year: int
    read: bool = False


class Library:
    def __init__(self, books=None):
        self._books = list(books or [])

    def add(self, book):
        if book in self:
            raise ValueError(f"already in the library: {book.title}")
        self._books.append(book)

    def __len__(self):
        return len(self._books)

    def __iter__(self):
        return iter(self._books)

    def __contains__(self, book):
        for b in self._books:
            if b.title.lower() == book.title.lower():
                return True
        return False

    def __repr__(self):
        return f"Library({len(self)} books)"


library = Library(
    [
        Book("The Hobbit", "J. R. R. Tolkien", 1937, True),
        Book("Dune", "Frank Herbert", 1965),
    ]
)
library.add(Book("Educated", "Tara Westover", 2018))
print(library, "| len:", len(library))
print(Book("DUNE", "?", 0) in library)
for book in library:
    print(f"- {book.title} ({book.year})")
try:
    library.add(Book("dune", "Frank Herbert", 1965))
except ValueError as error:
```

```
    print("Error:", error)
```

```
Library(3 books) | len: 3
True
- The Hobbit (1937)
- Dune (1965)
- Educated (2018)
Error: already in the library: dune
```

`len(library)`, `book in library` and `for book in library` all work, because `Library` defines `__len__`, `__contains__` and `__iter__`. `__iter__` simply hands the job to the list inside, with `iter(self._books)`; Chapter 14 explains iterators. The list is called `_books`, so other code is encouraged to use the library's methods instead of reaching inside. And `add` refuses duplicates by using the class's own `in` check.

`list(books or [])` in `__init__` is a common idiom: `books or []` gives an empty list when no books are passed, and `list(...)` makes a copy, so the library doesn't share a list with the caller.

Try It

1. Add a `Triangle(base, height)` subclass of `Shape` and add one to the list in `inheritance.py`.
2. Remove `__str__` from `Money` and print `coffee`. Which method does Python use now?
3. Create a `Book` dataclass object without a year. Read the error message.

Exercises

1. **Employees.** Write an `Employee` class with a name and monthly salary and a `yearly_cost` method, and a `Manager` subclass that adds a yearly bonus to the cost. Use `super()` in both `__init__` and `yearly_cost`.
2. **Fractions.** Write a `Fraction` class that stores fractions in lowest terms (hint: `math.gcd`), supports `+` and `==`, and prints as `5/6`. `Fraction(2, 4) == Fraction(1, 2)` should be `True`.
3. **Points.** Write a frozen dataclass `Point` with `x` and `y`, and a `distance_to` method. (Hint: `math.hypot`.)

Key Takeaways

- A subclass inherits its base class's methods and can override them; `super()` calls the base class's version. Use inheritance for "is a" relationships.
- Special methods such as `__str__`, `__repr__`, `__eq__`, `__add__`, `__len__`, `__contains__` and `__iter__` make your objects work with Python's operators and built-ins.
- `@dataclass` writes `__init__`, `__repr__` and `__eq__` for classes that mainly hold data; `frozen=True` makes them unchangeable and `order=True` sortable.
- Properties look like attributes but run code, for checks and calculated values.

CHAPTER 14 · INTERMEDIATE

Comprehensions, Iterators and Generators

Loops that build lists are so common that Python has a compact way to write them: **comprehensions**. Behind every `for` loop is a simple protocol, **iteration**, which you can use in your own classes. And **generators** let you produce values one at a time, as they're needed, instead of building a whole list in memory. These three ideas make Python code shorter, clearer and, for large data, much more efficient.

List Comprehensions

In earlier chapters you built lists like this:

```
to_read = []
for book in books:
    if not book["read"]:
        to_read.append(book["title"])
```

A **list comprehension** says the same in one line: `[book["title"] for book in books if not book["read"]]`. Read it as "the title, for each book in books, if it isn't read". Here are more:

```
numbers = [3, 8, 1, 12, 7, 6]

squares = [n * n for n in numbers]
evens = [n for n in numbers if n % 2 == 0]
labels = ["even" if n % 2 == 0 else "odd" for n in numbers]
print(squares)
```

```
print(evens)
print(labels)

words = ["apple", "Banana", "cherry", "date"]
lengths = {word: len(word) for word in words}
initials = {word[0].lower() for word in words}
print(lengths)
print(sorted(initials))

grid = [[row * 3 + col for col in range(3)] for row in range(3)]
print(grid)
flat = [value for row in grid for value in row]
print(flat)
```

```
[9, 64, 1, 144, 49, 36]
[8, 12, 6]
['odd', 'even', 'odd', 'even', 'odd', 'even']
{'apple': 5, 'Banana': 6, 'cherry': 6, 'date': 4}
['a', 'b', 'c', 'd']
[[0, 1, 2], [3, 4, 5], [6, 7, 8]]
[0, 1, 2, 3, 4, 5, 6, 7, 8]
```

The pattern is always `[expression for item in iterable if condition]`, with the `if` part optional. The expression can be anything, including a conditional expression, as in `labels`. Notice the difference between the two uses of `if`: one after the `for` filters which items are included; a conditional expression before the `for` chooses what each item becomes.

With curly braces you get a **dictionary comprehension**, with a colon between key and value, or a **set comprehension**. Comprehensions can also be nested: `grid` builds a list of rows, and `flat` reads two `for` clauses in the same order as nested loops, giving every value in every row.

> **Tip:** Use a comprehension when it fits comfortably on one line and reads clearly. If you need several conditions, nested loops or side effects like printing, write a normal loop.

A Few More in the Shell

```
>>> [c.upper() for c in "hey"]
['H', 'E', 'Y']
>>> {n: n ** 2 for n in range(4)}
{0: 0, 1: 1, 2: 4, 3: 9}
>>> any(n > 10 for n in [3, 15, 7]), all(n > 1 for n in [3, 15, 7])
(True, True)
>>> squares = (n * n for n in range(3))
>>> squares
```

```
<generator object <genexpr> at 0x...>
>>> list(squares)
[0, 1, 4]
>>> list(squares)
[]
```

`any` is true if at least one value is true, and `all` if every value is. They take a **generator expression**: a comprehension in parentheses, which you'll meet properly below. When a generator expression is the only argument to a function, you can leave out its own parentheses.

The last three lines show something important. A generator expression doesn't build a list; it produces values when asked. `list(squares)` asks for all of them. Asking again gives nothing, because the generator is used up.

How for Loops Work

A `for` loop works with any **iterable**: anything that can produce an **iterator**. An iterator is an object that hands out values one at a time when you call `next` on it, and raises `StopIteration` when it has no more. The `for` loop does exactly that behind the scenes:

```
colors = ["red", "green", "blue"]
it = iter(colors)
print(next(it))
print(next(it))
print(next(it))
print(next(it, "no more colors"))  # a default instead of an error


class Countdown:
    """An iterator that counts down from start to 1."""

    def __init__(self, start):
        self.current = start

    def __iter__(self):
        return self

    def __next__(self):
        if self.current <= 0:
            raise StopIteration
        value = self.current
        self.current -= 1
        return value


for n in Countdown(3):
    print(n)
print(list(Countdown(5)))
```

```
red
green
blue
no more colors
3
2
1
[5, 4, 3, 2, 1]
```

`iter(colors)` gets an iterator for the list, and each `next(it)` returns the next value. `next(it, default)` returns the default instead of raising `StopIteration` at the end.

The `Countdown` class is an iterator of its own. It needs two special methods: `__iter__`, which returns the iterator (itself), and `__next__`, which returns the next value or raises `StopIteration`. With those, it works in a `for` loop and with `list`, like any built-in sequence. That's what `Library.__iter__` did in Chapter 13, more simply, by returning the iterator of the list inside.

Generators

Writing an iterator class is tedious. A **generator function** does the same job with much less code: it's an ordinary function that uses `yield` instead of `return`:

```
def countdown(start):
    while start > 0:
        yield start
        start -= 1


def fibonacci(limit):
    a, b = 0, 1
    while a <= limit:
        yield a
        a, b = b, a + b


def read_records(lines):
    """Turn 'name,score' lines into (name, score) pairs; skip bad ones."""
    for line in lines:
        name, _, score = line.partition(",")
        if score.strip().isdigit():
            yield name.strip(), int(score)


print(list(countdown(4)))
print(list(fibonacci(100)))
```

```
# No list of a million numbers is ever built in memory
total = sum(n * n for n in range(1, 1_000_001))
print(total)

data = ["asha, 91", "ben, absent", "chen, 78", "", "dev, 85"]
for name, score in read_records(data):
    print(name, score)

gen = countdown(2)
print(next(gen), next(gen))
print(next(gen, "finished"))
```

```
[4, 3, 2, 1]
[0, 1, 1, 2, 3, 5, 8, 13, 21, 34, 55, 89]
333333833333500000
asha 91
chen 78
dev 85
2 1
finished
```

Calling `countdown(4)` doesn't run the body. It returns a generator. Each time the generator is asked for a value, the body runs until it reaches `yield`, hands out that value, and **pauses**, keeping all its variables. The next request resumes from the same place. When the function ends, the generator stops.

Generators have three great strengths:

- **Memory.** `sum(n * n for n in range(1, 1_000_001))` adds a million squares without ever building a list of a million numbers. Each value exists only while it's being added.
- **Endless sequences.** A generator can produce values forever, and the caller takes as many as it needs. `fibonacci` stops at a limit, but it doesn't have to.
- **Pipelines.** `read_records` cleans data as it's read: it splits each line, skips bad ones, and yields tidy pairs. The code that uses it sees only good records, one at a time, however large the input. (`partition(",")` splits a string at the first comma into three parts: before, the comma, and after.)

itertools

The `itertools` module is a toolbox of iterator building blocks:

```
import itertools

print(list(itertools.islice(itertools.count(10, 5), 4)))
print(list(itertools.chain([1, 2], (3, 4), "ab")))
print(list(itertools.accumulate([5, 3, 8, 1])))
print(list(itertools.product("AB", [1, 2])))
print(list(itertools.combinations(["tea", "coffee", "juice"], 2)))


def region_of(row):
    return row[0]


sales = [
    ("north", 120),
    ("north", 80),
    ("south", 50),
    ("south", 70),
    ("east", 30),
]
for region, rows in itertools.groupby(sales, key=region_of):
    print(region, sum(amount for _, amount in rows))
```

```
[10, 15, 20, 25]
[1, 2, 3, 4, 'a', 'b']
[5, 8, 16, 17]
[('A', 1), ('A', 2), ('B', 1), ('B', 2)]
[('tea', 'coffee'), ('tea', 'juice'), ('coffee', 'juice')]
north 200
south 120
east 30
```

Function	What it produces
`count(start, step)`	numbers forever; `islice` takes the first few
`chain(a, b, ...)`	the items of several iterables, one after another
`accumulate(items)`	running totals
`product(a, b)`	every combination of one item from each
`combinations(items, n)`	every way to choose `n` items, ignoring order
`groupby(items, key)`	runs of consecutive items with the same key

`groupby` groups only **consecutive** items, so sort the data by the key first if it isn't already. `key=region_of` passes a function that tells `groupby` which part

of each row to group by, just as `key=` did for `sorted` in Chapter 7. Chapter 20 shows a shorter way to write such small functions.

Try It

1. Rewrite the "still to read" loop from `library_v2.py` as a list comprehension.
2. Use a dictionary comprehension to build `{1: 1, 2: 8, 3: 27, ...}` for the numbers 1 to 10 and their cubes.
3. Write a generator `evens()` that yields 0, 2, 4, ... forever, and use `itertools.islice` to print the first five.

Exercises

1. **Even squares.** Write `even_squares(numbers)`, which returns the squares of the even numbers, using a list comprehension.
2. **Word lengths.** Write `word_lengths(text)`, which returns a dictionary from each word, in lower case, to its length, using a dictionary comprehension.
3. **Chunks.** Write a generator `chunked(items, size)` that yields lists of `size` items at a time, with a shorter list at the end if needed. `list(chunked("abcdefg", 3))` should give `[['a', 'b', 'c'], ['d', 'e', 'f'], ['g']]`.

Key Takeaways

- List, dictionary and set comprehensions build collections in one readable line: `[expr for item in items if condition]`.
- A `for` loop calls `iter` once and `next` repeatedly until `StopIteration`. Classes with `__iter__` and `__next__` work in loops.
- Generator functions use `yield` to produce values one at a time, pausing in between. Generator expressions are comprehensions in parentheses.
- Generators save memory, handle endless sequences and make data pipelines. `itertools` adds counting, chaining, running totals, combinations and grouping.

CHAPTER 15 · INTERMEDIATE

Testing with pytest

How do you know your program works? So far, you've run it and looked at the output. That's fine for a short script, but it doesn't scale: every time you change something, you'd have to try every case again by hand, and sooner or later you'd forget one. **Automated tests** are small programs that check your code for you, in seconds, every time. In this chapter you'll write tests with **pytest**, the most popular testing tool for Python, see what a failing test tells you, and learn the habits that make testing worthwhile.

Installing pytest

pytest isn't part of the standard library, so you need to install it. The safe way is inside a **virtual environment**: a private folder of packages for one project, which keeps it separate from other projects and from your system's Python. Chapter 17 explains virtual environments properly; for now, follow these steps in your project folder.

Create the environment (once per project):

```
$ python -m venv .venv
```

Activate it (each time you open a new terminal). On Windows:

```
$ .venv\Scripts\activate
```

On macOS and Linux:

```
$ source .venv/bin/activate
```

Your prompt now starts with `(.venv)`. Install pytest:

```
$ python -m pip install pytest
```

The book's code repository (see the Introduction) has a `requirements.txt` file listing the exact versions the examples were tested with. `python -m pip install -r requirements.txt` installs them all, including the packages for later chapters.

Your First Tests

Here's a small module of text statistics, the kind of functions you wrote in Chapter 9:

```
"""Small text statistics functions, tested in test_textstats.py."""


def word_count(text):
    return len(text.split())


def average_word_length(text):
    words = text.split()
    if not words:
        return 0.0
    letters = sum(len(word.strip(".,!?;:")) for word in words)
    return letters / len(words)


def reading_time_minutes(text, words_per_minute=200):
    """Whole minutes to read text, rounded up; at least 1 for any text."""
    words = word_count(text)
    if words == 0:
        return 0
    return -(-words // words_per_minute)
```

`reading_time_minutes` uses a trick worth knowing: `-(-words // words_per_minute)` divides and rounds **up**, because `//` always rounds down and the two minus signs turn that into rounding up. 201 words at 200 words per minute is 2 minutes, not 1. (`math.ceil(words / words_per_minute)` does the same.)

Tests for it go in a file whose name starts with `test_`:

```
import pytest

from textstats import average_word_length, reading_time_minutes, word_count


def test_word_count_simple():
    assert word_count("the cat sat") == 3


def test_word_count_ignores_extra_spaces():
    assert word_count("  the   cat  ") == 2


def test_average_word_length_ignores_punctuation():
    assert average_word_length("Hi, you!") == 2.5


def test_average_word_length_of_empty_text():
    assert average_word_length("") == 0.0


@pytest.mark.parametrize(
    "words, minutes", [(0, 0), (1, 1), (200, 1), (201, 2), (1000, 5)]
)
def test_reading_time(words, minutes):
    text = "word " * words
    assert reading_time_minutes(text) == minutes
```

A test is a function whose name starts with `test_`, containing `assert` statements. `assert` checks that a condition is true; if it isn't, the test fails. Run pytest in the folder:

```
$ python -m pytest -q test_textstats.py
```

```
.........                                                                [100%]
9 passed in 0.01s
```

Each dot is a passing test. There are five test functions but nine tests, because `test_reading_time` is **parametrized**: `@pytest.mark.parametrize` runs it once for each pair of values in the list, five times in all. That's the easiest way to test many cases without writing many functions.

> **Note:** pytest prints how long the tests took. Your times will differ; the book always shows `0.01s`.

Choosing What to Test

Look at the cases in `test_textstats.py`. They aren't random:

- **A normal case**, such as three words, to show the function does its main job.
- **Edge cases**: empty text, extra spaces, punctuation. Bugs live at the edges.
- **Boundaries**: 200 words is exactly one minute and 201 words just over, so the test checks both sides of the line where the answer changes, and 0 words, where the rule is different.

Each test checks one thing and has a name that says what. When a test fails, its name tells you what's broken before you read any code.

When a Test Fails

Here's a function that applies a discount, and four tests for it:

```
def apply_discount(price, percent):
    """Return price reduced by percent, rounded to cents."""
    if not 0 <= percent <= 100:
        raise ValueError("percent must be between 0 and 100")
    return round(price * (1 - percent / 100), 2)
```

```
import pytest

from discount import apply_discount


def test_ten_percent_off():
    assert apply_discount(50.0, 10) == 45.0


def test_rounds_to_cents():
    assert apply_discount(19.99, 15) == 16.99


def test_rejects_bad_percent():
    with pytest.raises(ValueError):
        apply_discount(10.0, 120)


def test_full_discount():
    assert apply_discount(10.0, 100) == 1.0  # the test is wrong
```

```
...F                                                                    [100%]
================================= FAILURES =================================
_____________________________ test_full_discount _____________________________

    def test_full_discount():
>       assert apply_discount(10.0, 100) == 1.0  # the test is wrong
       ^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^
E       assert 0.0 == 1.0
E        +  where 0.0 = apply_discount(10.0, 100)

test_discount.py:20: AssertionError
========================= short test summary info ==========================
FAILED test_discount.py::test_full_discount - assert 0.0 == 1.0
1 failed, 3 passed in 0.01s
```

The `F` is a failure, and pytest explains it in detail: which test, the line, the values on both sides of the comparison (`0.0 == 1.0`) and where the left-hand value came from (`apply_discount(10.0, 100)`). Here, the code is right and the test is wrong: 100% off 10.00 is 0.00, not 1.00. That happens. A failing test means the code and the test disagree, and you have to decide which one is mistaken.

`pytest.raises` checks that code raises an exception. `test_rejects_bad_percent` passes only if `apply_discount(10.0, 120)` raises `ValueError`. Testing that bad input is rejected is as important as testing that good input works.

> **Warning:** Compare floats with care. `0.1 + 0.2 == 0.3` is `False`, as you saw in Chapter 2. `apply_discount` rounds to cents, which makes exact comparison safe here. For results that aren't rounded, use `pytest.approx`: `assert result == pytest.approx(0.3)`.

Fixtures

Many tests need something set up first: a file, a database, an object with some data. A **fixture** is a function that prepares it. A test asks for a fixture by naming it as a parameter:

```
import json

import pytest


def load_titles(path):
```

```
    return [
        book["title"]
        for book in json.loads(path.read_text(encoding="utf-8"))
    ]


@pytest.fixture
def library_file(tmp_path):
    """A temporary library.json, made fresh for each test that uses it."""
    path = tmp_path / "library.json"
    path.write_text(
        json.dumps([{"title": "Dune"}, {"title": "Emma"}]), encoding="utf-8"
    )
    return path


def test_load_titles(library_file):
    assert load_titles(library_file) == ["Dune", "Emma"]


def test_empty_library(tmp_path):
    path = tmp_path / "empty.json"
    path.write_text("[]", encoding="utf-8")
    assert load_titles(path) == []
```

```
..                                                                   [100%]
2 passed in 0.01s
```

`@pytest.fixture` marks `library_file` as a fixture. When pytest sees a test with a parameter called `library_file`, it calls the fixture and passes in the result. `tmp_path` is a fixture that pytest provides itself: a new, empty temporary folder for each test. Tests that use it never touch your real files, and never affect each other.

Testing Habits

- **Run the tests often**: after every change. They're only useful if they run.
- **Write a test for every bug you fix.** First write a test that fails because of the bug, then fix the code and watch the test pass. The bug can never come back unnoticed.
- **Keep tests fast and independent.** Each test should set up what it needs and not depend on other tests running first.
- **Test behavior, not details.** Test what a function returns for given inputs, not how it works inside, so you can improve the code without rewriting the tests.

- **Design for testing.** Functions that take inputs and return results are easy to test. Functions that ask questions with `input` or print their results are not. That's why this book keeps calculations separate from input and output.

Some programmers write the test before the code. This is called **test-driven development**. Even if you don't work that way, writing a test makes you decide exactly what a function should do, which often improves the design.

Try It

1. Fix the mistaken test in `test_discount.py` and run pytest again.
2. Add a parametrized test for `word_count` with four cases of your own.
3. Break `reading_time_minutes` on purpose by using `/` instead of the `-(-a // b)` trick. Which tests fail?

Exercises

1. **Leap years.** Turn your leap-year program from Chapter 4 into a function `is_leap_year(year)` and write a parametrized test with 2024, 2023, 1900 and 2000.
2. **FizzBuzz.** Turn FizzBuzz from Chapter 5 into a function `fizzbuzz(n)` that returns the text for one number, and test it with 1, 3, 5, 15 and 30.

Key Takeaways

- Tests are functions named `test_...` in files named `test_*.py`, using `assert`. Run them with `python -m pytest`.
- Install packages like pytest inside a virtual environment: `python -m venv .venv`, activate it, then `python -m pip install`.
- Test normal cases, edge cases and boundaries. `parametrize` runs one test with many inputs; `pytest.raises` checks exceptions.
- Fixtures prepare what tests need; `tmp_path` gives each test its own temporary folder. Write a test for every bug you fix.

CHAPTER 16 · INTERMEDIATE

Debugging, Logging and Reading Tracebacks

Every programmer spends a large part of their time finding out why a program doesn't do what they expected. That's **debugging**, and it's a skill you can learn like any other. In this chapter you'll read a long traceback with confidence, find a bug that produces no error at all, step through a running program with Python's debugger, and use **logging** to record what a program does without cluttering it with `print` calls.

Reading a Long Traceback

When an error happens deep inside a chain of function calls, the traceback shows the whole chain:

```
def parse_price(text):
    return float(text.replace("€", ""))


def order_total(lines):
    total = 0
    for line in lines:
        name, price = line.split(":")
        total += parse_price(price)
    return total


def print_receipt(lines):
    print(f"Total: {order_total(lines):.2f}")


print_receipt(["coffee: €3.20", "cake: €4.75", "tea: free"])
```

```
Traceback (most recent call last):
  File "deep_traceback.py", line 17, in <module>
    print_receipt(["coffee: €3.20", "cake: €4.75", "tea: free"])
    ~~~~~~~~~~~~~^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^
  File "deep_traceback.py", line 14, in print_receipt
    print(f"Total: {order_total(lines):.2f}")
                    ~~~~~~~~~~~^^^^^^^
  File "deep_traceback.py", line 9, in order_total
    total += parse_price(price)
             ~~~~~~~~~~~^^^^^^^
  File "deep_traceback.py", line 2, in parse_price
    return float(text.replace("€", ""))
           ~~~~~^^^^^^^^^^^^^^^^^^^^^^
ValueError: could not convert string to float: ' free'
```

Read it in this order:

1. **The last line**: the type of error and its message. Python couldn't convert `' free'` to a float. Notice the space at the start of the string; the message quotes the value exactly.
2. **The frames, from the bottom up.** Each frame is one function call, with the file, the line number and the function's name. The bottom frame is where the error happened: `float(...)` in `parse_price`. The one above it is the line that called `parse_price`, and so on, up to the line at the top of your program that started it all.
3. **Decide where the fix belongs.** The error happened in `parse_price`, but is the bug there? `parse_price` assumes it gets a price, and `order_total` passed it the text after the colon, without checking. The real question is what should happen when an item is free. The traceback tells you where the problem showed up; you decide where it should be fixed.

The ~~~^^^ markers under each line show which part of the line was running. They're especially useful when a line contains several calls.

Bugs Without Errors

The hardest bugs produce no error, only a wrong answer:

```
def average(numbers):
    total = 0
    # Bug: starts at 1, skipping the first number
    for i in range(1, len(numbers)):
        total += numbers[i]
    return total / len(numbers)
```

```
print(average([10, 20, 30]))  # we expect 20.0
```

```
16.666666666666668
```

When a result is wrong, work like a scientist:

1. **Reproduce it** with the smallest input that shows the problem. A list of three numbers is easier to reason about than a file of thousands.
2. **Know what you expect.** The average of 10, 20 and 30 is 20. The program says 16.67, which is 50 divided by 3: the total is missing 10, the first number.
3. **Form a hypothesis and test it.** "The loop skips the first item." Print `i` in the loop, or check `range(1, len(numbers))` in the shell. It starts at 1.
4. **Fix it, then check the fix**, preferably with a test, so the bug can't return.

The fixed version is shorter, because Python's `sum` does the work, and it handles the empty list, which would have divided by zero:

```
def average(numbers):
    if not numbers:
        raise ValueError("can't average an empty list")
    return sum(numbers) / len(numbers)


print(average([10, 20, 30]))
assert average([10, 20, 30]) == 20.0
assert average([5]) == 5.0
print("All checks passed.")
```

```
20.0
All checks passed.
```

The `assert` statements check the results right in the program. They're a lightweight form of the tests from Chapter 15.

> **Tip:** Explaining your code, line by line, to someone else, or even to a rubber duck on your desk, is a famously effective way to find bugs. Saying what each line does forces you to notice what it actually does, not what you meant it to do.

Print Debugging

The simplest debugging tool is `print`. Add `print(f"{i=} {total=}")` inside the loop, and you see each step. The `=` in an f-string, from Chapter 3, prints the name with the value. Print debugging is quick and works everywhere; just remember to remove the prints when you're done.

The Python Debugger

For harder problems, a **debugger** lets you pause a running program and look around. Python's debugger is called **pdb**, and the easiest way to start it is to put `breakpoint()` where you want to stop. Here's a program that should take 10% off each price:

```
def apply_discounts(prices, percent):
    result = []
    for price in prices:
        breakpoint()  # stop here and look around
        result.append(price - price * percent // 100)
    return result


print(apply_discounts([19.99, 5.00], 10))
```

It prints `[18.99, 5.0]`, but 10% off 19.99 is 17.99, and 10% off 5.00 is 4.50. Run it, and it stops at the breakpoint with a `(Pdb)` prompt, where you can type commands. Here's the session, with the commands that were typed after each prompt:

```
> debug_me.py(4)apply_discounts()
-> breakpoint()  # stop here and look around
(Pdb) p price
19.99
(Pdb) p percent
10
(Pdb) p price * percent // 100
1.0
(Pdb) c
> debug_me.py(4)apply_discounts()
-> breakpoint()  # stop here and look around
(Pdb) c
[18.99, 5.0]
```

`p` prints an expression using the program's variables at that moment. `price` and `percent` are as expected, but `price * percent // 100` is `1.0`, not `1.999`. There's the bug: `//` rounds down. It should be `/`. `c` continues until the

next breakpoint, which here is the next pass through the loop, and finally the program finishes.

The most useful pdb commands:

Command	What it does
`p expression`	Print the value of an expression
`n`	Run the next line (stepping over function calls)
`s`	Step into the function called on this line
`c`	Continue until the next breakpoint
`l`	List the code around the current line
`w`	Show where you are: the chain of calls
`q`	Quit the program

In a real terminal, the first line shows the full path to the file. Python 3.12 and older stop on the line **after** `breakpoint()` rather than on it; the commands work the same. Code editors such as Visual Studio Code have a graphical debugger with the same abilities: click in the margin to set a breakpoint, and hover over variables to see their values.

> **Note:** Remove `breakpoint()` calls when you've found the bug. A program that stops and waits for debugger commands is very confusing for its users.

Logging

Print statements are for you, while you work. **Logging** is for the life of a program: a record of what it did, which you can turn up when investigating a problem and down when everything's fine. The `logging` module is part of the standard library:

```
import logging

logging.basicConfig(
    level=logging.DEBUG, format="%(levelname)-8s %(name)s: %(message)s"
)
log = logging.getLogger("library")


def borrow(stock, title):
```

```
    log.debug("borrow(%r) called; stock=%r", title, stock)
    if stock.get(title, 0) == 0:
        log.warning("%s is not available", title)
        return False
    stock[title] -= 1
    log.info("lent out %s; %d left", title, stock[title])
    return True


stock = {"Dune": 1}
borrow(stock, "Dune")
borrow(stock, "Dune")
# From now on, show only warnings and worse
logging.getLogger().setLevel(logging.WARNING)
borrow(stock, "Emma")
borrow(stock, "Dune")
```

```
DEBUG    library: borrow('Dune') called; stock={'Dune': 1}
INFO     library: lent out Dune; 0 left
DEBUG    library: borrow('Dune') called; stock={'Dune': 0}
WARNING  library: Dune is not available
WARNING  library: Emma is not available
WARNING  library: Dune is not available
```

Each message has a **level**, from least to most serious: `DEBUG` (details for developers), `INFO` (normal events), `WARNING` (something unexpected, but the program carries on), `ERROR` and `CRITICAL`. `basicConfig` sets which levels are shown and how messages look. Halfway through, the program raises the level to `WARNING`, and from then on, only warnings appear. The code that logs doesn't change at all; only the setting does.

Give each module its own logger with `logging.getLogger(name)`; the name appears in each message, so you can see where it came from. Notice the style of the calls: `log.debug("borrow(%r) called", title)` passes the values separately instead of using an f-string. The logging module fills them in only if the message is actually shown, which saves work for messages that are switched off.

In a real application, you'd send the log to a file as well as, or instead of, the screen: `logging.basicConfig(filename="app.log", ...)`. When a user reports a problem, the log tells you what happened.

Try It

1. Fix `parse_price` so that `"free"` counts as 0. Where else could you fix it?

2. Run `debug_me.py` yourself. At the `(Pdb)` prompt, try `l`, `w` and `p result`, then fix the bug.
3. In `logging_demo.py`, change the first level to `logging.INFO`. Which messages disappear?

Exercises

1. **Safe conversion.** Write `safe_int(text, default=0)`, which converts text to an integer and returns the default if it can't, logging a warning that shows the bad text.
2. **Median.** The function below should return the middle value of a list, or the average of the two middle values when the list has an even length. It has two bugs. Find and fix them, and make it raise a `ValueError` for an empty list.

```
def median(numbers):
    """Return the middle value of a list of numbers. (It has two bugs.)"""
    middle = len(numbers) // 2
    return numbers[middle]


print(median([7, 1, 3]))  # we expect 3
print(median([4, 1, 3, 2]))  # we expect 2.5
```

```
1
3
```

Key Takeaways

- Read tracebacks from the bottom: the error, then the frames from where it happened back to where it started. Then decide where the fix belongs.
- For wrong answers without errors, reproduce with a small input, know the expected result, test a hypothesis, and fix with a test.
- `breakpoint()` starts the pdb debugger: `p` prints, `n` steps, `c` continues, `q` quits.
- Use `logging` instead of `print` for messages that should stay in a program. Levels let you turn detail up or down without changing the code.

CHAPTER 17 · INTERMEDIATE

Virtual Environments, Packages and Project Layout

Small scripts live happily in a single file. Real projects grow into folders of modules, depend on packages written by other people, and need to be installed on other computers. In this chapter you'll learn how Python projects are organized: **virtual environments** to keep each project's packages separate, **pip** to install packages, and **pyproject.toml** to turn your own code into a package that installs with one command and gives you a real command-line tool. Pocket Library becomes an installable app with its own `pocket` command.

Why Virtual Environments?

Suppose one of your projects needs version 2 of a package and another needs version 3. If both are installed into the same Python, one project breaks. A **virtual environment** solves this: it's a folder containing its own copy of the `python` command and its own set of installed packages. Each project gets one, usually called `.venv`, inside the project folder.

You created one in Chapter 15. Here are the steps again, with the commands to leave and remove it:

Task	Windows	macOS and Linux
Create	`python -m venv .venv`	`python3 -m venv .venv`
Activate	`.venv\Scripts\activate`	`source .venv/bin/activate`
Leave	`deactivate`	`deactivate`
Remove	delete the `.venv` folder	delete the `.venv` folder

While it's active, your prompt starts with `(.venv)`, and `python` and `pip` refer to the environment's copies. Packages you install go into `.venv`, and nowhere else. If an environment gets into a mess, delete the folder and create a new one; it holds nothing you can't reinstall.

> **Tip:** Never put your own files inside `.venv`, and don't copy it to other computers or add it to version control. It's specific to one computer. What you share is the list of packages, as you'll see below.

Many Linux systems and Homebrew on macOS refuse to install packages into the system's Python and print an error mentioning an *externally managed environment*. That's deliberate protection. The answer is always the same: use a virtual environment.

Installing Packages with pip

pip installs packages from the **Python Package Index** (PyPI) at pypi.org, which holds hundreds of thousands of packages. Run it as `python -m pip`, which makes sure you use the pip that belongs to the active Python:

```
$ python -m pip install pandas
$ python -m pip install "pandas==3.0.6"
$ python -m pip list
$ python -m pip install --upgrade pandas
$ python -m pip uninstall pandas
```

The first command installs the newest version, along with any packages pandas itself needs. The second installs an exact version. `list` shows what's installed, `--upgrade` moves to the newest version, and `uninstall` removes it.

To record what a project needs, list the packages in a `requirements.txt` file, one per line, so that anyone can recreate the environment with `python -m pip install -r requirements.txt`. Here's the one for this book:

```
# Packages for Parts III to V, at the versions the book was tested with.
pytest==9.1.1
mypy==2.4.0
pandas==3.0.6
matplotlib==3.11.2
fastapi==0.143.2
httpx==0.28.1
uvicorn==0.54.0
```

Pinning exact versions with == makes an environment reproducible: everyone gets the same versions, the ones that were tested. `python -m pip freeze` prints every installed package with its exact version, in this format.

> **Warning:** Anyone can publish a package on PyPI, and installing a package runs code on your computer. Check the spelling of a package name before installing it, because attackers publish packages with names one letter away from popular ones. Prefer well-known packages with active maintenance, and look at a package's page on pypi.org and its project site before installing something unfamiliar.

A Faster Alternative: uv

uv is a popular newer tool that creates environments, installs packages and even installs Python versions, much faster than pip. Its commands mirror the ones above: `uv venv` creates `.venv`, and `uv pip install pandas` installs into it. The book's examples were tested in environments made with uv, and they work the same with venv and pip. Use whichever you prefer; this book shows the standard tools, because they come with every Python.

Laying Out a Project

Here's Pocket Library as a proper project:

```
pocket-library/
    pyproject.toml
    README.md
    src/
        pocket_library/
            __init__.py
            __main__.py
            books.py
            cli.py
    tests/
        test_books.py
```

- `src/pocket_library/` is the **package**: a folder of modules that's imported as a unit, as in `from pocket_library.books import Book`. Putting it inside `src/` makes sure your tests run against the installed package, not against whatever files happen to be lying around.
- `__init__.py` marks the folder as a package and runs when it's first imported. Here, it just holds the version.
- `__main__.py` runs when you type `python -m pocket_library`.
- `tests/` holds the tests, outside the package.
- `pyproject.toml` describes the project: its name, version, requirements and commands.

The package code is the dataclass version of Pocket Library from Chapter 13, with a command-line interface built with `argparse` as in Chapter 11:

```
"""Pocket Library: your books, in your pocket."""

__version__ = "0.1.0"
```

```
import json
from dataclasses import asdict, dataclass
from pathlib import Path


@dataclass
class Book:
    title: str
    author: str
    year: int
    read: bool = False
```

```
class Library:
    def __init__(self, path):
        self.path = Path(path)
        self.books = []
        if self.path.exists():
            self.books = [
                Book(**item)
                for item in json.loads(
                    self.path.read_text(encoding="utf-8")
                )
            ]

    def add(self, book):
        if any(b.title.lower() == book.title.lower() for b in self.books):
            raise ValueError(f"already in the library: {book.title}")
        self.books.append(book)

    def mark_read(self, title):
        for book in self.books:
            if book.title.lower() == title.lower():
                book.read = True
                return book
        raise KeyError(title)

    def save(self):
        data = [asdict(book) for book in self.books]
        self.path.write_text(json.dumps(data, indent=2), encoding="utf-8")
```

```
import argparse
import sys

from . import __version__
from .books import Book, Library


def main(argv=None):
    parser = argparse.ArgumentParser(
        prog="pocket", description="Your book collection."
    )
    parser.add_argument(
        "--version", action="version", version=f"pocket {__version__}"
    )
    parser.add_argument(
        "--file", default="library.json", help="where the books are kept"
    )
    commands = parser.add_subparsers(dest="command", required=True)

    add = commands.add_parser("add", help="add a book")
    add.add_argument("title")
    add.add_argument("author")
    add.add_argument("year", type=int)
    commands.add_parser("list", help="list all books")
    read = commands.add_parser("read", help="mark a book as read")
    read.add_argument("title")

    args = parser.parse_args(argv)
    library = Library(args.file)
```

```
    try:
        if args.command == "add":
            library.add(Book(args.title, args.author, args.year))
            library.save()
            print(f"Added {args.title}.")
        elif args.command == "read":
            book = library.mark_read(args.title)
            library.save()
            print(f"Marked {book.title} as read.")
        elif args.command == "list":
            for book in library.books:
                mark = "x" if book.read else " "
                print(f"[{mark}] {book.title} ({book.author}, {book.year})")
    except ValueError as error:
        print(f"pocket: {error}", file=sys.stderr)
        return 1
    except KeyError:
        print(f"pocket: no book called {args.title!r}", file=sys.stderr)
        return 1
    return 0


if __name__ == "__main__":
    sys.exit(main())
```

```
import sys

from .cli import main

sys.exit(main())
```

`from .books import Book, Library` is a **relative import**: the dot means "from this package". `add_subparsers` gives the tool subcommands, like `pocket add` and `pocket list`, each with its own arguments. `main` takes an optional list of arguments, which lets the tests call it directly, and returns an **exit code**: 0 for success and 1 for an error, the convention every command-line tool follows. Error messages go to `sys.stderr`, the error stream, so they aren't mixed into output that another program might read.

pyproject.toml

The project file is written in **TOML**, a simple settings format:

```
[build-system]
requires = ["setuptools>=77"]
build-backend = "setuptools.build_meta"

[project]
name = "pocket-library"
version = "0.1.0"
description = "Keep track of the books you own and the books you've read."
```

```
readme = "README.md"
requires-python = ">=3.12"
license = "MIT"
authors = [{ name = "Your Name" }]

[project.scripts]
pocket = "pocket_library.cli:main"

[project.optional-dependencies]
test = ["pytest"]
```

- `[build-system]` says which tool builds the package: here, **setuptools**, the most widely used.
- `[project]` gives the name people install it by, its version, a one-line description, the Python versions it supports, and its license.
- `[project.scripts]` creates a command. `pocket = "pocket_library.cli:main"` means: make a command called `pocket` that calls `main` in `pocket_library.cli`. When the package is installed, pip creates a small program called `pocket` in the environment.
- `[project.optional-dependencies]` lists extras. `test = ["pytest"]` means `pip install ".[test]"` also installs pytest. Packages that the code itself needs would go in a `dependencies` list in `[project]`; Pocket Library needs none.

Installing and Running Your Package

Here are the commands, run in a fresh virtual environment from inside the `pocket-library` folder, with the environment activated:

```
$ python -m venv .venv
$ pip install -q -e ".[test]"
$ pocket --version
pocket 0.1.0
$ pocket add "Dune" "Frank Herbert" 1965
Added Dune.
$ pocket add "Emma" "Jane Austen" 1815
Added Emma.
$ pocket read "dune"
Marked Dune as read.
$ pocket list
[x] Dune (Frank Herbert, 1965)
[ ] Emma (Jane Austen, 1815)
$ pocket read "Ulysses"
pocket: no book called 'Ulysses'
$ python -m pytest -q
....                                                                     [100%]
```

```
4 passed in 0.01s
```

`pip install -e .` installs the project in **editable** mode: the `-e` makes the installed package point at your source files, so changes take effect without reinstalling. `.[test]` installs it with its test extra. After that, `pocket` is a command like any other. It saves its books in `library.json` in the current folder.

This transcript was produced by a script that ran these commands in a fresh environment on Python 3.12, 3.13, 3.14 and 3.15, and got exactly the same output each time.

Here are the tests, which use the `tmp_path` fixture from Chapter 15 and another built-in fixture, `capsys`, which captures what a function prints:

```
import pytest

from pocket_library.books import Book, Library
from pocket_library.cli import main


def test_add_and_reload(tmp_path):
    path = tmp_path / "books.json"
    library = Library(path)
    library.add(Book("Dune", "Frank Herbert", 1965))
    library.save()
    assert Library(path).books == [Book("Dune", "Frank Herbert", 1965)]


def test_no_duplicates(tmp_path):
    library = Library(tmp_path / "books.json")
    library.add(Book("Dune", "Frank Herbert", 1965))
    with pytest.raises(ValueError):
        library.add(Book("DUNE", "Frank Herbert", 1965))


def test_cli_add_list_read(tmp_path, capsys):
    file = str(tmp_path / "books.json")
    assert main(["--file", file, "add", "Emma", "Jane Austen", "1815"]) == 0
    assert main(["--file", file, "read", "emma"]) == 0
    assert main(["--file", file, "list"]) == 0
    assert (
        capsys.readouterr().out.splitlines()[-1]
        == "[x] Emma (Jane Austen, 1815)"
    )


def test_cli_unknown_title(tmp_path, capsys):
    assert (
        main(["--file", str(tmp_path / "b.json"), "read", "Nothing"]) == 1
    )
    assert "pocket:" in capsys.readouterr().err
```

To share your package, `python -m build` (after `python -m pip install build`) creates a **wheel**, a single `.whl` file that anyone can install with pip. Publishing to PyPI is beyond this book; the Python Packaging User Guide at packaging.python.org explains it step by step.

Try It

1. Create a new virtual environment, install the package from the book's code repository (`code/ch17/pocket-library`) with `python -m pip install -e ".[test]"`, and run the tests.
2. Run `pocket --help` and `pocket add --help`. Where does the help text come from?
3. Run `python -m pocket_library list`. Which file makes that work?

Exercises

1. **Statistics.** Add a `pocket stats` command that prints how many books there are, how many are read and still to read, and the oldest book. Put the calculation in a function that returns lines of text, so that it's easy to test.

Key Takeaways

- Give every project its own virtual environment: `python -m venv .venv`, then activate it. Never install into the system Python.
- Install packages with `python -m pip install`, and record them in `requirements.txt`, pinned with `==` for reproducibility. Check package names carefully.
- A package is a folder of modules with `__init__.py`; put it in `src/`, with tests in `tests/` and a `pyproject.toml` at the top.
- `[project.scripts]` in `pyproject.toml` turns a function into a command. `pip install -e .` installs a project for development.

PART IV

Advanced Python

CHAPTER 18 · ADVANCED

Decorators and Context Managers

You've used decorators, such as `@dataclass`, `@property` and `@pytest.fixture`, and context managers, in every `with open(...)`. In this chapter you'll learn how they work and write your own. Both rest on one idea from Chapter 8: functions are values, which can be passed around, returned and stored like any other. Decorators let you add behavior to functions without changing their code; context managers make sure that set-up is always followed by clean-up. Libraries use both everywhere, and understanding them will make those libraries much less mysterious.

Functions Are Objects

```
def shout(text):
    return text.upper() + "!"


def whisper(text):
    return text.lower() + "..."


def speak(style, text):
    return style(text)  # a function passed in like any value


say = shout  # a second name for the same function
print(say("hello"))
print(speak(whisper, "Hello"))


def make_multiplier(factor):
```

```
    def multiply(n):
        return n * factor  # remembers factor: this is a closure

    return multiply


double = make_multiplier(2)
triple = make_multiplier(3)
print(double(21), triple(7))
```

```
HELLO!
hello...
42 21
```

`say = shout` doesn't call `shout`; it gives the function a second name. `speak` takes a function as an argument and calls it. A function that takes or returns functions is called a **higher-order function**.

`make_multiplier` goes one step further: it defines a function inside itself and returns it. The inner function `multiply` uses `factor`, a variable of the outer function. Normally that variable would disappear when `make_multiplier` returns, but Python keeps it alive for as long as `multiply` needs it. A function that remembers variables from where it was created is called a **closure**. `double` and `triple` are two closures with different values of `factor`.

Writing a Decorator

A **decorator** is a function that takes a function and returns a new function, usually one that does something extra and calls the original:

```
import functools


def log_calls(func):
    """Print each call and its result."""

    @functools.wraps(func)
    def wrapper(*args, **kwargs):
        result = func(*args, **kwargs)
        print(f"{func.__name__}{args} -> {result!r}")
        return result

    return wrapper


def repeat(times):
    """A decorator with an argument: run the function several times."""

    def decorator(func):
```

```
        @functools.wraps(func)
        def wrapper(*args, **kwargs):
            for _ in range(times):
                result = func(*args, **kwargs)
            return result

        return wrapper

    return decorator


@log_calls
def add(a, b):
    """Add two numbers."""
    return a + b


@repeat(3)
def cheer(name):
    print(f"Go, {name}!")


add(2, 3)
add(10, -4)
cheer("Ravi")
print(add.__name__, "|", add.__doc__)
```

```
add(2, 3) -> 5
add(10, -4) -> 6
Go, Ravi!
Go, Ravi!
Go, Ravi!
add | Add two numbers.
```

Writing `@log_calls` above `def add` is exactly the same as writing `add = log_calls(add)` after it. `log_calls` receives the original `add`, defines `wrapper`, which prints each call and its result, and returns `wrapper`. From then on, the name `add` refers to the wrapper, which calls the original inside. `*args` and `**kwargs`, from Chapter 8, let the wrapper accept and pass on any arguments, so the same decorator works on any function.

`@functools.wraps(func)` copies the original function's name and docstring onto the wrapper. Without it, `add.__name__` would be `"wrapper"`, and `help(add)` would show nothing useful. Always use it in your decorators.

`repeat(3)` is a decorator that takes an argument, which needs one more level: `repeat(3)` is called first and returns the actual decorator, which then receives `cheer`. Three nested functions look complicated at first, but the pattern is always the same, and you can copy it.

Decorators are ideal for things that apply to many functions but aren't part of what any one of them does: logging, timing, checking permissions, retrying after errors, and caching.

Caching

Some functions are slow but always give the same result for the same arguments. A **cache** remembers results so that each one is calculated only once. The standard library has a decorator for it:

```
import functools

calls = 0


@functools.cache
def fibonacci(n):
    global calls
    calls += 1
    return n if n < 2 else fibonacci(n - 1) + fibonacci(n - 2)


print(fibonacci(80))
print("Function calls:", calls)  # without the cache: about 76 quadrillion
print(fibonacci.cache_info())
```

```
23416728348467685
Function calls: 81
CacheInfo(hits=78, misses=81, maxsize=None, currsize=81)
```

Calculating the 80th Fibonacci number this way, where each number is the sum of the two before it, is hopeless without the cache: the function would call itself about 76 quadrillion times, which would take years. With `@functools.cache`, each value from 0 to 80 is calculated once: 81 calls, and the answer appears instantly. `cache_info` shows that the cache answered 78 further requests itself.

The `global calls` line lets the function change the global variable `calls`, which it couldn't otherwise do, as you saw in Chapter 8. It's here only to count the calls; avoid `global` in real code.

> **Note:** Cache only functions whose result depends on nothing but their arguments. A function that reads a file or the current time would go on returning an old, stale answer. `functools.lru_cache(maxsize=1000)` keeps only the most recently used results, when memory matters.

Context Managers

A **context manager** is an object that does something when a `with` block starts and something else when it ends, however it ends: normally, through `return` or `break`, or with an exception. Files close themselves this way. You can write your own in two ways:

```
from contextlib import contextmanager, suppress
from pathlib import Path


class Indented:
    """Indent everything printed through it, one level per 'with'."""

    level = 0

    def __enter__(self):
        Indented.level += 1
        return self

    def __exit__(self, exc_type, exc, tb):
        Indented.level -= 1
        return False  # don't hide exceptions

    def print(self, text):
        print("    " * Indented.level + text)


@contextmanager
def tag(name):
    print(f"<{name}>")
    yield
    print(f"</{name}>")


with Indented() as out:
    out.print("Chapter 1")
    with Indented() as inner:
        inner.print("Section 1.1")
    out.print("Chapter 2")

with tag("ul"):
    for item in ["tea", "milk"]:
        print(f"  <li>{item}</li>")

with suppress(FileNotFoundError):
```

```
    Path("missing.txt").unlink()  # no error if it isn't there
print("Done.")
```

```
    Chapter 1
        Section 1.1
    Chapter 2
<ul>
  <li>tea</li>
  <li>milk</li>
</ul>
Done.
```

As a class, with two special methods. `__enter__` runs at the start of the `with` block, and whatever it returns becomes the name after `as`. `__exit__` runs at the end. It receives details of any exception that happened in the block, or three `None` values if there wasn't one; returning `False` lets the exception carry on as usual.

As a generator, with the `@contextmanager` decorator from `contextlib`. Everything before `yield` runs at the start of the block and everything after it at the end. This is usually shorter. In real use, put the `yield` inside `try` with the clean-up in `finally`, so that the clean-up runs even if the block raises an exception; the exercise solution in Appendix A does this.

`contextlib.suppress` is a ready-made context manager that ignores the exceptions you name: deleting a file that might not exist is the classic example.

Use a context manager whenever something must be undone: files to close, locks to release (Chapter 21), database transactions to commit or roll back (Chapter 23), a setting to change temporarily and restore.

Try It

1. Write a decorator `timed` that prints how long a function took, using `time.perf_counter()` before and after the call.
2. Remove `@functools.wraps(func)` from `log_calls` and run the program again. What does the last line print?
3. Run `fibonacci(30)` without the cache, counting the calls. How many are there?

Exercises

1. **Retry.** Write a decorator `retry(times)` that calls the function again if it raises an exception, up to `times` attempts in total, printing each failure, and lets the last exception through if every attempt fails.
2. **Call counter.** Write a decorator `count_calls` that counts how often a function is called, in an attribute: `hello.calls`. (Functions are objects, so you can give them attributes.)
3. **Temporary value.** Write a context manager `temporary_value(mapping, key, value)` that sets `mapping[key]` inside a `with` block and restores the old value afterward, or removes the key if it wasn't there before.

Key Takeaways

- Functions are objects: they can be passed as arguments, returned from functions and given new names. A closure remembers variables from where it was defined.
- A decorator takes a function and returns a replacement; `@decorator` above `def` is shorthand for `f = decorator(f)`. Use `functools.wraps` in every decorator.
- `functools.cache` remembers results, which can turn an impossible calculation into an instant one.
- Context managers run set-up and clean-up around a `with` block: write them as a class with `__enter__` and `__exit__`, or as a generator with `@contextmanager`.

CHAPTER 19 · ADVANCED

Type Hints and Static Checking

Python doesn't make you declare what type a variable holds. That makes it quick to write, but it also means some mistakes, like adding a string to a number, show up only when that line runs, which may be in front of a user. **Type hints** let you write down the types you intend. Python itself ignores them when it runs your program, but your editor and a **type checker** called **mypy** read them and find mistakes before you run anything. In larger programs, type hints are some of the most useful documentation you can write.

Writing Type Hints

You've seen type hints already, in dataclasses. Here they are on functions and variables:

```
def greeting(name: str, times: int = 1) -> str:
    return " ".join([f"Hi, {name}!"] * times)


def average(numbers: list[float]) -> float:
    return sum(numbers) / len(numbers)


def find_index(items: list[str], target: str) -> int | None:
    """Return the position of target, or None if it isn't there."""
    for i, item in enumerate(items):
        if item == target:
            return i
    return None
```

```
scores: dict[str, int] = {"asha": 91, "ben": 78}
print(greeting("Zara", 2))
print(average([2.5, 3.5, 6]))
print(find_index(["a", "b", "c"], "c"), find_index(["a"], "z"))
print(greeting.__annotations__)
```

```
Hi, Zara! Hi, Zara!
4.0
2 None
{'name': <class 'str'>, 'times': <class 'int'>, 'return': <class 'str'>}
```

- A parameter's type goes after a colon: `name: str`. With a default value, the default comes after the type: `times: int = 1`.
- The return type goes after an arrow before the colon: `-> str`.
- Collections say what they contain, in square brackets: `list[float]` is a list of floats; `dict[str, int]` maps strings to integers. A tuple lists each item's type: `tuple[int, int]`.
- `int | None` means "an `int` or `None`". Use it for functions that sometimes have nothing to return, like `find_index`.
- A variable can have a hint too: `scores: dict[str, int] = {...}`.

The hints change nothing when the program runs: the last line shows they're simply stored in the function's `__annotations__`. Python even lets you call `average` with integers, as here, because an `int` is accepted wherever a `float` is expected.

Checking Types with mypy

mypy reads your code and its type hints and reports anything that doesn't fit, without running the program. Install it in your virtual environment with `python -m pip install mypy`. Here's a file with two mistakes:

```
def total_pages(books: list[dict[str, int]]) -> int:
    return sum(book["pages"] for book in books)


def label(year: int) -> str:
    return "Published in " + year  # mistake 1: adding str and int


pages = total_pages([{"pages": 310}, {"pages": 412}])
print(label(1965))
```

```
count: int = "twelve"  # mistake 2: wrong type
```

Running `python -m mypy --pretty type_mistakes.py` reports both, with the line, a description and a marker under the problem:

```
type_mistakes.py:6: error: Unsupported operand types for + ("str" and
"int")  [operator]
        return "Published in " + year  # mistake 1: adding str and int
                                 ^~~~
type_mistakes.py:11: error: Incompatible types in assignment (expression
has type "str", variable has type "int")  [assignment]
    count: int = "twelve"  # mistake 2: wrong type
                 ^~~~~~~~
```

Running the program would crash at the first mistake, `label(1965)`, and never even reach the second. mypy finds both at once, without running anything. In a large program, that turns bugs that would surface weeks later into errors you fix in seconds.

Editors such as Visual Studio Code do the same checking as you type, underlining mistakes and suggesting attribute names, because type hints tell them what each value is.

Typing a Real Program

Here's part of Pocket Library with full type hints:

```
from dataclasses import dataclass, field
from operator import attrgetter
from typing import Protocol


@dataclass
class Book:
    title: str
    author: str
    year: int
    tags: list[str] = field(default_factory=list)


class Shelf(Protocol):
    """Anything with a books attribute counts as a Shelf."""

    books: list[Book]


@dataclass
class Library:
    books: list[Book] = field(default_factory=list)
```

```
    def by_decade(self) -> dict[int, list[str]]:
        result: dict[int, list[str]] = {}
        for book in self.books:
            result.setdefault(book.year // 10 * 10, []).append(book.title)
        return result

    def newest(self) -> Book | None:
        return max(self.books, key=attrgetter("year"), default=None)


def count_tagged(shelf: Shelf, tag: str) -> int:
    return sum(1 for book in shelf.books if tag in book.tags)


library = Library(
    [
        Book("Dune", "Frank Herbert", 1965, ["sci-fi"]),
        Book("Dune Messiah", "Frank Herbert", 1969, ["sci-fi"]),
        Book("Educated", "Tara Westover", 2018),
    ]
)
print(library.by_decade())
newest = library.newest()
if newest is not None:  # mypy insists on this check
    print("Newest:", newest.title)
print("Sci-fi:", count_tagged(library, "sci-fi"))
```

```
{1960: ['Dune', 'Dune Messiah'], 2010: ['Educated']}
Newest: Educated
Sci-fi: 2
```

A few things are new:

- `newest` returns `Book | None`, because an empty library has no newest book. When the program uses the result, mypy insists on checking for `None` first. Forgetting that check is one of the most common bugs in any language, and mypy catches every case of it.
- `attrgetter("year")`, from the `operator` module, makes a small function that gets an object's `year` attribute, which is just what `max` needs for its `key`.
- `setdefault` returns the value for a key, first storing the default if the key is missing. It's another way to build a dictionary of lists.
- `Protocol` describes what an object must be able to do, rather than what class it must be. `Shelf` says "anything with a `books` attribute that's a list of `Book`". `count_tagged` accepts any such object, and `Library` qualifies without inheriting from `Shelf` or knowing it exists. This matches Python's style, often called **duck typing**: if it has the attributes you need, it'll do.

The book's harness checks this file, `typed_basics.py` and the exercise solution with `mypy --strict`, which requires hints everywhere and turns on all checks:

```
Success: no issues found in 3 source files
```

How Much to Type

Type hints are optional, and you can add them gradually. Good places to start:

- **Function parameters and return values.** They're the contracts between parts of your program and give the most value for the least effort.
- **Data classes**, which require them anyway.
- **Code that other people use**, such as libraries and shared modules.

Local variables rarely need hints; mypy works their types out from the values. Short scripts often don't need hints at all. Run mypy in your project as you would run your tests, and fix what it finds.

Try It

1. Fix both mistakes in `type_mistakes.py` and run mypy again until it reports no errors.
2. Remove the `if newest is not None:` check in `typed_library.py` and run `python -m mypy typed_library.py`. Read the error.
3. Add type hints to the functions in your solutions to Chapter 8 and check them with mypy.

Exercises

1. **Shopping basket.** Write an `Item` dataclass with `name`, `price` and `quantity` (default 1), and fully typed functions: `order_total(items, discount=0.0)`, which returns the total rounded to cents; `cheapest(items)`, which returns the cheapest item or `None` for an empty list; and `parse_quantities(text)`, which turns `"tea=2, milk=1"` into `{"tea": 2, "milk": 1}`. Check them with `mypy --strict`.

Key Takeaways

- Type hints document intended types: `name: str`, `-> int`, `list[str]`, `dict[str, int]`, `int | None`. Python ignores them when running.
- mypy checks the hints without running the program and finds mistakes such as wrong types and missing `None` checks.
- `Protocol` describes what an object must have, not what class it is, which matches Python's duck typing.
- Add hints gradually, starting with function signatures and shared code, and run mypy as part of your checks.

CHAPTER 20 · ADVANCED

Functional Tools and Pattern Matching

This chapter collects tools that make everyday code shorter and clearer. **Lambdas** are tiny functions you can write in the middle of an expression, perfect for sorting. `map`, `filter`, `reduce` and `partial` come from **functional programming**, a style built on passing functions around. And **structural pattern matching**, the `match` statement, checks the shape of data and pulls it apart in one step, which is ideal for commands, messages and nested data.

Lambdas and Sorting Keys

Several times in this book you've passed a function as `key=` to tell `sorted`, `max` or `groupby` what to compare: `key=counts.get` in Chapter 7, `key=region_of` in Chapter 14, `key=attrgetter("year")` in Chapter 19. When the function is short and used only once, a **lambda** saves you from defining it separately:

```
from operator import itemgetter

books = [
    ("Dune", 1965, 4.3),
    ("Educated", 2018, 4.5),
    ("The Hobbit", 1937, 4.6),
    ("Sapiens", 2011, 4.4),
]

by_year = sorted(books, key=lambda book: book[1])
print([title for title, _, _ in by_year])

best_first = sorted(books, key=itemgetter(2), reverse=True)
print([title for title, _, _ in best_first])
```

```
print(max(books, key=lambda book: len(book[0]))[0])

words = ["kiwi", "Apple", "fig", "banana"]
print(sorted(words, key=lambda w: (len(w), w.lower())))
```

```
['The Hobbit', 'Dune', 'Sapiens', 'Educated']
['The Hobbit', 'Educated', 'Sapiens', 'Dune']
The Hobbit
['fig', 'kiwi', 'Apple', 'banana']
```

`lambda book: book[1]` is a function that takes `book` and returns `book[1]`. It does exactly what this `year_of` function does, but it's written as an expression, without a name:

```
def year_of(book):
    return book[1]


dune = ("Dune", 1965, 4.3)
print(year_of(dune))
print((lambda book: book[1])(dune))  # the same function, without a name
```

```
1965
1965
```

A lambda can contain only a single expression, whose value is returned automatically; no statements, no `return`. That's deliberate: anything longer deserves a proper `def` with a name.

`itemgetter(2)` from the `operator` module is a ready-made alternative to `lambda book: book[2]`, just as `attrgetter` gets attributes. The last example sorts by a tuple: first by length, then, for words of the same length, alphabetically ignoring case. Tuples compare item by item, which makes sorting by several keys easy. Use `reverse=True` for descending order, or a minus sign on a numeric key, as in `-score`, to reverse just that one part.

map, filter, reduce and partial

```
from functools import partial, reduce

prices = [12.5, 8.0, 30.0, 4.25]

with_tax = list(map(lambda p: round(p * 1.2, 2), prices))
expensive = list(filter(lambda p: p > 10, prices))
print(with_tax)
```

```
print(expensive)

# The same with comprehensions, which most Python programmers prefer:
print([round(p * 1.2, 2) for p in prices])
print([p for p in prices if p > 10])

product = reduce(lambda a, b: a * b, [1, 2, 3, 4, 5])
print(product)


def power(base, exponent):
    return base**exponent


square = partial(power, exponent=2)
cube = partial(power, exponent=3)
print(square(9), cube(2))
```

```
[15.0, 9.6, 36.0, 5.1]
[12.5, 30.0]
[15.0, 9.6, 36.0, 5.1]
[12.5, 30.0]
120
81 8
```

- `map(function, items)` applies a function to every item.
- `filter(function, items)` keeps the items for which the function returns true.
- `reduce(function, items)`, from `functools`, combines items pairwise into one result: ((((1 × 2) × 3) × 4) × 5).
- `partial(function, ...)` makes a new function with some arguments filled in. `square` is `power` with the exponent fixed at 2.

`map` and `filter` return iterators, like generators, so the program wraps them in `list` to see the results. In Python, list comprehensions usually do the same job more readably, as the middle lines show, and most Python programmers prefer them. `map` is still handy with an existing function, such as `map(int, ["1", "2"])`. For reduction, Python has built-ins for the common cases: `sum`, `max`, `min`, `any`, `all` and `math.prod`.

Pattern Matching

The `match` statement compares a value against a series of **patterns** and runs the first case that fits. Unlike a chain of `if` and `elif`, a pattern can check a value's shape and pick out its parts at the same time. Here it interprets typed commands:

```
def handle(command):
    match command.split():
        case ["quit" | "exit"]:
            return "Goodbye."
        case ["add", title]:
            return f"Adding {title!r}."
        case ["add", title, year] if year.isdigit():
            return f"Adding {title!r} from {year}."
        case ["remove", *titles] if titles:
            return f"Removing {len(titles)} book(s): {', '.join(titles)}."
        case ["help" | "?"]:
            return "Commands: add, remove, quit."
        case []:
            return "Please type a command."
        case _:
            return f"I don't understand {command!r}."


for text in [
    "add Dune",
    "add Emma 1815",
    "remove Dune Emma",
    "?",
    "",
    "fly",
    "exit",
]:
    print(f"{text!r:20} -> {handle(text)}")
```

```
'add Dune'           -> Adding 'Dune'.
'add Emma 1815'      -> Adding 'Emma' from 1815.
'remove Dune Emma'   -> Removing 2 book(s): Dune, Emma.
'?'                  -> Commands: add, remove, quit.
''                   -> Please type a command.
'fly'                -> I don't understand 'fly'.
'exit'               -> Goodbye.
```

`command.split()` makes a list of words, and each `case` describes a list:

- `["add", title]` matches a list of exactly two words whose first is `"add"`, and stores the second in `title`.
- `["add", title, year] if year.isdigit()` matches three words, but only if the **guard**, the `if` after the pattern, is true.

- `["remove", *titles]` matches `"remove"` followed by any number of words, collected into a list.
- `"quit" | "exit"` matches either word.
- `[]` matches an empty list, and _ matches anything: the default case at the end.

Doing the same with `if` statements would need checks of the length, the first word and the types, all mixed together. The patterns read like a description of the valid commands.

Patterns can also match objects and dictionaries:

```
from dataclasses import dataclass


@dataclass
class Point:
    x: float
    y: float


@dataclass
class Circle:
    center: Point
    radius: float


def describe(shape):
    match shape:
        case Point(x=0, y=0):
            return "the origin"
        case Point(x=0, y=y):
            return f"a point on the y-axis at {y}"
        case Point(x=x, y=y):
            return f"a point at ({x}, {y})"
        case Circle(center=Point(x=0, y=0), radius=r):
            return f"a circle of radius {r} around the origin"
        case Circle(radius=r) if r <= 0:
            return "an impossible circle"
        case Circle():
            return "a circle somewhere else"
        case {"type": "point", "x": x, "y": y}:
            return f"a point from JSON at ({x}, {y})"
        case _:
            return "something unknown"


for shape in [
    Point(0, 0),
    Point(0, 5),
    Point(2, 3),
    Circle(Point(0, 0), 4),
    Circle(Point(1, 1), -1),
    Circle(Point(1, 1), 2),
```

```
    {"type": "point", "x": 7, "y": 8},
    42,
]:
    print(describe(shape))
```

```
the origin
a point on the y-axis at 5
a point at (2, 3)
a circle of radius 4 around the origin
an impossible circle
a circle somewhere else
a point from JSON at (7, 8)
something unknown
```

`Point(x=0, y=0)` matches a `Point` whose attributes are both 0. `Point(x=0, y=y)` matches any point with `x` 0, and captures its `y`. Patterns can nest: `Circle(center=Point(x=0, y=0), radius=r)` looks inside the circle at its center. A dictionary pattern like `{"type": "point", "x": x, "y": y}` matches any dictionary that has those keys, which is ideal for JSON data from files and web APIs. As with `if` and `elif`, order matters: put specific cases before general ones.

> **Note:** In a pattern, a plain name like `y` or `title` always captures a value; it never compares with a variable of that name. To compare with a constant, use a literal (`0`, `"add"`) or a dotted name such as `Color.RED`.

Try It

1. Sort the books in `lambdas_and_sorting.py` by rating, lowest first, and then by title length.
2. Use `map` with `str.upper` to convert a list of names to capitals, then do the same with a list comprehension.
3. Add a `list` command to `handle` that accepts `list` alone or `list unread`.

Exercises

1. **Top scores.** Write `top_n(scores, n)`, which takes a dictionary of names and scores and returns the `n` best `(name, score)` pairs, highest first, with ties in alphabetical order.

2. **Compose.** Write `compose(*functions)`, which returns a single function applying the given functions from right to left, so that `compose(f, g, h)(x)` is `f(g(h(x)))`. Try it with `str.strip`, `str.upper` and a lambda that adds `"!"`.
3. **Status codes.** Write `http_status(code)` with `match`: 200, 201 and 204 are "success"; 301 and 302 "redirect"; 404 "not found"; other codes from 400 to 499 "client error"; 500 to 599 "server error"; anything else "unknown".

Key Takeaways

- A lambda is a one-expression function without a name; use it for short `key=` functions, and `def` for anything longer. `itemgetter` and `attrgetter` are ready-made keys.
- Sorting by a tuple sorts by several keys; `reverse=True` or a negated number reverses the order.
- `map`, `filter`, `reduce` and `partial` apply, select, combine and pre-fill functions. Comprehensions are usually clearer than `map` and `filter`.
- `match` compares data against patterns that check its shape and capture its parts, with `|` for alternatives, `*rest` for the remainder, guards, class patterns and dictionary patterns.

CHAPTER 21 · ADVANCED

Concurrency: Threads, Processes and asyncio

Most programs do one thing at a time. But a program that downloads twenty files spends nearly all its time waiting for the network, and a program that crunches numbers on a computer with eight processor cores uses only one of them. **Concurrency** means making progress on several tasks at once. Python offers three tools for it, and choosing the right one depends on a single question: is your program mostly **waiting**, or mostly **calculating**?

Waiting versus Calculating

Your program mostly...	Examples	Use
Waits for input and output	Downloads, web APIs, databases, files	Threads or asyncio
Calculates	Simulations, image processing, number crunching	Processes

The standard Python interpreter has a **global interpreter lock**, the **GIL**, which lets only one thread run Python code at any moment. Threads still help with waiting, because a thread that's waiting for the network releases the lock and another can run. They don't make calculations faster, though, because the calculations can't run side by side. For that, you need separate processes, each with its own interpreter and its own lock.

> **Note:** Python 3.13 introduced an optional **free-threaded** build without the GIL, and it has been officially supported since Python 3.14. It isn't the default yet, and some packages don't support it. Everything in this chapter works the same with either build.

Threads for Waiting

The simplest way to use threads is a **thread pool** from `concurrent.futures`. You give it a function and a list of inputs, and it runs the calls on several threads at once:

```
import time
from concurrent.futures import ThreadPoolExecutor


def download(name):
    """Pretend to download a file: most of the time is spent waiting."""
    time.sleep(0.5)
    return f"{name}: {len(name) * 1000} bytes"


files = [
    "report.pdf",
    "photo.jpg",
    "data.csv",
    "song.mp3",
    "notes.txt",
    "map.png",
]

start = time.perf_counter()
results = [download(f) for f in files]
sequential = time.perf_counter() - start

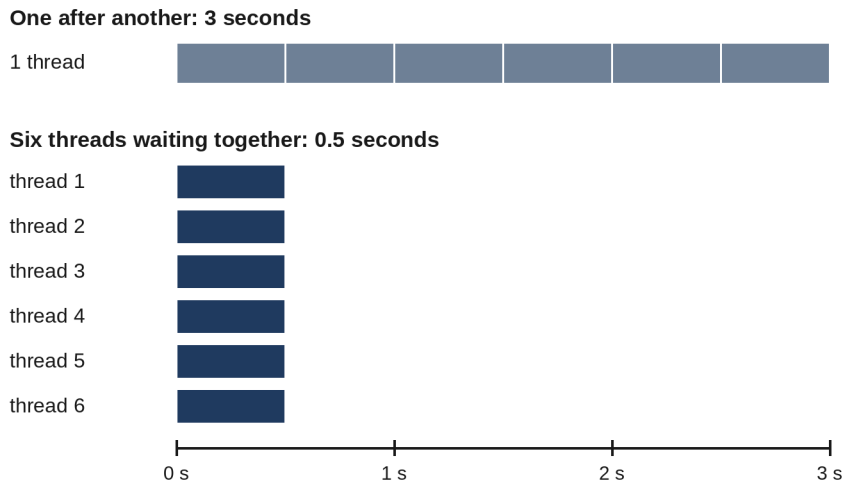
start = time.perf_counter()
with ThreadPoolExecutor(max_workers=6) as pool:
    # map gives the results in the original order
    threaded_results = list(pool.map(download, files))
threaded = time.perf_counter() - start

print("\n".join(threaded_results))
print("Same results:", results == threaded_results)
print("Threads at least 4 times faster:", sequential / threaded >= 4)
```

```
report.pdf: 10000 bytes
photo.jpg: 9000 bytes
data.csv: 8000 bytes
song.mp3: 8000 bytes
notes.txt: 9000 bytes
map.png: 7000 bytes
```

```
Same results: True
Threads at least 4 times faster: True
```

`download` pretends to fetch a file by sleeping for half a second, which is how waiting for a network looks to a program. Six downloads one after another take three seconds. With a pool of six threads, all six wait at the same time, and the whole job takes about half a second. `pool.map` works like `map` from Chapter 20: it calls the function with each item and gives back the results in the original order, whichever finishes first.



Six half-second waits take 3 seconds one after another on one thread, but only half a second when six threads wait at the same time.

The program prints `True` or `False` rather than the times, because timings differ on every computer and every run. On the computer used for this book, the threaded version was about six times faster, as you'd expect from six threads that spend all their time waiting.

Sharing Data Safely

Threads share the program's variables, and that's dangerous. `counter += 1` looks like one step but is really three: read the value, add one, store it back. If two threads read the same value before either stores, one addition is lost. These bugs, called **race conditions**, appear only occasionally, which makes them very hard to find. A **lock** lets only one thread at a time into a section of code:

```
import threading

counter = 0
lock = threading.Lock()


def add_many(times):
    global counter
    for _ in range(times):
        with lock:  # only one thread at a time may change counter
            counter += 1


threads = [
    threading.Thread(target=add_many, args=(100_000,)) for _ in range(4)
]
for t in threads:
    t.start()
for t in threads:
    t.join()  # wait for every thread to finish
print("Counter:", counter)
```

```
Counter: 400000
```

`with lock:` waits until no other thread holds the lock, takes it, and releases it at the end of the block, even after an exception: it's a context manager, as in Chapter 18. `start` begins each thread and `join` waits for it to finish. With the lock, the counter is always exactly 400,000.

The safest design avoids sharing altogether: give each thread its own input and collect results through the pool, as `threads_download.py` does. Then there's nothing to lock.

Processes for Calculating

For calculations, a **process pool** works the same way as a thread pool but runs each call in a separate Python process, which can use a separate processor core:

```
from concurrent.futures import ProcessPoolExecutor


def count_primes(start, stop):
    """Count the primes in range(start, stop): pure calculation."""
    count = 0
    for n in range(max(start, 2), stop):
        if all(n % d for d in range(2, int(n**0.5) + 1)):
            count += 1
    return count


# Required: each worker process imports this file
if __name__ == "__main__":
    ranges = [
        (0, 50_000),
        (50_000, 100_000),
        (100_000, 150_000),
        (150_000, 200_000),
    ]
    with ProcessPoolExecutor() as pool:
        counts = list(pool.map(count_primes, *zip(*ranges)))
    for (start, stop), count in zip(ranges, counts):
        print(f"{start:>7,} to {stop:>7,}: {count:,} primes")
    print(f"Total: {sum(counts):,}")
```

```
      0 to  50,000: 5,133 primes
 50,000 to 100,000: 4,459 primes
100,000 to 150,000: 4,256 primes
150,000 to 200,000: 4,136 primes
Total: 17,984
```

Counting primes is pure calculation, so the four ranges are counted in four processes at the same time. On a computer with four or more cores, that's up to four times faster than counting them one after another.

`count_primes` takes two arguments, and `pool.map` needs one list for each. `zip(*ranges)` turns the list of pairs into two lists, the starts and the stops, and the outer `*` passes them as separate arguments: it's the same as `pool.map(count_primes, starts, stops)`.

Processes don't share variables: each worker gets its own copy of the arguments, and the results are sent back. That avoids race conditions, but

sending data between processes takes time, so processes pay off only when each call does a substantial amount of work.

The `if __name__ == "__main__":` line, from Chapter 11, is **required** here. On Windows and macOS, each worker process starts by importing your file. Without the check, each worker would start its own pool, and so on, without end.

asyncio

asyncio handles waiting in a different way: one thread runs many tasks, and each task explicitly hands over control whenever it waits. It's efficient for thousands of connections at once, and it's what many modern web frameworks use, including FastAPI in Chapter 25:

```
import asyncio
import time


async def fetch(name, seconds):
    """Pretend to fetch a web page, waiting without blocking other tasks."""
    await asyncio.sleep(seconds)
    return f"{name} ({seconds} s)"


async def main():
    start = time.perf_counter()
    pages = await asyncio.gather(
        fetch("news", 0.3), fetch("weather", 0.1), fetch("sport", 0.2)
    )
    elapsed = time.perf_counter() - start
    print("In the order asked:", pages)
    # One after another, the three would take 0.6 s
    print("Faster than one after another:", elapsed < 0.6)

    tasks = [fetch("news", 0.3), fetch("weather", 0.1), fetch("sport", 0.2)]
    for finished in asyncio.as_completed(tasks):
        print("Finished:", await finished)

    try:
        await asyncio.wait_for(fetch("slow server", 2), timeout=0.2)
    except TimeoutError:
        print("Gave up on the slow server after 0.2 s")


asyncio.run(main())
```

```
In the order asked: ['news (0.3 s)', 'weather (0.1 s)', 'sport (0.2 s)']
Faster than one after another: True
Finished: weather (0.1 s)
Finished: sport (0.2 s)
Finished: news (0.3 s)
Gave up on the slow server after 0.2 s
```

- `async def` defines a **coroutine function**. Calling it doesn't run it; it creates a coroutine, which runs when it's awaited.
- `await` waits for something to finish. While one task waits, the event loop runs others. `asyncio.sleep` stands in for real waiting, such as a network request.
- `asyncio.run(main())` starts the **event loop**, the scheduler that runs the tasks, and runs `main` until it finishes.
- `asyncio.gather` runs several coroutines at the same time and returns their results in the order you gave them. The three fetches take about 0.3 seconds in total, not 0.6. The program checks this against 0.6 seconds rather than printing the time; on a very heavily loaded computer even that check could fail, which is why the book never relies on exact timings.
- `asyncio.as_completed` gives the results in the order they finish, quickest first.
- `asyncio.wait_for` gives up after a timeout and raises `TimeoutError`. Always put a time limit on waiting for things that might never answer.

Inside async code, use async libraries, such as `asyncio.sleep` instead of `time.sleep`, and an async HTTP client. A normal blocking call freezes every task on the loop until it returns.

Choosing

- **A few dozen downloads, API calls or database queries**: a `ThreadPoolExecutor`. It's simple and works with ordinary libraries.
- **Thousands of connections, or an async framework**: asyncio.
- **Heavy calculation**: a `ProcessPoolExecutor`, or a library such as NumPy that does its calculations outside Python's lock.

- **Most programs**: none of these. Concurrency adds complexity; use it when measurement shows that waiting or calculation is the bottleneck. Chapter 22 shows how to measure.

Try It

1. Change `max_workers` in `threads_download.py` to 2. How much faster is the threaded version now, and why?
2. Remove the `with lock:` line from `lock_counter.py` (keeping `counter += 1`) and run it several times. Do you always get 400,000? (The answer depends on your Python version and computer.)
3. In `async_demo.py`, change the timeout to 3 seconds. What happens?

Exercises

1. **Parallel word counts.** Write `word_counts(texts)`, which counts the words in each text using a `ThreadPoolExecutor` with four workers and returns the counts in the original order.
2. **Limited concurrency.** Write an async function `fetch_all(names, limit)` that "fetches" every name (sleeping briefly and returning it in capitals) but never runs more than `limit` fetches at the same time. Track and return the largest number running at once to prove it. (Hint: `asyncio.Semaphore(limit)` is a lock that lets in up to `limit` tasks.)

Key Takeaways

- Use threads or asyncio when a program mostly waits; use processes when it mostly calculates.
- `ThreadPoolExecutor` and `ProcessPoolExecutor` run a function over many inputs at once and return results in order. Process pools need `if __name__ == "__main__":`.
- Shared data between threads needs a lock; better still, don't share it. Processes don't share variables at all.

- asyncio runs many tasks on one thread with `async def`, `await`, `gather` and timeouts. Add concurrency only when measurement shows you need it.

CHAPTER 22 · ADVANCED

Performance: Measuring, Profiling and Choosing Data Structures

Most programs are fast enough. When one isn't, the cause is almost never where you'd guess, and the fix is almost never clever code. It's usually a better choice of data structure or algorithm, in one small part of the program. This chapter teaches the working method: **measure** before changing anything, **profile** to find where the time goes, and **choose** the right data structure. You'll see one change make a program thousands of times faster.

How Work Grows

The most important question about any piece of code is how its work grows as the data grows. Here are two ways to find a number in a sorted list. **Linear search** checks each item in turn. **Binary search**, the strategy from the guessing game in Chapter 5, checks the middle and throws away the half that can't contain the target:

```
# Count the comparisons each search needs: a measure that doesn't
# depend on how fast your computer is.


def linear_search(items, target):
    steps = 0
    for item in items:
        steps += 1
        if item == target:
```

```
            break
    return steps


def binary_search(items, target):
    """items must be sorted."""
    steps = 0
    low, high = 0, len(items) - 1
    while low <= high:
        steps += 1
        middle = (low + high) // 2
        if items[middle] == target:
            break
        if items[middle] < target:
            low = middle + 1
        else:
            high = middle - 1
    return steps


for size in [1_000, 1_000_000]:
    numbers = list(range(size))
    target = size - 1  # the worst case for linear search
    linear = linear_search(numbers, target)
    binary = binary_search(numbers, target)
    print(
        f"{size:>9,} items: linear {linear:>9,} steps, "
        f"binary {binary:>2} steps"
    )
```

```
    1,000 items: linear     1,000 steps, binary 10 steps
1,000,000 items: linear 1,000,000 steps, binary 20 steps
```

The program counts steps instead of timing them, so its output is the same on every computer. With a thousand items, binary search needs 10 steps instead of 1,000. With a million, it needs 20 instead of a million. Linear search grows in proportion to the data; binary search grows by one step each time the data doubles.

Programmers describe this growth with **big-O notation**:

Notation	Name	Example
O(1)	constant	looking up a key in a dictionary or set
O(log n)	logarithmic	binary search in a sorted list
O(n)	linear	searching a list; `x in some_list`
O(n log n)	"n log n"	sorting
O(n^2)	quadratic	comparing every item with every other item

For small data, it hardly matters. For large data, nothing matters more: no amount of fine-tuning makes an $O(n^2)$ program keep up with an $O(n)$ one. The standard library's `bisect` module does binary search for you.

Measuring with timeit

To compare two pieces of code, time them with `timeit`, which runs each one many times to get a reliable figure:

```
import timeit

setup = "items = list(range(100_000)); lookup = set(items); target = 99_999"
in_list = timeit.timeit("target in items", setup=setup, number=1_000)
in_set = timeit.timeit("target in lookup", setup=setup, number=1_000)
print(f"1,000 searches in a list: {in_list * 1000:8.2f} ms")
print(f"1,000 searches in a set:  {in_set * 1000:8.2f} ms")
print(f"The set was about {in_list / in_set:,.0f} times faster")

join = timeit.timeit("''.join(str(n) for n in range(1000))", number=1_000)
plus = timeit.timeit(
    "s = ''\nfor n in range(1000): s += str(n)", number=1_000
)
print(f"join: {join * 1000:.1f} ms, +=: {plus * 1000:.1f} ms")
```

Here's one run. Your numbers will be different, but the proportions will be similar:

```
1,000 searches in a list:  1253.35 ms
1,000 searches in a set:      0.05 ms
The set was about 24,616 times faster
join: 121.5 ms, +=: 162.7 ms
```

Checking whether a value is `in` a list means looking at the items one by one: $O(n)$. A set uses **hashing** to jump straight to where the value would be: $O(1)$. With 100,000 items, that's the difference between about a millisecond and well under a microsecond, for every search. When you check membership often, use a set or a dictionary.

The second comparison is a classic: joining many strings with `''.join(...)` or adding them one by one with `+=`. The two are close here, because CPython optimizes `+=` on strings in simple cases like this one, and in some runs `+=` even wins. `join` is still the recommended way, because its speed doesn't depend on that optimization.

> **Tip:** To time a single line quickly, type `python -m timeit "sum(range(1000))"` in the terminal. It chooses the number of runs for you.

Because timings vary, the book's checks compare speeds instead of printing them. This program's output is the same on every computer:

```
import timeit

# Checks that hold on any computer: compare, don't print raw times.
setup = "items = list(range(100_000)); lookup = set(items)"
in_list = timeit.timeit("99_999 in items", setup=setup, number=200)
in_set = timeit.timeit("99_999 in lookup", setup=setup, number=200)
print("Set at least 100 times faster than list:", in_list / in_set > 100)

setup = "data = [n % 3000 for n in range(20_000)]"
slow = timeit.timeit(
    "r = []\nfor x in data:\n    if x not in r: r.append(x)",
    setup=setup,
    number=1,
)
fast = timeit.timeit("list(dict.fromkeys(data))", setup=setup, number=1)
print("dict.fromkeys at least 50 times faster:", slow / fast > 50)
```

```
Set at least 100 times faster than list: True
dict.fromkeys at least 50 times faster: True
```

Profiling: Finding the Slow Part

In a real program, you first have to find out where the time goes. A **profiler** runs your program and records how long each function takes:

```
import cProfile
import pstats


def slow_unique(items):
    result = []
    for item in items:
        if item not in result:  # searching a list: slow for big lists
            result.append(item)
    return result


def fast_unique(items):
    return list(dict.fromkeys(items))  # keeps the order, uses hashing


data = [n % 3000 for n in range(20_000)]
profiler = cProfile.Profile()
profiler.enable()
```

```
a = slow_unique(data)
b = fast_unique(data)
profiler.disable()
print("Same result:", a == b)
pstats.Stats(profiler).sort_stats("cumulative").print_stats(4)
```

One run:

```
Same result: True
         3004 function calls in 0.354 seconds

   Ordered by: cumulative time
   List reduced from 5 to 4 due to restriction <4>

   ncalls  tottime  percall  cumtime  percall filename:lineno(function)
        1    0.353    0.353    0.354    0.354 profile_demo.py:5(slow_unique)
     3000    0.001    0.000    0.001    0.000 {method 'append' of 'list'
     ↪ objects}
        1    0.000    0.000    0.000    0.000
        ↪ profile_demo.py:13(fast_unique)
        1    0.000    0.000    0.000    0.000 {built-in method fromkeys}
```

The report lists functions by **cumulative time**, which includes the time spent in everything they call. `slow_unique` took about 0.35 seconds; `fast_unique`, which gives the same result, took under a millisecond. `slow_unique` checks `item not in result` against a list that grows to 3,000 items, for each of 20,000 items: O(n²). `fast_unique` uses `dict.fromkeys`, which keeps the first occurrence of each item, in order, using hashing: O(n).

To profile a whole program without changing it, run it with `python -m cProfile -s cumulative yourprogram.py`. Then look at the top few lines. That's where any improvement will come from.

Rules for Faster Programs

1. **Make it work, make it right, then make it fast**, and only if it needs to be. Clear code that's fast enough beats clever code.
2. **Measure first.** Profile to find the bottleneck; don't guess. Usually one function, or one line, takes most of the time.
3. **Choose the right data structure.** Sets and dictionaries for lookups, `collections.deque` for adding and removing at both ends, `bisect` for sorted data, `heapq` for repeatedly taking the smallest item.

4. **Do less work.** Move calculations out of loops, cache results (Chapter 18), stop early when you have the answer, and process data as a stream with generators (Chapter 14).
5. **Use built-ins and libraries.** `sum`, `sorted`, `min`, `max`, `Counter` and `str.join` run in fast compiled code. For numerical work on large arrays, NumPy and pandas (Chapter 24) are many times faster than Python loops.
6. **Measure again** after each change, and keep your tests passing.

Try It

1. Run `timeit_demo.py` a few times. How much do the numbers vary between runs?
2. Profile one of your own programs with `python -m cProfile -s cumulative`.
3. Use `bisect.bisect_left` to find where 42 would go in `[10, 20, 30, 40, 50]`.

Exercises

1. **Duplicates.** Write `has_duplicates(items)`, which returns `True` if any item appears more than once, in one pass through the items, using a set.
2. **Grades with bisect.** Write `grade_for(score)`, which finds the letter grade (below 60 is F, 60 to 69 D, and so on up to 90 and above for A) using `bisect.bisect` on the list of cut-off scores instead of a chain of `if` statements.
3. **Pair with a sum.** Write `pair_with_sum(numbers, target)`, which returns two numbers from the list that add up to `target`, or `None`, in O(n) time. (Hint: for each number, check whether `target - n` is in a set of the numbers seen so far.)

Key Takeaways

- How work grows with the data matters more than anything else: O(1) and O(log n) beat O(n), which beats O(n^2).

- Measure with `timeit` and find bottlenecks with `cProfile` before changing code.
- `in` on a list is O(n); on a set or dictionary it's O(1). Choosing the right data structure is the most effective speed-up.
- Make code work and right first, then fast where measurement shows it matters, and measure again.

PART V

Real-World Projects

CHAPTER 23 · PROJECT

Project: Pocket Library with SQLite

Pocket Library has kept its books in a JSON file since Chapter 10. That works well for a few hundred books, but every change rewrites the whole file, two programs saving at once can lose data, and every search means loading everything and looping through it. Real applications keep their data in a **database**. In this project, you'll move Pocket Library to **SQLite**, a complete database that comes built into Python, learn enough **SQL** to store and query your data, and finish with a tested command-line app with five commands.

SQLite and SQL

SQLite stores a whole database in a single file. There's no server to install or run, and the `sqlite3` module is part of the standard library. It's one of the most widely used pieces of software in the world: it's inside phones, web browsers and countless desktop applications.

A database holds **tables**. A table is like a spreadsheet with fixed columns: each **row** is one record, such as one book, and each **column** has a name and a type. You talk to the database in **SQL** (Structured Query Language). A few statements cover most needs:

SQL	What it does
`CREATE TABLE books (...)`	Creates a table with the given columns
`INSERT INTO books (title, author) VALUES (?, ?)`	Adds a row
`SELECT title FROM books WHERE year > 2000 ORDER BY title`	Reads rows that match a condition, in order
`UPDATE books SET rating = ? WHERE id = ?`	Changes rows
`DELETE FROM books WHERE id = ?`	Removes rows
`SELECT author, COUNT(*) FROM books GROUP BY author`	Summarizes groups of rows

SQL keywords are traditionally written in capitals, which makes them easy to tell apart from table and column names.

The Database Layer

All the database code goes in one module, `pocket/db.py`. It starts with the table's design, called its **schema**, and a dataclass for one row:

```
SCHEMA = """
CREATE TABLE IF NOT EXISTS books (
    id      INTEGER PRIMARY KEY,
    title   TEXT NOT NULL,
    author  TEXT NOT NULL,
    year    INTEGER,
    rating  INTEGER CHECK (rating BETWEEN 1 AND 5),
    read_on TEXT,                    -- a date such as 2026-10-15, or NULL
    UNIQUE (title, author)
)
"""


@dataclass
class Book:
    id: int
    title: str
    author: str
```

```
    year: int | None
    rating: int | None
    read_on: str | None
```

Each column has a type, `INTEGER` or `TEXT`, and the schema adds rules that the database enforces whatever program uses it:

- `id INTEGER PRIMARY KEY` gives each book a unique number, assigned automatically.
- `NOT NULL` means a title and an author are required. `NULL` is SQL's version of `None`.
- `CHECK (rating BETWEEN 1 AND 5)` rejects impossible ratings.
- `UNIQUE (title, author)` stops the same book being added twice.
- `-- ...` is a comment in SQL.

`CREATE TABLE IF NOT EXISTS` makes the table only if it isn't already there, so it's safe to run every time the program starts. The `Book` dataclass has one field per column, in the same order, so a row from the database becomes a `Book` with `Book(*row)`.

The `BookStore` class wraps the database connection:

```
def __init__(self, path):
    self.conn = sqlite3.connect(path)
    self.conn.execute(SCHEMA)


def close(self):
    self.conn.close()


def add(self, title, author, year=None):
    try:
        with self.conn:  # commits, or rolls back on error
            cur = self.conn.execute(
                "INSERT INTO books (title, author, year)"
                " VALUES (?, ?, ?)",
                (title, author, year),
            )
    except sqlite3.IntegrityError:
        raise ValueError(
            f"already in the library: {title} by {author}"
        ) from None
    return cur.lastrowid


def get(self, book_id):
    row = self.conn.execute(
        "SELECT * FROM books WHERE id = ?", (book_id,)
```

```
    ).fetchone()
    if row is None:
        raise KeyError(book_id)
    return Book(*row)
```

`sqlite3.connect(path)` opens the database file, creating it if it doesn't exist. `execute` runs one SQL statement.

Look closely at how values get into the SQL: never with an f-string, always as `?` **placeholders**, with the values passed separately in a tuple. The `sqlite3` module inserts them safely. If you built SQL with an f-string, a book called `x'); DROP TABLE books; --` could change your SQL into something else entirely. This attack is called **SQL injection**, and it has caused countless real data breaches. Placeholders make it impossible.

`with self.conn:` makes a **transaction**: if the block finishes, the changes are **committed**, that is, saved permanently; if an exception happens, they're **rolled back**, as if they never happened. A database never ends up with half a change. When the `UNIQUE` rule rejects a duplicate, `sqlite3` raises `IntegrityError`, and `add` turns it into a `ValueError` with a clear message. `from None` hides the original database error from the traceback, because the new message says all that matters. `cur.lastrowid` is the `id` the database gave the new row.

`get` uses `fetchone`, which returns one row as a tuple, or `None` if nothing matched.

Queries

```
def list(self, unread_only=False, search=None):
    sql = "SELECT * FROM books WHERE 1 = 1"
    params = []
    if unread_only:
        sql += " AND read_on IS NULL"
    if search:
        sql += " AND (title LIKE ? OR author LIKE ?)"
        params += [f"%{search}%", f"%{search}%"]
    sql += " ORDER BY author, year"
    return [Book(*row) for row in self.conn.execute(sql, params)]


def check_date(text):
    """Accept only real dates written as YYYY-MM-DD."""
    try:
        valid = datetime.date.fromisoformat(text).isoformat() == text
    except ValueError:
```

```
        valid = False
    if not valid:
        raise ValueError(f"not a date in the form YYYY-MM-DD: {text!r}")


def mark_read(self, book_id, date, rating=None):
    check_date(date)  # raises ValueError for a date that isn't real
    self.get(book_id)  # raises KeyError if there is no such book
    with self.conn:
        self.conn.execute(
            "UPDATE books SET read_on = ?, rating = ? WHERE id = ?",
            (date, rating, book_id),
        )


def stats(self):
    total, read, avg = self.conn.execute(
        "SELECT COUNT(*), COUNT(read_on), AVG(rating) FROM books"
    ).fetchone()
    top = self.conn.execute(
        "SELECT author, COUNT(*) AS n FROM books GROUP BY author "
        "ORDER BY n DESC, author LIMIT 1"
    ).fetchone()
    return {
        "books": total,
        "read": read,
        "average_rating": round(avg, 1) if avg else None,
        "top_author": top[0] if top else None,
    }
```

`list` builds its query from the options it's given. `WHERE 1 = 1` is always true, which simply lets every condition start with `AND`. `LIKE ?` with `%` around the search text matches it anywhere in the title or author, ignoring case for English letters. Even here, the search text goes in through a placeholder.

The `read_on` column is `TEXT`, and SQLite will store any text there, so the Python layer checks dates before saving them. `check_date` accepts only real dates written as `YYYY-MM-DD`: `date.fromisoformat` rejects text like `not-a-date` and impossible dates like `2026-02-30`, and comparing with `isoformat()` also rejects other layouts that `fromisoformat` accepts, such as `20261001`. `mark_read` calls it before changing anything.

`stats` asks the database to do the counting. `COUNT(*)` counts rows, `COUNT(read_on)` counts rows where `read_on` isn't `NULL`, and `AVG(rating)` averages the ratings, ignoring books without one. `GROUP BY author` makes one group per author, `ORDER BY n DESC` puts the largest first, and `LIMIT 1` keeps just the top author. With thousands of books, letting the database do this work is far faster than loading every row into Python.

`export_csv` writes every book to a CSV file with the `csv` module from Chapter 10, so you can open your library in a spreadsheet.

The Command-Line Interface

`pocket/cli.py` turns command-line arguments into calls to `BookStore`, with `argparse` subcommands as in Chapter 17:

```
import argparse
import sys

from .db import BookStore


def build_parser():
    parser = argparse.ArgumentParser(
        prog="pocket", description="Track your books."
    )
    parser.add_argument(
        "--db",
        default="pocket.db",
        help="database file (default: pocket.db)",
    )
    sub = parser.add_subparsers(dest="command", required=True)

    p = sub.add_parser("add", help="add a book")
    p.add_argument("title")
    p.add_argument("author")
    p.add_argument("--year", type=int)

    p = sub.add_parser("list", help="list books")
    p.add_argument(
        "--unread", action="store_true", help="only books not read yet"
    )
    p.add_argument("--search", help="match part of a title or author")

    p = sub.add_parser("read", help="mark a book as read")
    p.add_argument("id", type=int)
    p.add_argument(
        "--date", required=True, help="date read, such as 2026-10-15"
    )
    p.add_argument("--rating", type=int, choices=range(1, 6))

    sub.add_parser("stats", help="show statistics")
    p = sub.add_parser("export", help="export all books to a CSV file")
    p.add_argument("file")
    return parser


def show(books):
    if not books:
        print("No books found.")
    for b in books:
        year = b.year or "?"
        if b.read_on:
```

```
            status = f"read {b.read_on}"
            if b.rating:
                status += ", " + "*" * b.rating
        else:
            status = "unread"
        print(f"{b.id:3}  {b.title} ({b.author}, {year})  [{status}]")


def main(argv=None):
    args = build_parser().parse_args(argv)
    store = BookStore(args.db)
    try:
        if args.command == "add":
            book_id = store.add(args.title, args.author, args.year)
            print(f"Added #{book_id}: {args.title}")
        elif args.command == "list":
            show(store.list(unread_only=args.unread, search=args.search))
        elif args.command == "read":
            store.mark_read(args.id, args.date, args.rating)
            print(f"Marked #{args.id} as read on {args.date}")
        elif args.command == "stats":
            for key, value in store.stats().items():
                print(f"{key.replace('_', ' '):15} {value}")
        elif args.command == "export":
            count = store.export_csv(args.file)
            print(f"Exported {count} books to {args.file}")
    except ValueError as error:
        print(f"pocket: {error}", file=sys.stderr)
        return 1
    except KeyError as error:
        print(f"pocket: no book with id {error.args[0]}", file=sys.stderr)
        return 1
    finally:
        store.close()
    return 0
```

The design keeps the layers apart: `db.py` knows SQL but nothing about the command line, and `cli.py` knows the command line but no SQL. Either could be replaced, as you'll see in Chapter 25, where a web API uses the same idea.

![The layers of the app: you type a command; cli.py turns it into calls to BookStore in db.py; BookStore runs SQL against the pocket.db file. Tests use BookStore directly with a fresh database, and a web API could reuse it too.](images/diagram-pocket-layers.png) `choices=range(1, 6)` lets `argparse` reject a rating of 9 before the database ever sees it, and `finally` closes the database however the command ends.

Using Pocket Library

To show the whole app at work, this short script runs a series of commands exactly as you'd type them, using `subprocess.run` to start `python -m pocket` for each one, and prints what each command shows:

```
# Run Pocket Library from the command line, exactly as you would type it.
import shlex
import subprocess
import sys

commands = [
    'add "Dune" "Frank Herbert" --year 1965',
    'add "Children of Dune" "Frank Herbert" --year 1976',
    'add "Emma" "Jane Austen" --year 1815',
    'add "Educated" "Tara Westover" --year 2018',
    'add "Dune" "Frank Herbert"',
    "read 1 --date 2026-09-14 --rating 5",
    "read 4 --date 2026-10-02 --rating 4",
    "list",
    "list --unread",
    "list --search austen",
    "read 9 --date 2026-10-15",
    "read 2 --date 2026-02-30",
    "stats",
    "export books.csv",
]
for command in commands:
    print(f"$ python -m pocket {command}")
    result = subprocess.run(
        [sys.executable, "-m", "pocket", *shlex.split(command)],
        capture_output=True,
        text=True,
    )
    print(result.stdout + result.stderr, end="")
```

```
$ python -m pocket add "Dune" "Frank Herbert" --year 1965
Added #1: Dune
$ python -m pocket add "Children of Dune" "Frank Herbert" --year 1976
Added #2: Children of Dune
$ python -m pocket add "Emma" "Jane Austen" --year 1815
Added #3: Emma
$ python -m pocket add "Educated" "Tara Westover" --year 2018
Added #4: Educated
$ python -m pocket add "Dune" "Frank Herbert"
pocket: already in the library: Dune by Frank Herbert
$ python -m pocket read 1 --date 2026-09-14 --rating 5
Marked #1 as read on 2026-09-14
$ python -m pocket read 4 --date 2026-10-02 --rating 4
Marked #4 as read on 2026-10-02
$ python -m pocket list
  1  Dune (Frank Herbert, 1965)  [read 2026-09-14, *****]
  2  Children of Dune (Frank Herbert, 1976)  [unread]
  3  Emma (Jane Austen, 1815)  [unread]
  4  Educated (Tara Westover, 2018)  [read 2026-10-02, ****]
$ python -m pocket list --unread
```

```
  2  Children of Dune (Frank Herbert, 1976)  [unread]
  3  Emma (Jane Austen, 1815)  [unread]
$ python -m pocket list --search austen
  3  Emma (Jane Austen, 1815)  [unread]
$ python -m pocket read 9 --date 2026-10-15
pocket: no book with id 9
$ python -m pocket read 2 --date 2026-02-30
pocket: not a date in the form YYYY-MM-DD: '2026-02-30'
$ python -m pocket stats
books           4
read            2
average rating  4.5
top author      Frank Herbert
$ python -m pocket export books.csv
Exported 4 books to books.csv
```

Every feature works: adding, refusing a duplicate, marking books read with a date and rating, listing with filters, a clear message for an unknown ID, statistics and export.

Tests

The tests use a fixture that creates a fresh database in a temporary folder for each test, and closes it afterward. Code after `yield` in a fixture runs when the test is finished:

```
import csv

import pytest

from pocket.cli import main
from pocket.db import BookStore


@pytest.fixture
def store(tmp_path):
    s = BookStore(tmp_path / "test.db")
    yield s
    s.close()


def test_add_and_get(store):
    book_id = store.add("Dune", "Frank Herbert", 1965)
    book = store.get(book_id)
    assert (book.title, book.year, book.read_on) == ("Dune", 1965, None)


def test_duplicate_is_refused(store):
    store.add("Dune", "Frank Herbert")
    with pytest.raises(ValueError):
        store.add("Dune", "Frank Herbert")
```

```
def test_unread_and_search(store):
    a = store.add("Dune", "Frank Herbert", 1965)
    store.add("Emma", "Jane Austen", 1815)
    store.mark_read(a, "2026-10-01", 5)
    assert [b.title for b in store.list(unread_only=True)] == ["Emma"]
    assert [b.title for b in store.list(search="herb")] == ["Dune"]


def test_stats(store):
    store.add("Dune", "Frank Herbert")
    store.add("Children of Dune", "Frank Herbert")
    store.mark_read(1, "2026-10-01", 4)
    assert store.stats() == {
        "books": 2,
        "read": 1,
        "average_rating": 4.0,
        "top_author": "Frank Herbert",
    }


@pytest.mark.parametrize("bad", ["not-a-date", "2026-02-30", "20261001"])
def test_mark_read_rejects_bad_dates(store, bad):
    book_id = store.add("Dune", "Frank Herbert")
    with pytest.raises(ValueError):
        store.mark_read(book_id, bad)
    assert store.get(book_id).read_on is None


def test_mark_read_unknown_id(store):
    with pytest.raises(KeyError):
        store.mark_read(99, "2026-10-01")


def test_export_csv(store, tmp_path):
    store.add("Emma", "Jane Austen", 1815)
    out = tmp_path / "books.csv"
    assert store.export_csv(out) == 1
    rows = list(csv.DictReader(out.open(encoding="utf-8")))
    assert rows[0]["title"] == "Emma"


def test_cli_rating_must_be_1_to_5(tmp_path):
    with pytest.raises(SystemExit):
        main(
            [
                "--db",
                str(tmp_path / "x.db"),
                "read",
                "1",
                "--date",
                "2026-10-01",
                "--rating",
                "9",
            ]
        )
```

```
..........                                                              [100%]
10 passed in 0.01s
```

`test_cli_rating_must_be_1_to_5` checks that `argparse` refuses a rating of 9: it exits the program, which raises `SystemExit`.

Try It

1. Run the demo commands yourself from the chapter folder, starting with `python -m pocket add "Dune" "Frank Herbert" --year 1965`, then open `books.csv` in a spreadsheet.
2. Add a book with a title containing a single quote, such as `"The Handmaid's Tale"`. It works, thanks to placeholders.
3. Explore the database with the `sqlite3` command-line tool, if your system has it: `sqlite3 pocket.db "SELECT title, rating FROM books"`.

Exercises

1. **Remove.** Add a `remove(book_id)` method that deletes a book and raises `KeyError` if there's no book with that ID. (Hint: after a `DELETE`, the cursor's `rowcount` says how many rows were deleted.)
2. **Books per author.** Add a `books_per_author()` method that returns a dictionary from each author to their number of books, most first, using `GROUP BY`.

Key Takeaways

- SQLite is a complete database in one file, built into Python as `sqlite3`. Tables have typed columns, and constraints such as `NOT NULL`, `UNIQUE` and `CHECK` keep the data valid.
- `SELECT`, `INSERT`, `UPDATE` and `DELETE` read and change rows; `WHERE`, `ORDER BY`, `GROUP BY` and functions like `COUNT` and `AVG` let the database do the work.
- Always pass values through `?` placeholders, never with f-strings, to prevent SQL injection. `with conn:` commits a transaction or rolls it back.

- Keep the database layer and the user interface separate, and test the database code against a fresh temporary database.

CHAPTER 24 · PROJECT

Project: Data Analysis with CSV, pandas and Charts

Python is the most popular language for working with data, and for good reason: a few lines can read a spreadsheet of thousands of rows, answer questions about it, and draw a chart. In this project you'll analyze six months of sales from a small chain of bookshops, first with the standard library alone, then with **pandas**, the most widely used data analysis library, and finally with charts from **matplotlib**.

Install the two libraries in your virtual environment before you start:

```
$ python -m pip install pandas matplotlib
```

The Data

Paper Lantern Books is a fictional bookshop chain with three branches: Riverside, Old Town and Station. Its sales file, `sales.csv`, has one row for each sale, with the date, branch, category, title, number of copies and price per copy. Here are its first lines:

```
date,store,category,title,quantity,unit_price
2026-01-01,Old Town,Travel,Walking the Coast,3,13.5
2026-01-01,Station,Travel,Walking the Coast,1,13.5
2026-01-01,Station,Travel,Walking the Coast,3,13.5
2026-01-02,Station,Fiction,The Lighthouse Keeper,1,12.99
```

The data was made by a program, so you can see exactly how it was built, and so it's the same for every reader:

```
# Make the fictional sales data used in this chapter: six months of
# sales at the three branches of "Paper Lantern Books". The seed makes
# it the same every time.
import csv
import random
from datetime import date, timedelta

random.seed(2026)
STORES = {"Riverside": 1.0, "Old Town": 0.8, "Station": 1.3}
CATALOG = {
    "Fiction": [
        ("The Lighthouse Keeper", 12.99),
        ("Salt and Stars", 9.99),
        ("Northbound", 14.50),
    ],
    "Children": [("Milo the Brave Mouse", 7.99), ("The Moon Kite", 6.50)],
    "Science": [("Tiny Worlds", 18.00), ("How Rivers Think", 21.50)],
    "Cookery": [("One Pan Suppers", 16.99), ("Spice Road Kitchen", 24.00)],
    "Travel": [("Walking the Coast", 13.50)],
}
SEASON = {1: 0.8, 2: 0.85, 3: 1.0, 4: 1.05, 5: 1.1, 6: 1.25}

rows = []
day = date(2026, 1, 1)
while day <= date(2026, 6, 30):
    for store, size in STORES.items():
        for _ in range(random.randint(0, 2)):
            category = random.choice(list(CATALOG))
            title, price = random.choice(CATALOG[category])
            weekend = 1.4 if day.weekday() >= 5 else 1.0
            quantity = max(
                1,
                round(
                    random.gauss(3, 1.5)
                    * size
                    * weekend
                    * SEASON[day.month]
                ),
            )
            rows.append(
                [day.isoformat(), store, category, title, quantity, price]
            )
    day += timedelta(days=1)

with open("sales.csv", "w", newline="", encoding="utf-8") as f:
    writer = csv.writer(f)
    writer.writerow(
        ["date", "store", "category", "title", "quantity", "unit_price"]
    )
    writer.writerows(rows)
print(f"Wrote {len(rows)} sales to sales.csv")
```

```
Wrote 540 sales to sales.csv
```

Each day, each branch makes zero to two sales, of a random book. The number of copies follows a **normal distribution** around 3 (`random.gauss(3, 1.5)`), scaled up for bigger branches, weekends and the busier months toward summer. Making data with known patterns is a good way to practice analysis: you know what you should find.

Analysis with the Standard Library

You already know enough to answer the main questions with `csv`, `defaultdict` and `Counter` from Chapters 10 and 11:

```
import csv
from collections import Counter, defaultdict

revenue_by_store = defaultdict(float)
revenue_by_month = defaultdict(float)
copies_by_title = Counter()

with open("sales.csv", newline="", encoding="utf-8") as f:
    for row in csv.DictReader(f):
        revenue = int(row["quantity"]) * float(row["unit_price"])
        revenue_by_store[row["store"]] += revenue
        revenue_by_month[row["date"][:7]] += revenue
        copies_by_title[row["title"]] += int(row["quantity"])

print("Revenue by store:")
for store, revenue in sorted(
    revenue_by_store.items(), key=lambda kv: kv[1], reverse=True
):
    print(f"  {store:10} {revenue:>10,.2f}")

print("Revenue by month:")
for month in sorted(revenue_by_month):
    print(f"  {month} {revenue_by_month[month]:>10,.2f}")

print("Best sellers (copies):")
for title, copies in copies_by_title.most_common(3):
    print(f"  {copies:4}  {title}")
```

```
Revenue by store:
  Station     10,856.16
  Riverside    9,689.64
  Old Town     7,060.35
Revenue by month:
  2026-01   4,054.56
  2026-02   4,071.34
  2026-03   4,079.50
  2026-04   5,291.63
  2026-05   4,675.96
```

```
  2026-06   5,433.16
Best sellers (copies):
   413  Walking the Coast
   254  One Pan Suppers
   184  Spice Road Kitchen
```

Station, the largest branch, takes the most money, and April and June are the best months. `row["date"][:7]` takes the year and month from a date like `2026-01-01`. That works because ISO dates have a fixed layout. Walking the Coast sells the most copies, partly because it's the only travel book, so it gets every travel sale.

This approach works for any size of file and needs nothing beyond Python. But each new question means another loop and another dictionary.

Analysis with pandas

pandas works with a **DataFrame**: a table with named columns, like a spreadsheet, that you can filter, group and summarize with single commands. Here's the same analysis and more:

```
import pandas as pd

sales = pd.read_csv("sales.csv", parse_dates=["date"])
sales["revenue"] = sales["quantity"] * sales["unit_price"]

print(sales.shape)
columns = ["date", "store", "title", "quantity", "revenue"]
print(sales[columns].head(3).to_string(index=False))
print()

by_store = (
    sales.groupby("store")["revenue"].sum().sort_values(ascending=False)
)
print(by_store.round(2).to_string())
print()

table = (
    sales.pivot_table(
        index="category", columns="store", values="revenue", aggfunc="sum"
    )
    .round(0)
    .astype(int)
)
print(table.to_string())
print()

monthly = sales.groupby(sales["date"].dt.to_period("M"))["revenue"].sum()
print(monthly.round(2).to_string())
print()
```

```
sales["weekend"] = sales["date"].dt.dayofweek >= 5
print(sales.groupby("weekend")["quantity"].mean().round(2).to_string())
```

```
(540, 7)
      date    store             title  quantity  revenue
2026-01-01 Old Town Walking the Coast         3     40.5
2026-01-01  Station Walking the Coast         1     13.5
2026-01-01  Station Walking the Coast         3     40.5

store
Station      10856.16
Riverside     9689.64
Old Town      7060.35

store     Old Town  Riverside  Station
category
Children       813        675     1136
Cookery       2179       3537     3015
Fiction       1083       1887     1467
Science       1688       2172     2377
Travel        1296       1418     2862

date
2026-01    4054.56
2026-02    4071.34
2026-03    4079.50
2026-04    5291.63
2026-05    4675.96
2026-06    5433.16
Freq: M

weekend
False    3.08
True     4.55
```

Step by step:

- `pd.read_csv` reads the whole file into a DataFrame. `parse_dates=["date"]` turns the date column into real dates. `shape` gives the number of rows and columns.
- `sales["revenue"] = sales["quantity"] * sales["unit_price"]` creates a column by multiplying two columns, all 540 rows at once, without a loop. Operations on whole columns are called **vectorized**, and they're both shorter and much faster than Python loops.
- `sales[columns]` selects some columns; `head(3)` shows the first three rows.
- `groupby("store")["revenue"].sum()` does in one line what the dictionary loop did: group the rows by store and add up each group's revenue.

- `pivot_table` makes a cross-table: categories down the side, stores across the top, total revenue in each cell. Cookery is the best category in every branch, and Children's books sell best at Station.
- `dt.to_period("M")` turns each date into its month for grouping, and `dt.dayofweek` gives the day of the week as a number, Monday being 0, so `>= 5` means Saturday or Sunday.

The last result confirms a pattern built into the data: on average, weekend sales are bigger, 4.55 copies against 3.08. Finding the patterns that were put in is a good check that the analysis is right.

`to_string()` prints a whole table without abbreviation; in the shell or a Jupyter notebook you'd usually just type the variable's name.

> **Tip: Jupyter notebooks** are a popular way to explore data: you run code in small cells and see tables and charts right below each one. Install with `python -m pip install notebook` and start with `jupyter notebook`. The code in this chapter works there unchanged.

Charts with matplotlib

Numbers in a table are precise; a chart shows the pattern at a glance:

```
import matplotlib

matplotlib.use("Agg")  # draw to files, without opening a window
import matplotlib.pyplot as plt  # noqa: E402
import pandas as pd  # noqa: E402

sales = pd.read_csv("sales.csv", parse_dates=["date"])
sales["revenue"] = sales["quantity"] * sales["unit_price"]
monthly = (
    sales.groupby([sales["date"].dt.to_period("M"), "store"])["revenue"]
    .sum()
    .unstack()
)

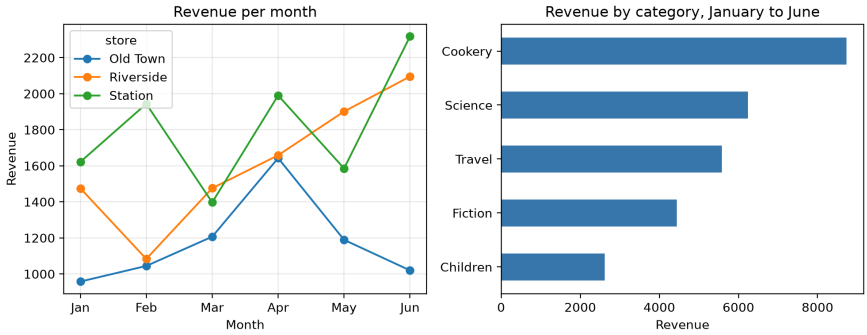
fig, (left, right) = plt.subplots(1, 2, figsize=(10, 4))
monthly.index = monthly.index.strftime("%b")
monthly.plot(ax=left, marker="o")
left.set_title("Revenue per month")
left.set_xlabel("Month")
left.set_ylabel("Revenue")
left.grid(alpha=0.3)

by_category = sales.groupby("category")["revenue"].sum().sort_values()
```

```
by_category.plot.barh(ax=right, color="#3776ab")
right.set_title("Revenue by category, January to June")
right.set_xlabel("Revenue")
right.set_ylabel("")

fig.tight_layout()
fig.savefig("sales_charts.png", dpi=200)
print("Saved sales_charts.png")
```

```
Saved sales_charts.png
```



Two charts: revenue per month for each of the three stores, January to June, and total revenue by category as horizontal bars, with Cookery the largest.

`plt.subplots(1, 2)` creates a figure with two charts side by side. `unstack()` turns the store part of the grouping into columns, one per store, which pandas' `plot` draws as one line each, with a legend. `plot.barh` draws horizontal bars, and `savefig` saves the figure as an image file. `matplotlib.use("Agg")` tells matplotlib to draw straight to a file without opening a window; leave it out when you want to see the chart on screen, and call `plt.show()` at the end instead.

The line chart shows what the totals hid: Station's sales jump up and down from month to month, Riverside grows steadily after a weak February, and Old Town falls back after April.

Try It

1. Find the total number of copies sold for each category, with the standard library and then with pandas.

2. Use `sales[sales["store"] == "Old Town"]` to keep only Old Town's rows, and find its best month.
3. Add a third chart that shows the number of copies sold on each day of the week.

Exercises

1. **Busiest day.** Write `busiest_day(sales)`, which returns the name of the weekday with the most copies sold. (Hint: `dt.day_name()` and `idxmax()`.)
2. **Best title per store.** Write `best_title_per_store(sales)`, which returns a dictionary from each store to its best-selling title by copies.
3. **Growth.** Write `month_on_month_growth(sales)`, which returns the percentage change in revenue from each month to the next. (Hint: `pct_change()`.)

Key Takeaways

- The `csv` module with `defaultdict` and `Counter` answers simple questions about data with no extra installs.
- pandas reads data into a DataFrame and answers questions with vectorized column operations, `groupby`, `pivot_table` and date tools, usually in one line each.
- matplotlib draws charts, and pandas' `plot` methods make common ones in a single call. Save them with `savefig`.
- Check an analysis by confirming patterns you already know are there.

CHAPTER 25 · PROJECT

Project: Web APIs: Calling Them and Building One

Much of today's software talks to other software over the web. A weather app asks a weather service for a forecast; a shop's website asks a payment service to take a payment. These conversations go through **web APIs**: addresses on the web that accept requests from programs and answer with data, usually in JSON. In this project you'll learn how web APIs work, build one for Pocket Library with **FastAPI**, test it, and call it from Python in two ways.

Install the packages for this chapter in your virtual environment:

```
$ python -m pip install fastapi uvicorn httpx pytest
```

How Web APIs Work

A program talks to a web API with **HTTP**, the same protocol your browser uses. Each request has:

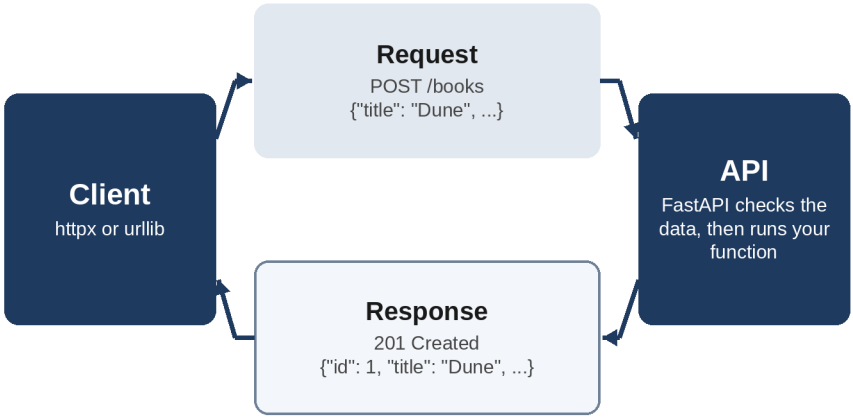
- a **URL**, the address of a **resource**, such as `http://127.0.0.1:8000/books/1` for book 1;
- a **method**, which says what to do with it: `GET` to read, `POST` to create, `PUT` or `PATCH` to change, `DELETE` to remove;
- optionally, a **body** with data, usually JSON, for example the details of a new book;

- optionally, **query parameters** after a `?`, such as `/books?author=austen`.

Each response has a **status code**, a body, and some headers. The status codes you'll meet most are:

Code	Meaning
200 OK	It worked; the body has the result
201 Created	A new resource was created
204 No Content	It worked; there's nothing to send back
404 Not Found	There's no such resource
422 Unprocessable Content	The data sent was invalid
500 Internal Server Error	The server failed

Codes in the 200s mean success, the 400s mean the client made a mistake, and the 500s mean the server did.



The client sends a request, such as POST /books with a JSON body; the API checks the data, runs your function, and sends back a response, such as 201 Created with the new book as JSON.

JSON, Again

Web APIs send and receive JSON, which you met in Chapter 10. The `json` module turns it into dictionaries and lists, and back:

```
# What a web API sends back: text in JSON format, which json.loads
# turns into ordinary dictionaries and lists.
import json

response_text = """
{
  "count": 2,
  "books": [
    {"id": 1, "title": "Dune", "author": "Frank Herbert", "year": 1965},
    {"id": 2, "title": "Emma", "author": "Jane Austen", "year": 1815}
  ]
}
"""

data = json.loads(response_text)
print(type(data))
print("Books returned:", data["count"])
for book in data["books"]:
    print(
        f"  #{book['id']} {book['title']} by {book['author']}"
        f" ({book['year']})"
    )

# Sending data works the other way round: a dictionary becomes JSON text
new_book = {"title": "Educated", "author": "Tara Westover", "year": 2018}
body = json.dumps(new_book)
print(body)
print(type(body))
```

```
<class 'dict'>
Books returned: 2
  #1 Dune by Frank Herbert (1965)
  #2 Emma by Jane Austen (1815)
{"title": "Educated", "author": "Tara Westover", "year": 2018}
<class 'str'>
```

Building an API with FastAPI

FastAPI is a popular framework for building web APIs. You write ordinary functions with type hints, from Chapter 19, and FastAPI turns them into web endpoints, checks the data that comes in, and converts what you return into JSON:

```
# A small web API for Pocket Library, built with FastAPI.
# Run it with:  python -m uvicorn books_api:app --reload
from fastapi import FastAPI, HTTPException
from pydantic import BaseModel, Field

app = FastAPI(title="Pocket Library API")


class NewBook(BaseModel):
    """What a client sends to add a book."""

    title: str = Field(min_length=1)
    author: str = Field(min_length=1)
    year: int | None = Field(default=None, ge=0, le=2100)


class Book(NewBook):
    """What the API sends back: the book plus its id."""

    id: int


books: dict[int, Book] = {}  # kept in memory: empty again on every restart


@app.get("/books")
def list_books(author: str | None = None) -> list[Book]:
    found = list(books.values())
    if author:
        found = [b for b in found if author.lower() in b.author.lower()]
    return found


@app.get("/books/{book_id}")
def get_book(book_id: int) -> Book:
    if book_id not in books:
        raise HTTPException(
            status_code=404, detail=f"No book with id {book_id}"
        )
    return books[book_id]


@app.post("/books", status_code=201)
def add_book(new: NewBook) -> Book:
    book_id = max(books, default=0) + 1
    book = Book(id=book_id, **new.model_dump())
    books[book_id] = book
    return book
```

```
@app.delete("/books/{book_id}", status_code=204)
def delete_book(book_id: int) -> None:
    if books.pop(book_id, None) is None:
        raise HTTPException(
            status_code=404, detail=f"No book with id {book_id}"
        )
```

- `app = FastAPI(...)` creates the application.
- `@app.get("/books")` is a decorator, as in Chapter 18, which registers `list_books` to answer `GET` requests for `/books`. Its parameter `author` isn't part of the path, so FastAPI takes it from the query string: `/books?author=austen`.
- In `@app.get("/books/{book_id}")`, the `{book_id}` part of the path becomes the function's `book_id` parameter. Because it's typed `int`, FastAPI converts it, and rejects a request for `/books/abc` with a clear error.
- `NewBook` is a **Pydantic** model: a class like a dataclass that checks its data. `Field(min_length=1)` rejects an empty title, and `ge=0, le=2100` (greater or equal, less or equal) rejects impossible years. When `add_book` takes a `NewBook`, FastAPI reads the request's JSON body, checks it against the model, and answers with status 422 and an explanation if it doesn't fit, without your function ever running.
- `raise HTTPException(status_code=404, ...)` sends an error response.
- The books are kept in a dictionary in memory, so they disappear when the server stops. A real service would store them in a database, such as the SQLite store from Chapter 23.

To run the server, type this in the chapter folder:

```
$ python -m uvicorn books_api:app --reload
```

uvicorn is the web server that runs FastAPI applications; `--reload` restarts it whenever you save a change. Open `http://127.0.0.1:8000/books` in your browser to see the (empty) list. Then open `http://127.0.0.1:8000/docs`: FastAPI generates an interactive documentation page from your code, where you can try every endpoint. Press Ctrl+C to stop the server.

Testing the API

FastAPI's `TestClient` sends requests to the application directly, without starting a server, so API tests are as quick as any other tests:

```
import pytest
from fastapi.testclient import TestClient

import books_api

client = TestClient(books_api.app)


@pytest.fixture(autouse=True)
def empty_library():
    books_api.books.clear()  # every test starts with no books


def add(title, author, year=None):
    return client.post(
        "/books", json={"title": title, "author": author, "year": year}
    )


def test_add_and_get():
    response = add("Dune", "Frank Herbert", 1965)
    assert response.status_code == 201
    assert response.json() == {
        "id": 1,
        "title": "Dune",
        "author": "Frank Herbert",
        "year": 1965,
    }
    assert client.get("/books/1").json()["title"] == "Dune"


def test_list_filters_by_author():
    add("Dune", "Frank Herbert")
    add("Emma", "Jane Austen")
    titles = [
        b["title"]
        for b in client.get("/books", params={"author": "austen"}).json()
    ]
    assert titles == ["Emma"]


def test_missing_book_is_404():
    response = client.get("/books/99")
    assert response.status_code == 404
    assert response.json() == {"detail": "No book with id 99"}


def test_invalid_book_is_rejected():
    response = client.post("/books", json={"title": "", "author": "Nobody"})
    assert response.status_code == 422
    assert client.get("/books").json() == []
```

```
def test_delete():
    add("Dune", "Frank Herbert")
    assert client.delete("/books/1").status_code == 204
    assert client.delete("/books/1").status_code == 404


def test_docs_page_is_generated():
    assert client.get("/docs").status_code == 200
    schema = client.get("/openapi.json").json()
    assert "/books/{book_id}" in schema["paths"]
```

```
......                                                                   [100%]
6 passed in 0.01s
```

The `autouse=True` fixture runs before every test, even though no test names it, and empties the library so that each test starts afresh. The tests check the status codes, the JSON that comes back, the 404 for a missing book, the 422 for an empty title, and the documentation page.

The chapter folder also has a `pytest.ini` file:

```
[pytest]
# Recent Starlette versions suggest the newer httpx2 package for TestClient.
# httpx 0.28.1 works fine, so hide just that one message (see Chapter 25).
filterwarnings =
    ignore:Using `httpx` with `starlette.testclient` is deprecated
```

Recent versions of Starlette, the toolkit FastAPI is built on, recommend a newer package called `httpx2` for `TestClient`. Using `httpx` is deprecated but still supported, and Starlette prints a warning when it's used. This book deliberately stays with the long-established `httpx` 0.28.1, pinned in `requirements.txt`, because it's also the client the rest of the chapter uses, so this setting hides that one message. To follow Starlette's recommendation instead, run `python -m pip install httpx2` and delete `pytest.ini`: `TestClient` uses `httpx2` automatically when it's installed, and the tests are unchanged. This is typical of real projects: libraries change, and you decide when to follow, rather than letting warnings pile up unread.

Calling an API from Python

Now the client side. This program starts the API server in a background thread, using the threads from Chapter 21, and then calls it, first with `urllib.request` from the standard library and then with **httpx**, a popular HTTP client:

```
# Call the Pocket Library API as a client would: first with urllib from the
# standard library, then with httpx. To keep the example self-contained, the
# server runs in a background thread of this same program.
import json
import socket
import threading
import time
import urllib.error
import urllib.request

import httpx
import uvicorn

import books_api


def start_server():
    with socket.socket() as s:  # ask the system for a free port
        s.bind(("127.0.0.1", 0))
        port = s.getsockname()[1]
    server = uvicorn.Server(
        uvicorn.Config(books_api.app, port=port, log_level="warning")
    )
    threading.Thread(target=server.run, daemon=True).start()
    while not server.started:  # wait until it accepts requests
        time.sleep(0.05)
    return f"http://127.0.0.1:{port}"


base = start_server()

# 1. urllib: GET and POST with the standard library
with urllib.request.urlopen(f"{base}/books") as response:
    print("GET /books ->", response.status, json.loads(response.read()))

request = urllib.request.Request(
    f"{base}/books",
    data=json.dumps(
        {"title": "Dune", "author": "Frank Herbert", "year": 1965}
    ).encode(),
    headers={"Content-Type": "application/json"},
    method="POST",
)
with urllib.request.urlopen(request) as response:
    print("POST /books ->", response.status)
    print("  ", json.loads(response.read()))

try:
    urllib.request.urlopen(f"{base}/books/42")
```

```
except urllib.error.HTTPError as error:
    print("GET /books/42 ->", error.code, json.loads(error.read()))

# 2. httpx: the same work, with less code
with httpx.Client(base_url=base, timeout=5) as client:
    r = client.post(
        "/books",
        json={"title": "Emma", "author": "Jane Austen", "year": 1815},
    )
    print("POST /books ->", r.status_code)
    print("  ", r.json())

    r = client.get("/books", params={"author": "austen"})
    titles = [book["title"] for book in r.json()]
    print("GET /books?author=austen ->", r.status_code, titles)

    r = client.post("/books", json={"title": "Untitled"})
    print(
        "POST without author ->",
        r.status_code,
        r.json()["detail"][0]["msg"],
    )

    r = client.get("/books/42")
    try:
        r.raise_for_status()
    except httpx.HTTPStatusError as error:
        print(
            "raise_for_status:",
            error.response.status_code,
            error.response.json()["detail"],
        )
```

```
GET /books -> 200 []
POST /books -> 201
   {'title': 'Dune', 'author': 'Frank Herbert', 'year': 1965, 'id': 1}
GET /books/42 -> 404 {'detail': 'No book with id 42'}
POST /books -> 201
   {'title': 'Emma', 'author': 'Jane Austen', 'year': 1815, 'id': 2}
GET /books?author=austen -> 200 ['Emma']
POST without author -> 422 Field required
raise_for_status: 404 No book with id 42
```

With `urllib`, you build the request yourself: encode the JSON as bytes, set the `Content-Type` header to say that the body is JSON, and decode the response. An error status raises `HTTPError`.

httpx does the same work with less code. `json=` encodes the body and sets the header, `params=` builds the query string, and `r.json()` decodes the response. Unlike urllib, httpx doesn't raise an exception for error statuses; you check `r.status_code`, or call `r.raise_for_status()`, which raises `HTTPStatusError` for any 400 or 500 code. `base_url` saves repeating the

server's address in every call, and `timeout=5` stops a request from waiting forever, as recommended in Chapter 21.

The 422 response for a book without an author comes from Pydantic, which explains each problem in a list under `"detail"`: here, `Field required`.

Calling Real Web APIs

Everything here works the same with real APIs on the internet: you call `client.get("https://...")` with the service's address. Before you use one:

- **Read its documentation** for the URLs, parameters and response format.
- **Keep keys secret.** Most APIs need an **API key**. Store it in an environment variable and read it with `os.environ["SERVICE_API_KEY"]`; never write it in your code or share it in a repository.
- **Respect the limits.** Most APIs limit how many requests you may make per minute. Cache results you need often, and wait and retry politely when you get status 429 (Too Many Requests).
- **Expect failure.** Networks fail and services go down. Set timeouts, check status codes and handle errors.

Try It

1. Run the server with uvicorn and add a book on the `/docs` page. Then fetch `/books` in your browser.
2. Add a `GET /books/count` endpoint that returns `{"count": ...}`. Where must you put it in the file, and why? (Hint: try putting it after `/books/{book_id}`.)
3. Change `client_demo.py` to ask for `/books/abc`. What status code comes back, and why?

Exercises

1. **Rating.** Add a `PUT /books/{book_id}/rating` endpoint that takes a JSON body like `{"rating": 4}`, accepts only 1 to 5, and returns 404 for an unknown book.

2. **Statistics.** Add a `GET /stats` endpoint that returns the number of books, the number rated, the average rating, and the year of the oldest book.
3. **Friendly client.** Write `find_book(client, book_id)`, which returns a `Book`, or `None` for a 404, and raises for any other error.

Key Takeaways

- Web APIs answer HTTP requests (a method, a URL, maybe a JSON body) with a status code and, usually, JSON.
- FastAPI turns typed functions into endpoints; Pydantic models check incoming data and produce 422 errors automatically. `TestClient` tests an API without a server.
- `urllib.request` calls APIs with the standard library; httpx is simpler, with `json=`, `params=`, `r.json()` and `raise_for_status()`.
- With real APIs, read the documentation, keep keys in environment variables, set timeouts and handle errors and rate limits.

CHAPTER 26 · PROJECT

Project: Automating Everyday Tasks

Many people learn Python for one reason: to stop doing boring things by hand. Renaming a hundred files, finding duplicates, turning a spreadsheet into a web page, backing up a folder every day, emailing a report: each takes minutes by hand and seconds with a script, every time. In this project you'll write a toolbox of small automation scripts, each built safely, with a way to see what it would do before it does it.

Safety First

Automation does exactly what you tell it, very quickly, including the wrong thing. A script that renames files can overwrite them; one that deletes old backups can delete new ones. Every script in this chapter follows three rules:

- **Show before doing.** A **dry run** prints what would happen and changes nothing. You add an option such as `--apply` only when the plan looks right.
- **Never overwrite silently.** Check whether a target already exists.
- **Practice on copies.** The chapter folder has sample folders for each script. Try scripts on copies of real folders until you trust them.

Tidying File Names

Downloads arrive with names like `Scan 1.PDF` and `holiday photo (1).jpg`. This script gives them a consistent form: lower case, hyphens instead of spaces, no brackets:

```
# Give messy file names a tidy form: lower case, hyphens instead of
# spaces, no brackets. Shows the plan first, and changes nothing
# until you add --apply.
import argparse
import re
from pathlib import Path


def tidy(name):
    stem, suffix = Path(name).stem, Path(name).suffix.lower()
    stem = stem.lower()
    stem = re.sub(r"[()\[\]]", "", stem)  # drop brackets
    # Spaces and underscores become one hyphen
    stem = re.sub(r"[\s_]+", "-", stem.strip())
    return stem + suffix


def plan_renames(folder):
    plan, taken = [], {p.name for p in folder.iterdir()}
    for path in sorted(folder.iterdir()):
        if not path.is_file():  # leave folders alone
            continue
        new_name = tidy(path.name)
        if new_name == path.name:
            continue
        if new_name in taken:  # never overwrite another file
            print(f"  skip {path.name!r}: {new_name!r} already exists")
            continue
        taken.add(new_name)
        plan.append((path, path.with_name(new_name)))
    return plan


parser = argparse.ArgumentParser(
    description="Tidy the file names in a folder."
)
parser.add_argument("folder", type=Path)
parser.add_argument(
    "--apply",
    action="store_true",
    help="really rename (default: only show)",
)
args = parser.parse_args()

plan = plan_renames(args.folder)
for old, new in plan:
    print(f"  {old.name!r:28} -> {new.name!r}")
if args.apply:
    for old, new in plan:
        old.rename(new)
    print(f"Renamed {len(plan)} files.")
```

```
else:
    print(f"{len(plan)} files would be renamed. Add --apply to do it.")
```

`tidy` works out the new name with string methods and regular expressions from Chapter 9. `plan_renames` builds a list of changes without touching anything. It leaves folders alone, renaming only files, and skips any rename that would overwrite an existing file. Only with `--apply` does the script rename. This short program runs it twice on the sample `downloads` folder, which also contains a folder called `Old Photos`, first as a dry run, then for real:

```
# Run tidy_names.py twice: first to see the plan, then to apply it.
import subprocess
import sys
from pathlib import Path

for extra in ([], ["--apply"]):
    command = ["tidy_names.py", "downloads", *extra]
    print("$ python", " ".join(command))
    subprocess.run([sys.executable, *command])
print("Now in downloads:")
for path in sorted(Path("downloads").iterdir()):
    print("  ", path.name)
```

```
$ python tidy_names.py downloads
  'Holiday Photo.JPG'          -> 'holiday-photo.jpg'
  'My Notes.TXT'               -> 'my-notes.txt'
  'Scan 1.PDF'                 -> 'scan-1.pdf'
  'budget 2026.xlsx'           -> 'budget-2026.xlsx'
  'holiday photo (1).jpg'      -> 'holiday-photo-1.jpg'
  'scan 2.pdf'                 -> 'scan-2.pdf'
6 files would be renamed. Add --apply to do it.
$ python tidy_names.py downloads --apply
  'Holiday Photo.JPG'          -> 'holiday-photo.jpg'
  'My Notes.TXT'               -> 'my-notes.txt'
  'Scan 1.PDF'                 -> 'scan-1.pdf'
  'budget 2026.xlsx'           -> 'budget-2026.xlsx'
  'holiday photo (1).jpg'      -> 'holiday-photo-1.jpg'
  'scan 2.pdf'                 -> 'scan-2.pdf'
Renamed 6 files.
Now in downloads:
   Old Photos
   budget-2026.xlsx
   holiday-photo-1.jpg
   holiday-photo.jpg
   my-notes.txt
   scan-1.pdf
   scan-2.pdf
```

Reporting on a Folder

Before cleaning up a folder, find out what's in it. This report counts files by type, adds up their sizes, lists the largest, and finds files with identical contents:

```
# Report what a folder tree contains: files per type, total size,
# the biggest files, and files with the same contents.
import hashlib
from collections import Counter, defaultdict
from pathlib import Path

root = Path("documents")
files = sorted(p for p in root.rglob("*") if p.is_file())

by_type = Counter(p.suffix or "(none)" for p in files)
total = sum(p.stat().st_size for p in files)
print(f"{len(files)} files, {total} bytes in {root}/")
for suffix, count in by_type.most_common():
    print(f"  {suffix:6} {count}")

print("Largest files:")
for p in sorted(files, key=lambda p: p.stat().st_size, reverse=True)[:3]:
    print(f"  {p.stat().st_size:5} bytes  {p.relative_to(root)}")

# Same contents = same fingerprint (hash), whatever the file is called
groups = defaultdict(list)
for p in files:
    groups[hashlib.sha256(p.read_bytes()).hexdigest()].append(
        p.relative_to(root)
    )
for same in groups.values():
    if len(same) > 1:
        print("Duplicates:", ", ".join(str(p) for p in same))
```

```
6 files, 306 bytes in documents/
  .txt   3
  .md    2
  .csv   1
Largest files:
     82 bytes  letters/thanks.md
     65 bytes  notes/garden.txt
     45 bytes  budget.csv
Duplicates: notes/shopping-copy.txt, notes/shopping.txt
```

To find duplicates, the script computes a **hash** of each file's contents with `hashlib.sha256`: a fingerprint, 64 hexadecimal characters long, that's the same for identical contents and, in practice, different for any other contents. Files are grouped by fingerprint, whatever they're called, so `shopping-copy.txt` is found to be a copy of `shopping.txt`. This works just

as well for large photos and videos, and is much faster than comparing every file with every other.

From CSV to a Web Page

Here's a reading log, kept as a CSV file:

```
title,author,finished,pages,rating
Dune,Frank Herbert,2026-01-19,612,5
Emma,Jane Austen,2026-02-27,474,4
Educated,Tara Westover,2026-04-02,334,5
The Hobbit,J. R. R. Tolkien,2026-05-11,310,4
Pride & Prejudice,Jane Austen,2026-07-30,432,3
```

This script turns it into an HTML page that you can open in any browser or put on a website:

```
# Turn a CSV file into an HTML page you can open in any browser.
import csv
import html
from pathlib import Path

with open("reading_log.csv", newline="", encoding="utf-8") as f:
    rows = list(csv.DictReader(f))


def cell(text, tag="td"):
    return f"<{tag}>{html.escape(text)}</{tag}>"  # & < > become safe HTML


header = "".join(cell(name.title(), "th") for name in rows[0])
body = "\n".join(
    "<tr>" + "".join(cell(v) for v in row.values()) + "</tr>"
    for row in rows
)
pages = sum(int(row["pages"]) for row in rows)

page = f"""<!doctype html>
<html lang="en">
<head><meta charset="utf-8"><title>Reading log</title>
<style>
  body {{ font-family: sans-serif; }}
  td, th {{ padding: 4px 12px; }}
</style></head>
<body>
<h1>Reading log</h1>
<p>{len(rows)} books, {pages:,} pages.</p>
<table>
<tr>{header}</tr>
{body}
</table>
</body>
</html>
```

```
"""
Path("reading_log.html").write_text(page, encoding="utf-8")
print(f"Wrote reading_log.html ({len(rows)} rows)")
print(cell("Pride & Prejudice"))  # & becomes &amp;
```

```
Wrote reading_log.html (5 rows)
<td>Pride &amp;amp; Prejudice</td>
```

An f-string builds the page, with doubled braces `{{ }}` where the page's style rules need real braces. Everything that comes from the data goes through `html.escape`, which turns `&`, `<` and `>` into codes that a browser shows as text. Without it, the ampersand in *Pride & Prejudice* would be misread, and a title containing `<script>` could run code in the reader's browser. Escape data whenever you put it into HTML, just as you put values through placeholders in SQL in Chapter 23.

Backups with Rotation

This script zips a folder into a file named by date and keeps only the newest three backups. To show the rotation in one run, it pretends to run on five days in a row:

```
# Zip a folder into backups/, named by date, and keep only the newest three.
import zipfile
from datetime import date, timedelta
from pathlib import Path

SOURCE = Path("documents")
BACKUPS = Path("backups")
KEEP = 3


def make_backup(day):
    """Zip SOURCE into backups/; None if today's backup already exists."""
    BACKUPS.mkdir(exist_ok=True)
    target = BACKUPS / f"documents-{day.isoformat()}.zip"
    if target.exists():  # never overwrite a backup silently
        return None
    with zipfile.ZipFile(
        target, "w", compression=zipfile.ZIP_DEFLATED
    ) as zf:
        for path in sorted(SOURCE.rglob("*")):
            if path.is_file():
                zf.write(path, path.relative_to(SOURCE.parent))
    return target


def remove_old_backups():
    # ISO dates (2026-10-15) sort in time order
```

```
    backups = sorted(BACKUPS.glob("documents-*.zip"))
    for old in backups[:-KEEP]:
        old.unlink()
        print("  removed", old.name)


# Pretend the script ran on five days in a row
# (normally you would use date.today()), then run again on the last day
start = date(2026, 10, 11)
days = [start + timedelta(days=n) for n in range(5)]
for day in [*days, days[-1]]:
    target = make_backup(day)
    if target is None:
        print("Skipped: a backup for", day, "already exists")
        continue
    print("Created", target.name)
    last = target
    remove_old_backups()

print("Kept:")
for path in sorted(BACKUPS.iterdir()):
    print("  ", path.name)
with zipfile.ZipFile(last) as zf:
    print(f"{last.name} contains {len(zf.namelist())} files:")
    for name in zf.namelist():
        print("  ", name)
    print("Archive test:", zf.testzip() or "all files OK")
```

```
Created documents-2026-10-11.zip
Created documents-2026-10-12.zip
Created documents-2026-10-13.zip
Created documents-2026-10-14.zip
  removed documents-2026-10-11.zip
Created documents-2026-10-15.zip
  removed documents-2026-10-12.zip
Skipped: a backup for 2026-10-15 already exists
Kept:
   documents-2026-10-13.zip
   documents-2026-10-14.zip
   documents-2026-10-15.zip
documents-2026-10-15.zip contains 6 files:
   documents/budget.csv
   documents/letters/renewal.md
   documents/letters/thanks.md
   documents/notes/garden.txt
   documents/notes/shopping-copy.txt
   documents/notes/shopping.txt
Archive test: all files OK
```

Before writing anything, `make_backup` checks whether a backup for that day already exists, and if so returns `None` instead of replacing it: opening a zip file with `"w"` would silently empty an existing one. The demo runs the last day twice to show it. `zipfile` creates the archive; each file is stored with its path relative to the folder's parent, so the archive unpacks into a `documents` folder.

Because the names use ISO dates, sorting them alphabetically sorts them by date, and `backups[:-KEEP]` is every backup except the newest three. `testzip` reads every file back and checks it, returning `None` if all is well. A backup you haven't checked isn't a backup.

Running Tasks on a Schedule

Some tasks should run regularly. The standard library's `sched` module runs functions at set times within a running program:

```
# Run tasks at set intervals with the standard library's sched module.
# The intervals here are tenths of a second so the example finishes quickly;
# for real use, think minutes or hours.
import sched
import time

scheduler = sched.scheduler(time.monotonic, time.sleep)
runs = {"check inbox": 0, "back up notes": 0}


def every(seconds, name, times, start):
    runs[name] += 1
    print(f"run {runs[name]} of {name!r}")
    if runs[name] < times:
        # Book the next run at an exact time, so small delays don't add up
        next_time = start + seconds * (runs[name] + 1)
        scheduler.enterabs(
            next_time, 1, every, (seconds, name, times, start)
        )


start = time.monotonic()
scheduler.enterabs(start + 0.1, 1, every, (0.1, "check inbox", 4, start))
scheduler.enterabs(
    start + 0.25, 1, every, (0.25, "back up notes", 2, start)
)
scheduler.run()  # blocks until no tasks are left
print("All tasks done.")
```

```
run 1 of 'check inbox'
run 2 of 'check inbox'
run 1 of 'back up notes'
run 3 of 'check inbox'
run 4 of 'check inbox'
run 2 of 'back up notes'
All tasks done.
```

`enterabs` books a task at an exact time on the `time.monotonic` clock, a clock that only ever moves forward, and each run books the next one. Scheduling

from a fixed start time, rather than "0.1 seconds after the last run finished", stops small delays from adding up over hours.

For real schedules, like a backup every night, it's usually better to let the operating system start your script: **Task Scheduler** on Windows, or **cron** on macOS and Linux. Then nothing needs to keep running between jobs, and the job still runs after a restart. A cron line like `0 2 * * * /path/to/.venv/bin/python /path/to/backup.py` runs the backup at 2:00 every night, using the virtual environment's Python.

Sending Email

The last script builds an email with the reading log attached, and sends it only when asked:

```
# Build an email with an attachment, and send it only when asked to.
# The SMTP server and password come from environment variables,
# never from the code.
import argparse
import os
import smtplib
from email.message import EmailMessage
from pathlib import Path

parser = argparse.ArgumentParser()
parser.add_argument(
    "--send", action="store_true", help="really send (default: only show)"
)
args = parser.parse_args()

message = EmailMessage()
message["From"] = "reports@pocket-library.example"
message["To"] = "reader@pocket-library.example"
message["Subject"] = "Your reading log"
message.set_content(
    "Hello,\n\nYour reading log is attached.\n\nPocket Library\n"
)
message.add_attachment(
    Path("reading_log.csv").read_bytes(),
    maintype="text",
    subtype="csv",
    filename="reading_log.csv",
)

if args.send:
    host = os.environ["SMTP_HOST"]  # for example smtp.your-provider.example
    user = os.environ["SMTP_USER"]
    password = os.environ["SMTP_PASSWORD"]
    with smtplib.SMTP(host, 587, timeout=30) as server:
        server.starttls()  # encrypt before logging in
        server.login(user, password)
        server.send_message(message)
```

```
    print("Sent.")
else:
    print("Not sent (add --send). The message would be:")
    for header in ("From", "To", "Subject"):
        print(f"  {header}: {message[header]}")
    for part in message.iter_parts():
        kind = "attachment" if part.is_attachment() else "body"
        size = len(part.get_content())
        print(
            f"  part: {part.get_content_type()}, {size} characters, {kind}"
        )
    print(
        "  attachment name:",
        next(message.iter_attachments()).get_filename(),
    )
```

Without `--send`, it shows the message it would send:

```
Not sent (add --send). The message would be:
  From: reports@pocket-library.example
  To: reader@pocket-library.example
  Subject: Your reading log
  part: text/plain, 54 characters, body
  part: text/csv, 237 characters, attachment
  attachment name: reading_log.csv
```

`EmailMessage` builds the message: headers, a plain-text body, and the CSV file as an attachment. To send it, `smtplib` connects to a mail server, encrypts the connection with `starttls`, logs in and sends. The server's address, username and password come from **environment variables**, never from the code, so the script can be shared and committed safely. Most email providers require an app-specific password for scripts; check your provider's help pages.

Sending needs a real mail account, so for this book the sending code was tested without one. The test below replaces `smtplib.SMTP` with a fake class that records what it's asked to do, using pytest's `monkeypatch` fixture, which undoes the change after the test. `runpy.run_path` runs the script as if from the command line:

```
# Test the --send branch of email_report.py without a real mail server:
# monkeypatch swaps smtplib.SMTP for a fake that records what it's asked.
import runpy
import smtplib
import sys


class FakeSMTP:
    instances = []

    def __init__(self, host, port, timeout):
        self.calls = [("connect", host, port)]
```

```
        FakeSMTP.instances.append(self)

    def __enter__(self):
        return self

    def __exit__(self, *exc):
        return False

    def starttls(self):
        self.calls.append(("starttls",))

    def login(self, user, password):
        self.calls.append(("login", user))

    def send_message(self, message):
        self.calls.append(("send", message["To"]))


def test_send(monkeypatch, capsys):
    monkeypatch.setattr(smtplib, "SMTP", FakeSMTP)
    monkeypatch.setattr(sys, "argv", ["email_report.py", "--send"])
    monkeypatch.setenv("SMTP_HOST", "smtp.example")
    monkeypatch.setenv("SMTP_USER", "reports")
    monkeypatch.setenv("SMTP_PASSWORD", "not-a-real-password")
    runpy.run_path("email_report.py")
    assert FakeSMTP.instances[-1].calls == [
        ("connect", "smtp.example", 587),
        ("starttls",),  # encrypted before the password is sent
        ("login", "reports"),
        ("send", "reader@pocket-library.example"),
    ]
    assert capsys.readouterr().out == "Sent.\n"
```

```
.                                                                    [100%]
1 passed in 0.01s
```

The test confirms the order that matters for security: the connection is encrypted before the password is sent. Replacing a piece of a program with a fake for a test is called **mocking**, and it's the standard way to test code that talks to mail servers, web APIs, payment services and anything else you can't or shouldn't use in a test.

Try It

1. Run `tidy_names.py` on a copy of your own downloads folder, without `--apply`. Do you agree with every rename?
2. Change `folder_report.py` to report on a folder of yours and list the ten largest files.

3. Add a column to `reading_log.csv` and check that `csv_to_html.py` shows it without any change to the code.

Exercises

1. **Log summary.** Write `log_summary(path)`, which reads a log file like `app.log` (each line has a date, a time, a level and a message), and returns a `Counter` of the levels and a `Counter` of the error messages.
2. **Sort by type.** Write `sort_by_type(folder, apply=False)`, which plans moves of each file into a subfolder by type (documents, pictures, spreadsheets or other), returns the plan, and makes the moves only when `apply` is true.
3. **Text table.** Write `text_table(rows)`, which formats a list of dictionaries, such as rows from `csv.DictReader`, as a plain-text table, with each column as wide as its widest value.

Key Takeaways

- Automation scripts should show a dry run first, never overwrite silently, and be tried on copies.
- `pathlib`, `re`, `hashlib`, `csv`, `html`, `zipfile`, `sched`, `email` and `smtplib`, all in the standard library, cover most everyday automation.
- Escape data that goes into HTML with `html.escape`. Keep passwords in environment variables.
- Let the operating system schedule regular jobs. Test code that talks to outside services with mocks.

CHAPTER 27

Where to Go Next

You started this book with `print("Hello, Python!")`. Since then you've written programs that make decisions and loop, organized data in lists, dictionaries and classes, read and written files, handled errors, tested and debugged your code, packaged it, typed it, made it concurrent and fast, and built a database app, a data analysis, a web API and a set of automation tools. That's a solid foundation for nearly anything you want to do with Python. This last chapter suggests where to go from here, and how to keep improving.

Build Things

The single best way to get better is to build programs you care about. Start small, finish them, then make them bigger. Some ideas, grouped by the chapters they build on:

- **Pocket Library, continued**: combine the SQLite store from Chapter 23 with the web API from Chapter 25, so that the API keeps its books between restarts. Add a search endpoint, and tests for everything.
- **A personal finance tracker**: import your bank's CSV exports, categorize spending with rules, and chart it by month with pandas and matplotlib (Chapter 24).
- **A habit or reading tracker** for the command line, with streaks and statistics.
- **A file organizer** that watches a folder and sorts new files by type and date (Chapters 11 and 26).

- **A quiz game** that reads its questions from a JSON file, keeps scores, and gets harder as you improve.
- **A small website** with Flask or Django, two popular web frameworks, showing your data in HTML pages.

When you get stuck, break the problem into smaller pieces, write a test for the piece that doesn't work, and read the error messages from the bottom up.

Directions to Explore

Python is used in almost every field. Here are the main directions, with the tools to look up first:

Direction	Start with
Web development	FastAPI (Chapter 25), Flask, Django
Data analysis and science	pandas, NumPy, Jupyter, matplotlib, seaborn
Machine learning and AI	scikit-learn, then PyTorch; the Python libraries that AI companies provide for their APIs
Automation and scripting	the standard library (Chapter 26), Playwright for web browsers, openpyxl for Excel files
Desktop applications	tkinter (included with Python), Qt for Python
Games	pygame, Arcade
Hardware and electronics	MicroPython and CircuitPython on microcontrollers, Raspberry Pi
Testing and quality	pytest (Chapter 15), ruff for linting and formatting, mypy (Chapter 19)

Pick one that matches what you want to build, and go deep before you go wide.

Habits of Good Programmers

- **Use version control.** Learn **Git**, which records every change to your code so that you can undo mistakes, see what changed when, and work with

others. Put your projects on a hosting service such as GitHub or GitLab. This is the next skill to learn after this book.

- **Format and lint automatically.** This book's code was formatted with `ruff format` and checked with `ruff check`, which catch mistakes and keep style consistent with no effort from you.
- **Write tests as you go**, and run them, together with mypy, before every change you keep.
- **Read other people's code.** The standard library's own source is readable Python; so are many popular packages. You'll pick up idioms you won't find in tutorials.
- **Read the documentation.** docs.python.org has the official tutorial, the library reference and the "What's New" pages for each version. Python releases a new version every October, each supported with fixes for five years, so it's worth reading what changed once a year.
- **Keep learning the fundamentals.** Data structures, algorithms and how computers work make you better in every language.

Learning with AI Assistants

AI coding assistants can explain code, suggest fixes and write programs from descriptions. Used well, they're excellent tutors: ask one to explain an error message, review your solution to an exercise, or suggest a better data structure, and then make sure you understand the answer. Used badly, they stop you from learning: if you paste in every exercise and copy every answer, you'll be no better at the end.

A good rule while you're learning is to write the code yourself first, then ask for a review. And always treat generated code like code from a stranger: read it, run it, and test it, exactly as this book has tested every example.

Join the Community

Python has a famously welcoming community. You can ask questions and read discussions on the official forum at discuss.python.org, find a local Python user group or meetup, and attend a PyCon conference, which are held around the world, often with financial aid for those who need it. When you're

ready, contribute to an open-source project: fixing a typo in documentation is a perfectly good first contribution.

And you'll find that explaining what you've learned to someone else, the rubber duck from Chapter 16 included, is the best way to understand it yourself.

Happy coding.

Key Takeaways

- Build projects you care about, starting small and finishing them; extend Pocket Library or try one of the ideas above.
- Choose a direction (web, data, AI, automation, desktop, games or hardware) and go deep with its main tools.
- Learn Git next, and keep the habits from this book: formatting, linting, tests, type checks and reading documentation.
- Use AI assistants as tutors and reviewers, not as a substitute for writing code yourself, and test everything they produce.

Appendix A: Solutions to the Exercises

Here are worked solutions to every exercise in the book. Each one was run on Python 3.12, 3.13, 3.14 and 3.15, and the output shown is the output it produced. There's usually more than one good answer: if yours works and you can explain it, it's right. Compare it with the one here for ideas.

Chapter 1: Welcome to Python

Your text will differ: any program that prints your details and lets Python calculate `365 * 24` is correct.

```
# Exercise 1: a program that introduces you (put in your own details)
print("My name is Sam.")
print("I am learning Python.")
print("One year has", 365 * 24, "hours.")
```

```
My name is Sam.
I am learning Python.
One year has 8760 hours.
```

The calculations in the shell:

```
>>> 7 * 24 * 60
10080
>>> 365 * 24
8760
>>> print("Hip, hip, " * 2 + "hooray!")
Hip, hip, Hip, hip, hooray!
```

Chapter 2: Variables, Numbers and Strings

```
width = 7.5
height = 4
area = width * height
perimeter = 2 * (width + height)
print("Area:", area)
print("Perimeter:", perimeter)
```

```
Area: 30.0
Perimeter: 23.0
```

```
total_seconds = 10_000
hours = total_seconds // 3600
minutes = total_seconds % 3600 // 60
seconds = total_seconds % 60
print(f"{total_seconds} seconds is {hours} h {minutes} min {seconds} s")
```

```
10000 seconds is 2 h 46 min 40 s
```

```
first = "grace"
last = "hopper"
initials = first[0].upper() + "." + last[0].upper() + "."
print(initials)
```

```
G.H.
```

Chapter 3: Input, Output and Conversions

Each program is shown with a sample run.

```
celsius = float(input("Temperature in °C: "))
fahrenheit = celsius * 9 / 5 + 32
print(f"{celsius:.1f} °C is {fahrenheit:.1f} °F")
```

```
Temperature in °C: 36.6
36.6 °C is 97.9 °F
```

```
years = int(input("Your age in years: "))
days = years * 365
print(f"You are about {days:,} days old.")
```

```
Your age in years: 34
You are about 12,410 days old.
```

```
item = input("Item: ")
quantity = int(input("Quantity: "))
unit_price = float(input("Unit price: "))
total = quantity * unit_price
print()
print(f"{'Item':<12}{'Qty':>5}{'Price':>10}{'Total':>10}")
print(f"{item:<12}{quantity:>5}{unit_price:>10.2f}{total:>10.2f}")
```

```
Item: Pencil
Quantity: 12
Unit price: 0.35

Item          Qty     Price     Total
Pencil         12      0.35      4.20
```

Chapter 4: Making Decisions

```
year = int(input("Year: "))
if (year % 4 == 0 and year % 100 != 0) or year % 400 == 0:
    print(year, "is a leap year.")
else:
    print(year, "is not a leap year.")
```

```
Year: 1900
1900 is not a leap year.
```

```
a = float(input("First number: "))
b = float(input("Second number: "))
c = float(input("Third number: "))
largest = a
if b > largest:
    largest = b
if c > largest:
    largest = c
print("The largest is", largest)
```

```
First number: 4
Second number: 19.5
Third number: 7
The largest is 19.5
```

```
weight = float(input("Weight (kg): "))
height = float(input("Height (m): "))
bmi = weight / height**2
if bmi < 18.5:
    category = "underweight"
elif bmi < 25:
    category = "healthy weight"
elif bmi < 30:
    category = "overweight"
else:
    category = "obese"
print(f"BMI {bmi:.1f}: {category}")
```

```
Weight (kg): 68
Height (m): 1.72
BMI 23.0: healthy weight
```

Chapter 5: Loops

For FizzBuzz, test divisibility by 15 first; otherwise 15 would print `Fizz`. The password program uses the `for` loop's `else` block, which runs only when the loop ends without `break`.

```
for n in range(1, 21):
    if n % 15 == 0:
        print("FizzBuzz")
    elif n % 3 == 0:
        print("Fizz")
    elif n % 5 == 0:
        print("Buzz")
    else:
        print(n)
```

```
1
2
Fizz
4
Buzz
Fizz
7
8
Fizz
Buzz
11
Fizz
13
14
FizzBuzz
16
17
Fizz
```

```
19
Buzz
```

```
number = int(input("A whole number: "))
total = 0
n = abs(number)
while n > 0:
    total += n % 10
    n //= 10
print(f"The digits of {number} add up to {total}.")
```

```
A whole number: 90517
The digits of 90517 add up to 22.
```

```
PASSWORD = "open sesame"
for attempt in range(1, 4):
    entry = input(f"Password (attempt {attempt} of 3): ")
    if entry == PASSWORD:
        print("Welcome in!")
        break
    print("Wrong password.")
else:
    print("Too many attempts. Locked out.")
```

```
Password (attempt 1 of 3): opensesame
Wrong password.
Password (attempt 2 of 3): Open Sesame
Wrong password.
Password (attempt 3 of 3): open sesame
Welcome in!
```

Chapter 6: Lists and Tuples

```
scores = [72, 88, 95, 61, 79, 88]
print("Highest:", max(scores))
print("Lowest:", min(scores))
print(f"Average: {sum(scores) / len(scores):.1f}")
print("Sorted:", sorted(scores, reverse=True))
```

```
Highest: 95
Lowest: 61
Average: 80.5
Sorted: [95, 88, 88, 79, 72, 61]
```

```
names = ["Asha", "Ben", "Asha", "Chen", "Ben", "Dev"]
unique = []
for name in names:
    if name not in unique:
        unique.append(name)
print(unique)
```

```
['Asha', 'Ben', 'Chen', 'Dev']
```

```
items = [1, 2, 3, 4, 5]
steps = 2
rotated = items[-steps:] + items[:-steps]
print(rotated)
```

```
[4, 5, 1, 2, 3]
```

Chapter 7: Dictionaries and Sets

```
capitals = {"France": "Paris", "Japan": "Tokyo", "Kenya": "Nairobi"}
countries = {}
for country, capital in capitals.items():
    countries[capital] = country
print(countries)
print(countries["Tokyo"])
```

```
{'Paris': 'France', 'Tokyo': 'Japan', 'Nairobi': 'Kenya'}
Japan
```

```
text = "Mississippi"
counts = {}
for letter in text.lower():
    counts[letter] = counts.get(letter, 0) + 1
for letter in sorted(counts):
    print(letter, counts[letter])
```

```
i 4
m 1
p 2
s 4
```

```
priya = {"Asha", "Ben", "Chen", "Dev"}
omar = {"Chen", "Dev", "Ella"}
print("Common friends:", sorted(priya & omar))
print("Priya could introduce Omar to:", sorted(priya - omar))
```

```
Common friends: ['Chen', 'Dev']
Priya could introduce Omar to: ['Asha', 'Ben']
```

Chapter 8: Functions

All three functions are in one file. The `if __name__ == "__main__":` block, explained in Chapter 11, runs the examples only when the file is run directly.

```
def is_palindrome(text):
    """True if text reads the same backwards, ignoring case,
    spaces and punctuation."""
    letters = ""
    for ch in text.lower():
        if ch.isalnum():
            letters += ch
    return letters == letters[::-1]


def count_vowels(text):
    """Return how many vowels (a, e, i, o, u) are in text."""
    count = 0
    for ch in text.lower():
        if ch in "aeiou":
            count += 1
    return count


def largest(numbers):
    """Return the largest number in a non-empty list, without max()."""
    best = numbers[0]
    for n in numbers[1:]:
        if n > best:
            best = n
    return best


if __name__ == "__main__":
    print(is_palindrome("Never odd or even"))
    print(is_palindrome("Python"))
    print(count_vowels("Programming in Python"))
    print(largest([3, 41, 7, 41, -2]))
```

```
True
False
5
41
```

Chapter 9: Working with Text

`ord` and `chr` convert between characters and numbers; `% 26` wraps from Z back to A.

```
import re


def caesar(text, shift):
    """Shift each letter by `shift` places in the alphabet, keeping case."""
    result = ""
    for ch in text:
        if ch.isalpha():
            base = ord("A") if ch.isupper() else ord("a")
            result += chr((ord(ch) - base + shift) % 26 + base)
        else:
            result += ch
    return result


def my_title(text):
    """Capitalize the first letter of each word, like str.title()."""
    words = text.split(" ")
    fixed = []
    for word in words:
        fixed.append(word[:1].upper() + word[1:].lower())
    return " ".join(fixed)


def hashtags(text):
    """Return the hashtags in text, in order, without the # sign."""
    return re.findall(r"#(\w+)", text)


if __name__ == "__main__":
    secret = caesar("Meet at noon!", 3)
    print(secret)
    print(caesar(secret, -3))
    print(my_title("the lord OF the rings"))
    print(hashtags("Loving #Python and #100DaysOfCode! #python"))
```

```
Phhw dw qrrq!
Meet at noon!
The Lord Of The Rings
['Python', '100DaysOfCode', 'python']
```

Chapter 10: Files and Exceptions

The examples use `books.csv` from the chapter.

```
import csv
from pathlib import Path


def file_stats(path):
    """Return (lines, words, characters) for a text file."""
    text = Path(path).read_text(encoding="utf-8")
    return len(text.splitlines()), len(text.split()), len(text)


def average_rating(csv_path):
    """Return the average of the 'rating' column of a CSV file."""
    with open(csv_path, newline="", encoding="utf-8") as f:
        ratings = []
        for row in csv.DictReader(f):
            ratings.append(int(row["rating"]))
    return sum(ratings) / len(ratings)


def divide(a_text, b_text):
    """Divide two numbers given as text; return a message on errors."""
    try:
        return f"{float(a_text) / float(b_text):g}"
    except ValueError:
        return "Please type numbers only."
    except ZeroDivisionError:
        return "You can't divide by zero."


if __name__ == "__main__":
    print(file_stats("books.csv"))
    print(f"{average_rating('books.csv'):.1f}")
    print(
        divide("7", "2"), "|", divide("7", "0"), "|", divide("seven", "2")
    )
```

```
(6, 17, 213)
4.2
3.5 | You can't divide by zero. | Please type numbers only.
```

Chapter 11: Modules and the Standard Library

The bars show the familiar shape of two dice: 7 is the most common total, and 2 and 12 the rarest.

```
import random
from collections import Counter
from datetime import date


def roll_dice(rolls, seed=None):
    """Roll two dice `rolls` times; return a Counter of the totals."""
    rng = random.Random(seed)
```

```
    totals = Counter()
    for _ in range(rolls):
        totals[rng.randint(1, 6) + rng.randint(1, 6)] += 1
    return totals


def days_between(a, b):
    """Days between two dates given as 'YYYY-MM-DD' text."""
    return abs((date.fromisoformat(b) - date.fromisoformat(a)).days)


if __name__ == "__main__":
    totals = roll_dice(1000, seed=3)
    for total in range(2, 13):
        print(f"{total:2} {'#' * (totals[total] // 5)}")
    print(days_between("2026-01-01", "2026-10-15"))
```

```
 2 #####
 3 ###########
 4 ###############
 5 ########################
 6 ##############################
 7 ##############################
 8 ############################
 9 ########################
10 ################
11 ########
12 #####
287
```

Chapter 12: Classes and Objects

```
class Rectangle:
    def __init__(self, width, height):
        self.width = width
        self.height = height

    def area(self):
        return self.width * self.height

    def perimeter(self):
        return 2 * (self.width + self.height)

    def is_square(self):
        return self.width == self.height


class Tally:
    """Count clicks, like a hand-held tally counter."""

    def __init__(self):
        self.count = 0

    def click(self, times=1):
        self.count += times
```

```
        return self.count

    def reset(self):
        self.count = 0


class Thermostat:
    def __init__(self, target=20.0, low=5.0, high=30.0):
        self.low = low
        self.high = high
        self.target = target

    def set_target(self, value):
        """Set the target within the allowed range; return it."""
        self.target = min(max(value, self.low), self.high)
        return self.target


if __name__ == "__main__":
    r = Rectangle(4, 4)
    print(r.area(), r.perimeter(), r.is_square())
    t = Tally()
    t.click()
    print(t.click(3))
    print(Thermostat().set_target(42))
```

```
16 16 True
4
30.0
```

Chapter 13: Inheritance, Dataclasses and Special Methods

`Fraction` stores fractions in lowest terms, with any minus sign on top, so that equal fractions compare equal.

```
import math
from dataclasses import dataclass


class Employee:
    def __init__(self, name, salary):
        self.name = name
        self.salary = salary

    def yearly_cost(self):
        return self.salary * 12


class Manager(Employee):
    def __init__(self, name, salary, bonus):
        super().__init__(name, salary)
        self.bonus = bonus
```

```
    def yearly_cost(self):
        return super().yearly_cost() + self.bonus


class Fraction:
    def __init__(self, top, bottom):
        if bottom == 0:
            raise ZeroDivisionError("bottom can't be zero")
        g = math.gcd(top, bottom)
        sign = -1 if bottom < 0 else 1
        self.top = sign * top // g
        self.bottom = sign * bottom // g

    def __add__(self, other):
        return Fraction(
            self.top * other.bottom + other.top * self.bottom,
            self.bottom * other.bottom,
        )

    def __eq__(self, other):
        return (self.top, self.bottom) == (other.top, other.bottom)

    def __str__(self):
        return f"{self.top}/{self.bottom}"


@dataclass(frozen=True)
class Point:
    x: float
    y: float

    def distance_to(self, other):
        return math.hypot(self.x - other.x, self.y - other.y)


if __name__ == "__main__":
    print(Manager("Ana", 5000, 8000).yearly_cost())
    print(Fraction(1, 2) + Fraction(1, 3), Fraction(2, 4) == Fraction(1, 2))
    print(Point(0, 0).distance_to(Point(3, 4)))
```

```
68000
5/6 True
5.0
```

Chapter 14: Comprehensions, Iterators and Generators

```
def even_squares(numbers):
    """Squares of the even numbers, in order."""
    return [n * n for n in numbers if n % 2 == 0]


def word_lengths(text):
```

```
    """A dictionary from each word (lower case) to its length."""
    return {word.lower(): len(word) for word in text.split()}


def chunked(items, size):
    """Yield lists of up to `size` items at a time."""
    chunk = []
    for item in items:
        chunk.append(item)
        if len(chunk) == size:
            yield chunk
            chunk = []
    if chunk:
        yield chunk


if __name__ == "__main__":
    print(even_squares(range(10)))
    print(word_lengths("The quick brown Fox"))
    print(list(chunked("abcdefg", 3)))
```

```
[0, 4, 16, 36, 64]
{'the': 3, 'quick': 5, 'brown': 5, 'fox': 3}
[['a', 'b', 'c'], ['d', 'e', 'f'], ['g']]
```

Chapter 15: Testing with pytest

The functions under test are in `solution_tests.py`; the tests are in `test_solution_tests.py`.

```
"""Chapter 15 solutions: tests for functions from earlier chapters."""


def is_leap_year(year):
    return (year % 4 == 0 and year % 100 != 0) or year % 400 == 0


def fizzbuzz(n):
    if n % 15 == 0:
        return "FizzBuzz"
    if n % 3 == 0:
        return "Fizz"
    if n % 5 == 0:
        return "Buzz"
    return str(n)
```

```
import pytest

from solution_tests import fizzbuzz, is_leap_year


@pytest.mark.parametrize(
    "year, leap", [(2024, True), (2023, False), (1900, False), (2000, True)]
```

```
)
def test_is_leap_year(year, leap):
    assert is_leap_year(year) is leap


@pytest.mark.parametrize(
    "n, expected",
    [
        (1, "1"),
        (3, "Fizz"),
        (5, "Buzz"),
        (15, "FizzBuzz"),
        (30, "FizzBuzz"),
    ],
)
def test_fizzbuzz(n, expected):
    assert fizzbuzz(n) == expected
```

```
.........                                                                [100%]
9 passed in 0.01s
```

Chapter 16: Debugging, Logging and Reading Tracebacks

The two bugs were that the list wasn't sorted, and that for an even number of values, the median is the average of the two middle ones.

```
import logging

log = logging.getLogger(__name__)


def median(numbers):
    """Return the median; fixed version of the buggy one in the exercise."""
    if not numbers:
        raise ValueError("median of an empty list")
    ordered = sorted(numbers)  # the bug: the list wasn't sorted
    middle = len(ordered) // 2
    if len(ordered) % 2 == 1:
        return ordered[middle]
    # The second bug: both middle values are needed
    return (ordered[middle - 1] + ordered[middle]) / 2


def safe_int(text, default=0):
    try:
        return int(text)
    except ValueError:
        log.warning("not a whole number: %r; using %d", text, default)
        return default


if __name__ == "__main__":
```

```
    logging.basicConfig(format="%(levelname)s: %(message)s")
    print(median([7, 1, 3]), median([4, 1, 3, 2]))
    print(safe_int("42"), safe_int("4x2"))
```

```
3 2.5
WARNING: not a whole number: '4x2'; using 0
42 0
```

Chapter 17: Virtual Environments, Packages and Project Layout

Here the new function is added in a separate file, so that it runs from the chapter folder; in your project, the function goes in `books.py` and the command in `cli.py`, as the comment shows.

```
# Chapter 17 exercise: a "stats" command for the pocket tool.
# In your own copy, put stats_lines in books.py and the new command in
# cli.py. Here, the package is imported from the chapter folder instead.
import sys

sys.path.insert(0, "pocket-library/src")

from pocket_library.books import Book, Library  # noqa: E402


def stats_lines(library):
    """Lines describing the library: totals, and the oldest book."""
    books = library.books
    if not books:
        return ["No books yet."]
    read = sum(1 for book in books if book.read)
    oldest = books[0]
    for book in books:
        if book.year < oldest.year:
            oldest = book
    return [
        f"{len(books)} books, {read} read, {len(books) - read} to read",
        f"Oldest: {oldest.title} ({oldest.year})",
    ]


# The changes to cli.main():
#     commands.add_parser("stats", help="show totals")
#     ...
#     elif args.command == "stats":
#         for line in stats_lines(library):
#             print(line)

library = Library("stats_demo.json")
print("\n".join(stats_lines(library)))
library.add(Book("Dune", "Frank Herbert", 1965, read=True))
library.add(Book("Emma", "Jane Austen", 1815))
library.add(Book("Educated", "Tara Westover", 2018))
```

```
print("\n".join(stats_lines(library)))
```

```
No books yet.
3 books, 1 read, 2 to read
Oldest: Emma (1815)
```

Chapter 18: Decorators and Context Managers

```
import functools
from contextlib import contextmanager


def retry(times, exceptions=(Exception,)):
    """Call the function up to `times` times until it doesn't raise."""

    def decorator(func):
        @functools.wraps(func)
        def wrapper(*args, **kwargs):
            for attempt in range(1, times + 1):
                try:
                    return func(*args, **kwargs)
                except exceptions as error:
                    print(f"attempt {attempt} failed: {error}")
                    if attempt == times:
                        raise

        return wrapper

    return decorator


def count_calls(func):
    @functools.wraps(func)
    def wrapper(*args, **kwargs):
        wrapper.calls += 1
        return func(*args, **kwargs)

    wrapper.calls = 0
    return wrapper


@contextmanager
def temporary_value(mapping, key, value):
    """Set mapping[key] = value inside the with block, then restore it."""
    missing = object()
    old = mapping.get(key, missing)
    mapping[key] = value
    try:
        yield mapping
    finally:
        if old is missing:
            del mapping[key]
        else:
            mapping[key] = old
```

```
if __name__ == "__main__":
    answers = iter([ValueError("busy"), ValueError("busy"), "ok"])

    @retry(3)
    def flaky():
        result = next(answers)
        if isinstance(result, Exception):
            raise result
        return result

    print(flaky())

    @count_calls
    def hello():
        return "hi"

    hello()
    hello()
    print(hello.calls)

    settings = {"mode": "normal"}
    with temporary_value(settings, "mode", "test"):
        print(settings)
    print(settings)
```

```
attempt 1 failed: busy
attempt 2 failed: busy
ok
2
{'mode': 'test'}
{'mode': 'normal'}
```

Chapter 19: Type Hints and Static Checking

This file passes `mypy --strict` (Chapter 19).

```
from dataclasses import dataclass
from operator import attrgetter


@dataclass
class Item:
    name: str
    price: float
    quantity: int = 1


def order_total(items: list[Item], discount: float = 0.0) -> float:
    """Total price of the items, minus a discount between 0 and 1."""
    total = sum(item.price * item.quantity for item in items)
    return round(total * (1 - discount), 2)


def cheapest(items: list[Item]) -> Item | None:
```

```
    return min(items, key=attrgetter("price"), default=None)


def parse_quantities(text: str) -> dict[str, int]:
    """'tea=2, milk=1' -> {'tea': 2, 'milk': 1}"""
    result: dict[str, int] = {}
    for part in text.split(","):
        name, _, number = part.partition("=")
        result[name.strip()] = int(number)
    return result


if __name__ == "__main__":
    basket = [Item("tea", 3.5, 2), Item("milk", 1.2)]
    print(order_total(basket, 0.1))
    found = cheapest(basket)
    print(found.name if found else "empty basket")
    print(parse_quantities("tea=2, milk=1"))
```

```
7.38
milk
{'tea': 2, 'milk': 1}
```

Chapter 20: Functional Tools and Pattern Matching

In `top_n`, the key `(-score, name)` sorts by score, highest first, and then by name. `compose` uses `reduce` to combine the functions in pairs.

```
from functools import reduce


def top_n(scores, n):
    """The n highest (name, score) pairs, best first; ties by name."""
    return sorted(scores.items(), key=lambda pair: (-pair[1], pair[0]))[:n]


def compose(*functions):
    """compose(f, g, h)(x) == f(g(h(x)))"""
    return reduce(lambda f, g: lambda x: f(g(x)), functions)


def http_status(code):
    match code:
        case 200 | 201 | 204:
            return "success"
        case 301 | 302:
            return "redirect"
        case 404:
            return "not found"
        case _ if 400 <= code < 500:
            return "client error"
        case _ if 500 <= code < 600:
            return "server error"
        case _:
```

```
            return "unknown"


if __name__ == "__main__":
    print(top_n({"ana": 82, "ben": 95, "cy": 82, "dee": 70}, 3))
    shout = compose(lambda s: s + "!", str.upper, str.strip)
    print(shout("  hello "))
    for code in (200, 302, 404, 418, 503, 99):
        print(code, http_status(code))
```

```
[('ben', 95), ('ana', 82), ('cy', 82)]
HELLO!
200 success
302 redirect
404 not found
418 client error
503 server error
99 unknown
```

Chapter 21: Concurrency

```
import asyncio
from concurrent.futures import ThreadPoolExecutor


def word_counts(texts):
    """Count words in many texts with a pool of threads, in order."""
    with ThreadPoolExecutor(max_workers=4) as pool:
        return list(pool.map(lambda text: len(text.split()), texts))


async def fetch_all(names, limit):
    """Fetch every name, but never more than `limit` at the same time."""
    semaphore = asyncio.Semaphore(limit)
    running = 0
    most = 0

    async def fetch(name):
        nonlocal running, most
        async with semaphore:
            running += 1
            most = max(most, running)
            await asyncio.sleep(0.05)
            running -= 1
            return name.upper()

    results = await asyncio.gather(*(fetch(n) for n in names))
    return results, most


if __name__ == "__main__":
    print(word_counts(["one two", "three", "", "four five six"]))
    results, most = asyncio.run(
        fetch_all(["a", "b", "c", "d", "e"], limit=2)
    )
```

```
    print(results, "at most", most, "at once")
```

```
[2, 1, 0, 3]
['A', 'B', 'C', 'D', 'E'] at most 2 at once
```

Chapter 22: Performance

`bisect.bisect` finds how many cut-offs a score has reached, which is exactly the position of its grade in `"FDCBA"`.

```
import bisect


def has_duplicates(items):
    """True if any item appears twice: one pass, using a set."""
    seen = set()
    for item in items:
        if item in seen:
            return True
        seen.add(item)
    return False


def grade_for(score, cut_offs=(60, 70, 80, 90), grades="FDCBA"):
    """Find the grade with a binary search over the cut-off scores."""
    return grades[bisect.bisect(cut_offs, score)]


def pair_with_sum(numbers, target):
    """Return a pair of numbers adding up to target, or None. O(n)."""
    seen = set()
    for n in numbers:
        if target - n in seen:
            return (target - n, n)
        seen.add(n)
    return None


if __name__ == "__main__":
    print(has_duplicates([3, 1, 4, 1, 5]), has_duplicates(range(100_000)))
    print([grade_for(s) for s in (55, 60, 75, 89, 90, 100)])
    print(pair_with_sum([8, 3, 11, 5, 2], 13), pair_with_sum([1, 2], 10))
```

```
True False
['F', 'D', 'C', 'B', 'A', 'A']
(8, 5) None
```

Chapter 23: Pocket Library with SQLite

The new methods are added in a subclass, so that this file runs on its own. `":memory:"` makes a temporary database in memory.

```
# Chapter 23 exercises: two new BookStore methods, added in a subclass so
# that this file runs on its own. In your project, add them to db.py.
from pocket.db import BookStore


class BetterStore(BookStore):
    def remove(self, book_id):
        """Delete a book; KeyError if there is no such book."""
        with self.conn:
            cur = self.conn.execute(
                "DELETE FROM books WHERE id = ?", (book_id,)
            )
        if cur.rowcount == 0:
            raise KeyError(book_id)

    def books_per_author(self):
        """{author: number of books}, most books first."""
        rows = self.conn.execute(
            "SELECT author, COUNT(*) FROM books GROUP BY author "
            "ORDER BY COUNT(*) DESC, author"
        )
        return dict(rows.fetchall())


store = BetterStore(":memory:")  # a database that lives only in memory
store.add("Dune", "Frank Herbert", 1965)
store.add("Children of Dune", "Frank Herbert", 1976)
store.add("Emma", "Jane Austen", 1815)
print(store.books_per_author())
store.remove(3)
print([book.title for book in store.list()])
try:
    store.remove(3)
except KeyError as error:
    print("No book with id", error.args[0])
store.close()
```

```
{'Frank Herbert': 2, 'Jane Austen': 1}
['Dune', 'Children of Dune']
No book with id 3
```

Chapter 24: Data Analysis

```
import pandas as pd


def busiest_day(sales):
    """The weekday name with the most copies sold."""
    copies = sales.groupby(sales["date"].dt.day_name())["quantity"].sum()
    return copies.idxmax()


def best_title_per_store(sales):
    """For each store, the title with the most copies sold."""
    copies = sales.groupby(["store", "title"])["quantity"].sum()
    return (
        copies.groupby(level="store")
        .idxmax()
        .map(lambda pair: pair[1])
        .to_dict()
    )


def month_on_month_growth(sales):
    """Percentage change in revenue from each month to the next."""
    revenue = (
        (sales["quantity"] * sales["unit_price"])
        .groupby(sales["date"].dt.month)
        .sum()
    )
    return (revenue.pct_change() * 100).round(1).dropna().to_dict()


if __name__ == "__main__":
    data = pd.read_csv("sales.csv", parse_dates=["date"])
    print(busiest_day(data))
    for store, title in best_title_per_store(data).items():
        print(f"  {store}: {title}")
    print(month_on_month_growth(data))
```

```
Sunday
  Old Town: Walking the Coast
  Riverside: Walking the Coast
  Station: Walking the Coast
{2: 0.4, 3: 0.2, 4: 29.7, 5: -11.6, 6: 16.2}
```

Chapter 25: Web APIs

The new endpoints are added to the app from the chapter, and checked with `TestClient`.

```
# Solutions to the Chapter 25 exercises, checked with FastAPI's TestClient.
import warnings

# The same message pytest.ini hides for the tests (see the chapter)
warnings.filterwarnings(
    "ignore", message="Using `httpx` with `starlette.testclient`"
)

from fastapi import HTTPException  # noqa: E402
from fastapi.testclient import TestClient  # noqa: E402
from pydantic import BaseModel, Field  # noqa: E402

import books_api  # noqa: E402
from books_api import Book  # noqa: E402

app = books_api.app


# Exercise 1: mark a book as read, with a rating from 1 to 5
class Reading(BaseModel):
    rating: int = Field(ge=1, le=5)


ratings: dict[int, int] = {}


@app.put("/books/{book_id}/rating")
def rate_book(book_id: int, reading: Reading) -> dict[str, int]:
    if book_id not in books_api.books:
        raise HTTPException(
            status_code=404, detail=f"No book with id {book_id}"
        )
    ratings[book_id] = reading.rating
    return {"id": book_id, "rating": reading.rating}


# Exercise 2: statistics
@app.get("/stats")
def stats() -> dict[str, float | int | None]:
    years = [b.year for b in books_api.books.values() if b.year is not None]
    average = None
    if ratings:
        average = round(sum(ratings.values()) / len(ratings), 2)
    return {
        "books": len(books_api.books),
        "rated": len(ratings),
        "average_rating": average,
        "oldest": min(years, default=None),
    }


# Exercise 3: a client function that returns None instead of raising for 404
def find_book(client: TestClient, book_id: int) -> Book | None:
    response = client.get(f"/books/{book_id}")
    if response.status_code == 404:
        return None
    response.raise_for_status()
    return Book(**response.json())
```

```
client = TestClient(app)
for title, author, year in [
    ("Dune", "Frank Herbert", 1965),
    ("Emma", "Jane Austen", 1815),
]:
    client.post(
        "/books", json={"title": title, "author": author, "year": year}
    )
print(client.put("/books/1/rating", json={"rating": 5}).json())
print(client.put("/books/2/rating", json={"rating": 4}).json())
print(client.put("/books/2/rating", json={"rating": 9}).status_code)
print(client.put("/books/7/rating", json={"rating": 3}).status_code)
print(client.get("/stats").json())
print(find_book(client, 2))
print(find_book(client, 7))
```

```
{'id': 1, 'rating': 5}
{'id': 2, 'rating': 4}
422
404
{'books': 2, 'rated': 2, 'average_rating': 4.5, 'oldest': 1815}
title='Emma' author='Jane Austen' year=1815 id=2
None
```

Chapter 26: Automating Everyday Tasks

The log file used in the first exercise:

```
2026-10-14 08:00:01 INFO Server started
2026-10-14 08:05:12 INFO GET /books 200
2026-10-14 08:05:13 WARNING Slow request: GET /books took 1.2 s
2026-10-14 09:12:44 ERROR Database locked
2026-10-14 09:12:45 INFO GET /books 200
2026-10-14 11:30:02 ERROR Database locked
2026-10-14 13:01:55 WARNING Disk 85% full
2026-10-14 17:45:10 ERROR Could not send email: connection refused
2026-10-14 18:00:00 INFO Server stopped
```

```
# Solutions to the Chapter 26 exercises.
import re
from collections import Counter
from pathlib import Path


# Exercise 1: summarize a log file by level, and list the error messages
def log_summary(path):
    levels = Counter()
    errors = Counter()
    pattern = re.compile(r"^\S+ \S+ (INFO|WARNING|ERROR) (.*)$")
    for line in Path(path).read_text().splitlines():
        match = pattern.match(line)
        if match:
            level, text = match.groups()
```

```
            levels[level] += 1
            if level == "ERROR":
                errors[text] += 1
    return levels, errors


levels, errors = log_summary("app.log")
print(dict(levels))
for text, count in errors.most_common():
    print(f"  {count} x {text}")


# Exercise 2: sort files into folders by type, with a dry run first
FOLDERS = {
    ".pdf": "documents",
    ".txt": "documents",
    ".jpg": "pictures",
    ".xlsx": "spreadsheets",
}


def sort_by_type(folder, apply=False):
    moves = []
    for path in sorted(Path(folder).iterdir()):
        if path.is_file():
            target = (
                Path(folder)
                / FOLDERS.get(path.suffix.lower(), "other")
                / path.name
            )
            moves.append((path, target))
            if apply:
                target.parent.mkdir(exist_ok=True)
                path.rename(target)
    return moves


for old, new in sort_by_type("downloads"):
    print(f"  {old.name} -> {new.parent.name}/")
sort_by_type("downloads", apply=True)
for path in sorted(Path("downloads").rglob("*")):
    if path.is_file():
        print("  ", path.relative_to("downloads").as_posix())


# Exercise 3: the reading log as a text table, widest value sets the width
def text_table(rows):
    widths = {k: max(len(k), *(len(r[k]) for r in rows)) for k in rows[0]}
    lines = ["  ".join(k.ljust(w) for k, w in widths.items()).rstrip()]
    lines.append("  ".join("-" * w for w in widths.values()))
    for r in rows:
        lines.append(
            "  ".join(r[k].ljust(w) for k, w in widths.items()).rstrip()
        )
    return "\n".join(lines)


import csv  # noqa: E402  (imported here to keep each exercise together)

with open("reading_log.csv", newline="") as f:
```

```
    print(text_table(list(csv.DictReader(f))[:3]))
```

```
{'INFO': 4, 'WARNING': 2, 'ERROR': 3}
  2 x Database locked
  1 x Could not send email: connection refused
  Holiday Photo.JPG -> pictures/
  My Notes.TXT -> documents/
  Scan 1.PDF -> documents/
  budget 2026.xlsx -> spreadsheets/
  holiday photo (1).jpg -> pictures/
  scan 2.pdf -> documents/
   Old Photos/beach.jpg
   documents/My Notes.TXT
   documents/Scan 1.PDF
   documents/scan 2.pdf
   pictures/Holiday Photo.JPG
   pictures/holiday photo (1).jpg
   spreadsheets/budget 2026.xlsx
title     author         finished    pages  rating
--------  -------------  ----------  -----  ------
Dune      Frank Herbert  2026-01-19  612    5
Emma      Jane Austen    2026-02-27  474    4
Educated  Tara Westover  2026-04-02  334    5
```

Appendix B: Python Cheat Sheet

A quick reminder of the syntax used most often in this book. Every example here is a tested shell session; the chapter numbers show where each topic is explained.

Values, Numbers and Strings

Chapters 2, 3 and 9.

```
>>> 7 / 2, 7 // 2, 7 % 2, 2 ** 10
(3.5, 3, 1, 1024)
>>> int("42"), float("2.5"), str(99), round(2.675, 2)
(42, 2.5, '99', 2.67)
>>> name, n = "Ada", 3.14159
>>> f"{name:>5}|{n:.2f}|{1234567:,}|{0.25:.0%}"
'  Ada|3.14|1,234,567|25%'
>>> s = "  Hello, World  "
>>> s.strip().lower().replace("world", "Python")
'hello, Python'
>>> "a,b,c".split(","), "-".join(["a", "b", "c"])
(['a', 'b', 'c'], 'a-b-c')
>>> "Python"[0], "Python"[-1], "Python"[1:4], "Python"[::-1]
('P', 'n', 'yth', 'nohtyP')
>>> "py" in "python", "Python".startswith("Py"), len("Python")
(True, True, 6)
```

`round(2.675, 2)` gives 2.67, not 2.68, because 2.675 can't be stored exactly as a float; use `decimal.Decimal` for money (Chapter 11).

Lists, Tuples, Dictionaries and Sets

Chapters 6, 7 and 14.

```
>>> nums = [3, 1, 2]
>>> nums.append(4); nums.insert(0, 9); nums
[9, 3, 1, 2, 4]
>>> nums.pop(), sorted(nums), sum(nums), max(nums)
(4, [1, 2, 3, 9], 15, 9)
>>> [n * n for n in range(5) if n % 2 == 0]
[0, 4, 16]
>>> point = (3, 4)
>>> x, y = point
>>> ages = {"ana": 31, "ben": 25}
>>> ages["cy"] = 40
>>> ages.get("dee", 0), "ana" in ages, list(ages)
(0, True, ['ana', 'ben', 'cy'])
>>> for name, age in ages.items():
...     print(name, age)
ana 31
ben 25
cy 40
>>> {k: v for k, v in ages.items() if v > 30}
{'ana': 31, 'cy': 40}
>>> a, b = {1, 2, 3}, {2, 3, 4}
>>> a & b, a | b, a - b
({2, 3}, {1, 2, 3, 4}, {1})
>>> from collections import Counter
>>> Counter("banana").most_common(2)
[('a', 3), ('n', 2)]
```

Decisions, Loops, Errors and Matching

Chapters 4, 5, 10 and 20.

```
>>> score = 82
>>> if score >= 90:
...     grade = "A"
... elif score >= 80:
...     grade = "B"
... else:
...     grade = "C"
>>> grade
'B'
>>> "pass" if score >= 50 else "fail"
'pass'
>>> for i, word in enumerate(["a", "b"], start=1):
...     print(i, word)
1 a
2 b
>>> n = 0
>>> while True:
...     n += 1
...     if n == 3:
...         break
```

```
>>> n
3
>>> try:
...     int("x")
... except ValueError as error:
...     print("Error:", error)
Error: invalid literal for int() with base 10: 'x'
>>> match ["add", "Dune"]:
...     case ["add", title]:
...         print("adding", title)
...     case _:
...         print("unknown")
adding Dune
```

Functions, Classes and Generators

Chapters 8, 12, 13, 14 and 20.

```
>>> def greet(name, greeting="Hello"):
...     """Return a greeting."""
...     return f"{greeting}, {name}!"
>>> greet("Ada"), greet("Ada", greeting="Hi")
('Hello, Ada!', 'Hi, Ada!')
>>> def total(*numbers, **options):
...     return sum(numbers), options
>>> total(1, 2, 3, rounding=2)
(6, {'rounding': 2})
>>> sorted(["kiwi", "fig", "apple"], key=lambda w: len(w))
['fig', 'kiwi', 'apple']
>>> class Counter:
...     def __init__(self, start=0):
...         self.count = start
...     def click(self):
...         self.count += 1
...         return self.count
>>> c = Counter()
>>> c.click(), c.click()
(1, 2)
>>> from dataclasses import dataclass
>>> @dataclass
... class Book:
...     title: str
...     year: int
>>> Book("Dune", 1965)
Book(title='Dune', year=1965)
>>> def squares(limit):
...     for n in range(limit):
...         yield n * n
>>> list(squares(4))
[0, 1, 4, 9]
```

Files

Chapter 10. Always give `encoding="utf-8"`.

Task	Code
Read a whole text file	`text = Path("notes.txt").read_text(encoding="utf-8")`
Write a whole text file	`Path("notes.txt").write_text(text, encoding="utf-8")`
Read line by line	`with open("notes.txt", encoding="utf-8") as f:` then `for line in f:`
Add to the end	`with open("log.txt", "a", encoding="utf-8") as f:` then `f.write("...\n")`
Read a CSV file	`rows = list(csv.DictReader(f))`, with `open(..., newline="")`
Read and write JSON	`json.loads(text)` and `json.dumps(data, indent=2)`
List a folder	`for path in Path("folder").iterdir():`
Join paths	`Path("folder") / "file.txt"`

Command Line

Task	Command
Run a program	`python program.py` (on macOS, `python3`)
Start the shell	`python`, and `exit()` to leave
Create a virtual environment	`python -m venv .venv`
Activate it	`.venv\Scripts\activate` (Windows) or `source .venv/bin/activate`
Install a package	`python -m pip install name`
Install from a list	`python -m pip install -r requirements.txt`
Run tests	`python -m pytest`
Check types	`python -m mypy program.py`
Run a package	`python -m package_name`

Task	Command
Stop a running program	Ctrl+C

Appendix C: Common Errors and How to Fix Them

Every programmer sees these errors every day. Read the last line of the traceback first: it names the error and gives the reason. Then look at the frames above it, from the bottom, for the first line in your own code (Chapter 16).

The Messages

These are the exact messages, tested on Python 3.12 to 3.15. Your tracebacks will also show the file and line:

```
>>> print(total)
Traceback (most recent call last):
  ...
NameError: name 'total' is not defined
>>> "Age: " + 30
Traceback (most recent call last):
  ...
TypeError: can only concatenate str (not "int") to str
>>> int("3.5")
Traceback (most recent call last):
  ...
ValueError: invalid literal for int() with base 10: '3.5'
>>> {"a": 1}["b"]
Traceback (most recent call last):
  ...
KeyError: 'b'
>>> [1, 2, 3][3]
Traceback (most recent call last):
  ...
IndexError: list index out of range
>>> "text".append("!")
Traceback (most recent call last):
  ...
```

```
AttributeError: 'str' object has no attribute 'append'
>>> 10 / 0
Traceback (most recent call last):
  ...
ZeroDivisionError: division by zero
>>> open("no_such_file.txt")
Traceback (most recent call last):
  ...
FileNotFoundError: [Errno 2] No such file or directory: 'no_such_file.txt'
>>> import no_such_module
Traceback (most recent call last):
  ...
ModuleNotFoundError: No module named 'no_such_module'
>>> len(42)
Traceback (most recent call last):
  ...
TypeError: object of type 'int' has no len()
>>> def add(a, b):
...     return a + b
>>> add(1)
Traceback (most recent call last):
  ...
TypeError: add() missing 1 required positional argument: 'b'
>>> count = 0
>>> def increase():
...     count += 1
>>> increase()
Traceback (most recent call last):
  ...
UnboundLocalError: cannot access local variable 'count' where it is not
↪ associated with a value
>>> None.title
Traceback (most recent call last):
  ...
AttributeError: 'NoneType' object has no attribute 'title'
```

What They Mean

Error	Usual cause	Fix
`SyntaxError`	A missing quote, bracket or colon; a misspelled keyword	Check the line shown and the line before it (Chapter 1)
`IndentationError`	A block not indented after a colon, or mixed tabs and spaces	Indent blocks by four spaces; let your editor do it (Chapter 4)
`NameError`	A typo in a name, or using a variable before assigning it	Check the spelling; Python often suggests the right name (Chapter 2)

Error	Usual cause	Fix
`TypeError`	An operation on the wrong type, such as `str + int`; a function called with the wrong number of arguments	Convert with `str()`, `int()` or `float()`, or use an f-string; check the call (Chapters 2 and 8)
`ValueError`	The right type with a wrong value, such as `int("twelve")`	Check input with `try` and `except`, and ask again (Chapter 10)
`KeyError`	A dictionary key that isn't there	Use `in` or `get(key, default)` (Chapter 7)
`IndexError`	A list index past the end; the last index is `len(items) - 1`	Check the length; loop over items rather than indexes (Chapter 6)
`AttributeError`	A method that the type doesn't have, or a value that's unexpectedly `None`	Check the type with `type(x)`; find where the `None` came from (Chapters 12 and 19)
`ZeroDivisionError`	Dividing by zero, often an average of an empty list	Check for zero or empty input first (Chapter 5)
`FileNotFoundError`	A wrong file name, or running the program from a different folder	Check the name and the current folder (Chapter 10)
`ModuleNotFoundError`	A package not installed, or installed in a different environment	Activate the right virtual environment and `python -m pip install` it (Chapter 17)
`UnboundLocalError`	Assigning to a global variable inside a function	Pass the value in as a parameter and `return` the result (Chapter 8)
`RecursionError`	A recursive function without a base case that stops it	Add or fix the base case (Chapter 8)

Mistakes Without Error Messages

These don't stop your program; they make it give wrong answers.

- **Adding input without converting it.** `input` returns text, so `"12" + "30"` is `"1230"`. Convert with `int` or `float` (Chapter 3).

- `==` **where you meant** `=`**.** A line such as `total == 0` compares and throws the answer away; `total` doesn't change. (The opposite mistake, `=` in an `if` condition, is caught with a `SyntaxError`.) (Chapter 4)
- **Off-by-one ranges.** `range(1, 10)` stops at 9. `range(len(items))` starts at 0 (Chapters 5 and 16).
- **Comparing floats exactly.** `0.1 + 0.2 == 0.3` is `False`. Round, use `math.isclose`, or use `Decimal` for money (Chapters 2 and 11).
- **Two names for one list.** `b = a` doesn't copy a list; changing `b` changes `a`. Use `a.copy()` (Chapter 6).
- **A list as a default value**, as in `def f(items=[])`: the same list is shared by every call. Use `None` and create the list inside the function, or `field(default_factory=list)` in a dataclass (Chapters 12 and 13).
- **Changing a list while looping over it.** Items get skipped. Loop over a copy, or build a new list (Chapter 6).
- `return` **inside a loop by accident.** The function stops after the first item. Check the indentation of `return`.
- **A generator used twice.** The second time it's empty. Make a list if you need the values more than once (Chapter 14).

Installation and Environment Problems

- `python` **is not recognized** (Windows) or **command not found**: Python isn't installed, or the terminal was opened before installing. Open a new terminal; on macOS and Linux, try `python3` (Chapter 1).
- `externally-managed-environment` when installing a package: your system protects its own Python. Create and activate a virtual environment first (Chapter 17).
- **A package is installed, but** `import` **fails**: it was installed for a different Python. Always install with `python -m pip`, inside the activated environment.
- `UnicodeDecodeError` when reading a file: the file isn't in the expected encoding. Pass `encoding="utf-8"` when reading and writing (Chapter 10).

Appendix D: Glossary

Argument. A value passed to a function when it's called, as in `len("hi")`. See *parameter*.

Assertion. A check with `assert` that stops the program, or fails a test, if a condition is false.

Attribute. A value that belongs to an object, reached with a dot: `book.title`.

Big-O notation. A way of describing how the work a piece of code does grows with the size of its input: O(1), O(n), O(n²).

Boolean. A value that's either `True` or `False`; type `bool`.

Class. A description of a kind of object: its attributes and methods. Calling a class creates an instance.

Closure. A function that remembers variables from the function it was defined in.

Comprehension. A compact way to build a list, dictionary or set: `[x * 2 for x in items]`.

Context manager. An object used with `with`, which runs set-up code at the start of a block and clean-up code at the end.

Coroutine. A function defined with `async def`, which can pause at `await` while other tasks run.

CSV. Comma-separated values: a simple text format for tables.

Dataclass. A class whose `__init__`, `__repr__` and `__eq__` are written by the `@dataclass` decorator.

Decorator. A function that takes a function or class and returns a modified one, applied with `@name` above a definition.

Dictionary. A collection that maps unique keys to values; type `dict`.

Docstring. A string at the start of a function, class or module that describes it.

Dry run. Running a script so that it shows what it would do without doing it.

Exception. An error that interrupts a program, which can be handled with `try` and `except`.

Expression. A piece of code that produces a value, such as `2 + 3` or `name.upper()`.

f-string. A string with `f` before the quote, in which expressions in braces are replaced by their values.

Fixture. In pytest, a function that prepares something a test needs.

Float. A number with a decimal point; type `float`.

Function. A named, reusable block of code, defined with `def`.

Generator. A function that uses `yield` to produce values one at a time, pausing in between.

GIL. The global interpreter lock, which lets only one thread run Python code at a time in the standard interpreter.

Hash. A fixed-length fingerprint calculated from data. Sets and dictionaries use hashes to find items quickly.

HTTP. The protocol that browsers and web APIs use to send requests and responses.

Immutable. Unchangeable after creation, like strings, tuples and frozen dataclasses.

Import. Making a module's contents available to your program.

Index. The position of an item in a sequence, starting at 0.

Inheritance. Defining a class as a specialized version of another, reusing and overriding its methods.

Instance. One object made from a class.

Integer. A whole number; type `int`.

Iterable. Anything a `for` loop can go through, such as a list, string, dictionary, file or generator.

Iterator. An object that hands out values one at a time when `next` is called on it.

JSON. JavaScript Object Notation: a text format for nested data, used by web APIs and settings files.

Lambda. A small function without a name, written as a single expression.

List. An ordered, changeable collection of values; type `list`.

Logging. Recording what a program does, at levels from DEBUG to CRITICAL, with the `logging` module.

Method. A function that belongs to an object, called with a dot: `text.upper()`.

Mocking. Replacing part of a program with a fake during a test.

Module. A Python file whose contents can be imported.

None. Python's value meaning "nothing" or "no value".

Object. Any value in Python. Every object has a type, and may have attributes and methods.

Package. A folder of modules that's imported as a unit; also, a project that can be installed with pip.

Parameter. A name in a function's definition that receives an argument: in `def greet(name)`, `name` is a parameter.

pip. The standard tool for installing packages from PyPI.

Property. A method that's used like an attribute, defined with `@property`.

PyPI. The Python Package Index at pypi.org, where packages are published.

Recursion. A function calling itself, with a base case that stops it.

Regular expression. A pattern for matching text, used with the `re` module.

Set. An unordered collection of unique items; type `set`.

Slice. Part of a sequence, taken with `[start:stop:step]`.

SQL. Structured Query Language, used to create, read and change data in databases.

Standard library. The modules that come with every Python installation.

String. A piece of text; type `str`.

Test. Code that checks that other code works as expected.

Thread. A sequence of instructions that can run alongside others in the same program.

Traceback. The report Python prints when an exception isn't handled, showing the chain of calls that led to it.

Tuple. An ordered, unchangeable collection of values; type `tuple`.

Type hint. An annotation that states the intended type of a variable, parameter or return value.

Variable. A name that refers to a value.

Virtual environment. A folder with its own Python and installed packages, used to keep each project's packages separate.

Appendix E: Resources

These official sources were checked in October 2026. Web addresses change, so if one doesn't work, search for the project's name.

Python

- **Python downloads:** python.org/downloads. Installers for Windows (the Python install manager), macOS and other systems.
- **Python documentation:** docs.python.org. The official tutorial, the library reference for every standard module, and "What's New" for each version.
- **Python Enhancement Proposals:** peps.python.org. The design documents behind new features, such as PEP 8, the style guide for Python code.
- **Python Packaging User Guide:** packaging.python.org. Virtual environments, `pyproject.toml`, building wheels and publishing to PyPI.
- **The Python Package Index:** pypi.org.
- **Python discussion forum:** discuss.python.org.

Tools Used in This Book

- **pytest:** docs.pytest.org.
- **mypy:** mypy.readthedocs.io.
- **ruff** (formatting and linting) and **uv** (environments and packages): docs.astral.sh.
- **pandas:** pandas.pydata.org.

- **matplotlib:** matplotlib.org.
- **FastAPI:** fastapi.tiangolo.com.
- **httpx:** python-httpx.org.
- **SQLite:** sqlite.org. The SQL language understood by the `sqlite3` module.
- **Visual Studio Code:** code.visualstudio.com, with its Python extension.

The Book's Code

All the programs, data files, exercise solutions and tests from this book are in its code repository, **github.com/aadhavanshines/python-programming-hands-on**, arranged by chapter, together with `requirements.txt`, which lists the package versions used for testing. Each listing was checked on Python 3.12, 3.13, 3.14 and 3.15, and the repository repeats those checks automatically after every change; its README explains how to run them on your own computer. If a newer version of Python or of a package changes an output, the change is usually small, such as a reworded error message; the ideas stay the same.

About the Author

Aadhavan Muthurengan is an automation and artificial intelligence professional with more than 15 years of experience in the automotive industry.

Throughout a career focused on product development and validation, Aadhavan has designed and built a wide range of in-house tools that automate complex engineering workflows, speed up testing, and help teams deliver reliable, high-quality results. Many of those tools were written in Python.

That experience shaped this book. Real tools have to work every time, so every program in these pages was run and checked on four versions of Python, and every exercise has a tested solution. The habits that make engineering tools trustworthy (small steps, clear code, tests, and checking results instead of assuming them) are the habits this book teaches from the first chapter.

Aadhavan is also the author of *Prompt Engineering Mastery*, a beginner-to-advanced guide to getting better results from AI; *Vibe Coding for Beginners*, a guide to building real software with AI coding tools; and *Building AI Agents with Claude*, a guide to agents that use tools, remember, and get real work done.

Thank you for reading *Python Programming Hands-On*. I hope it gives you the confidence to build the programs you've been imagining.